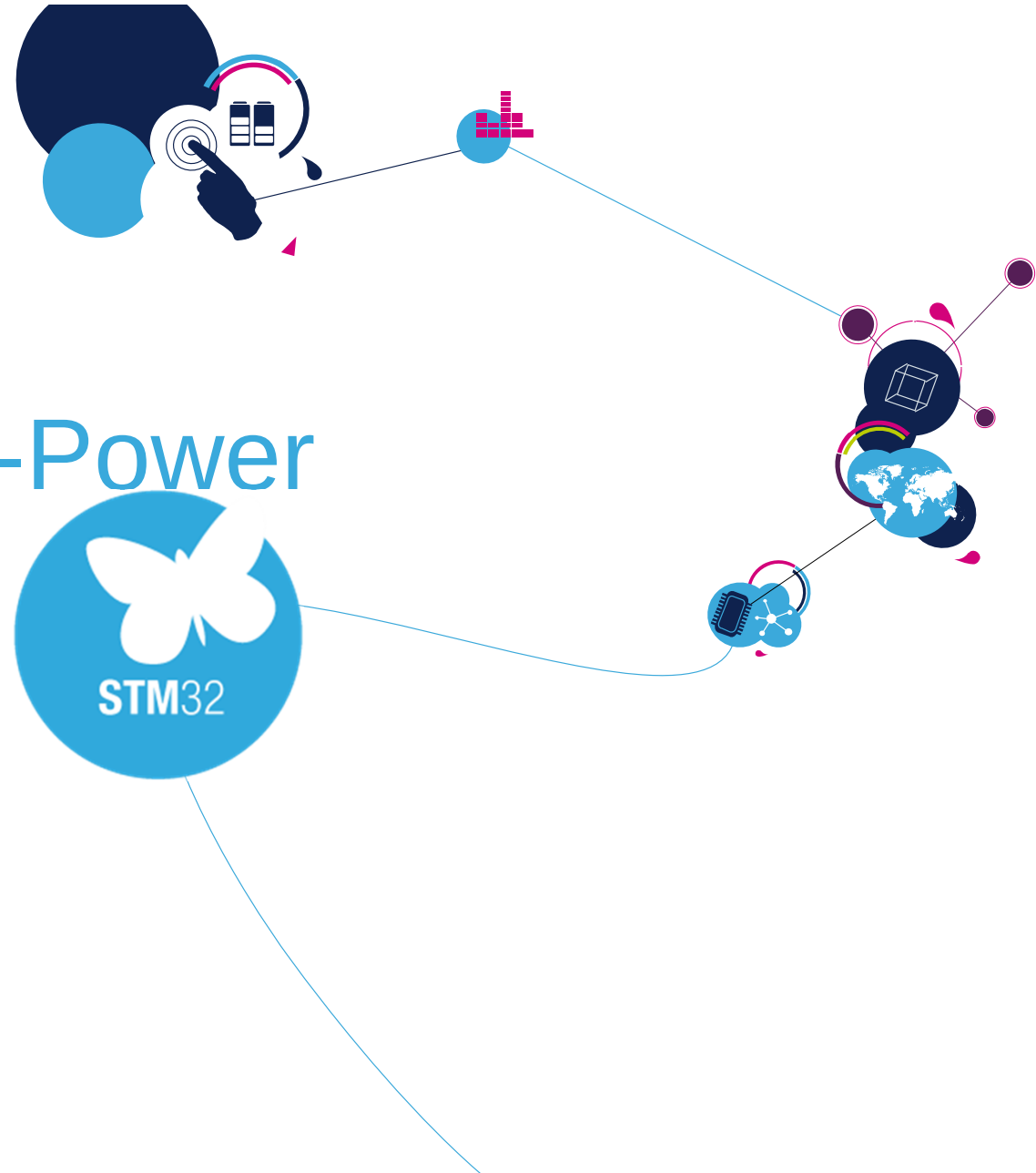


STM32 Ultra Low-Power

STM32 Euro 2017



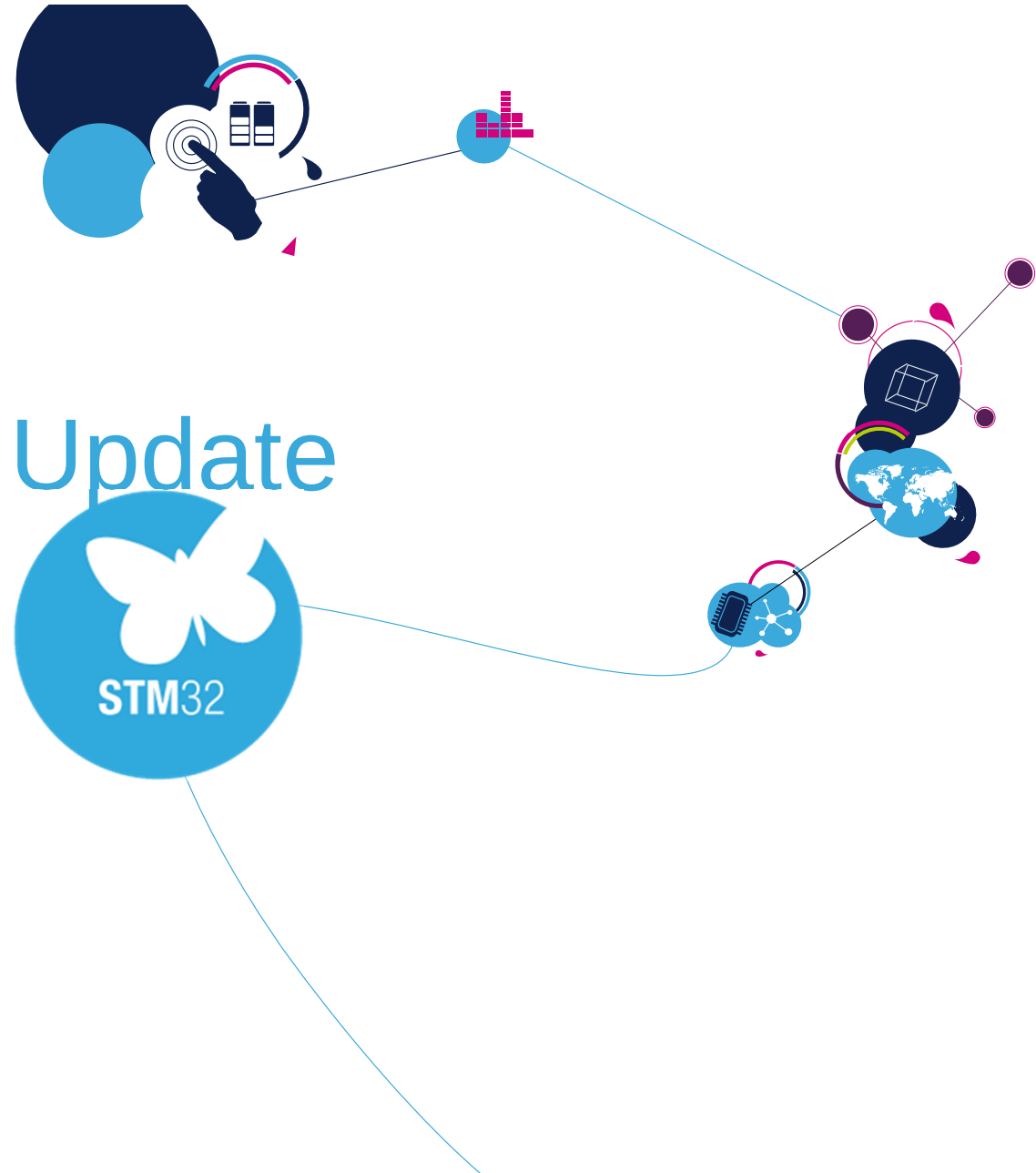
STM32 Ultra Low-Power

Agenda – 90mins

- 10 mins – Intro & Marketing Update
 - STM32L4 marketing intro / positioning
- 70 mins - Practical hands-on based on NUCLEO-L452RE-P
 1. RUN mode and external SMPS – example is applicable for all kind of low-power applications
 2. STOP mode and LPTIM – example is applicable for metering applications (water meter)
 3. STANDBY mode and RTC – example is applicable for devices with periodic activity (sensor device or data logger)
 4. Extra material to be done individually: SLEEP mode and BAM – example is applicable for always-on devices (fitness tracker)
- 10 mins – Conclusions, Q&A

Intro & Marketing Update

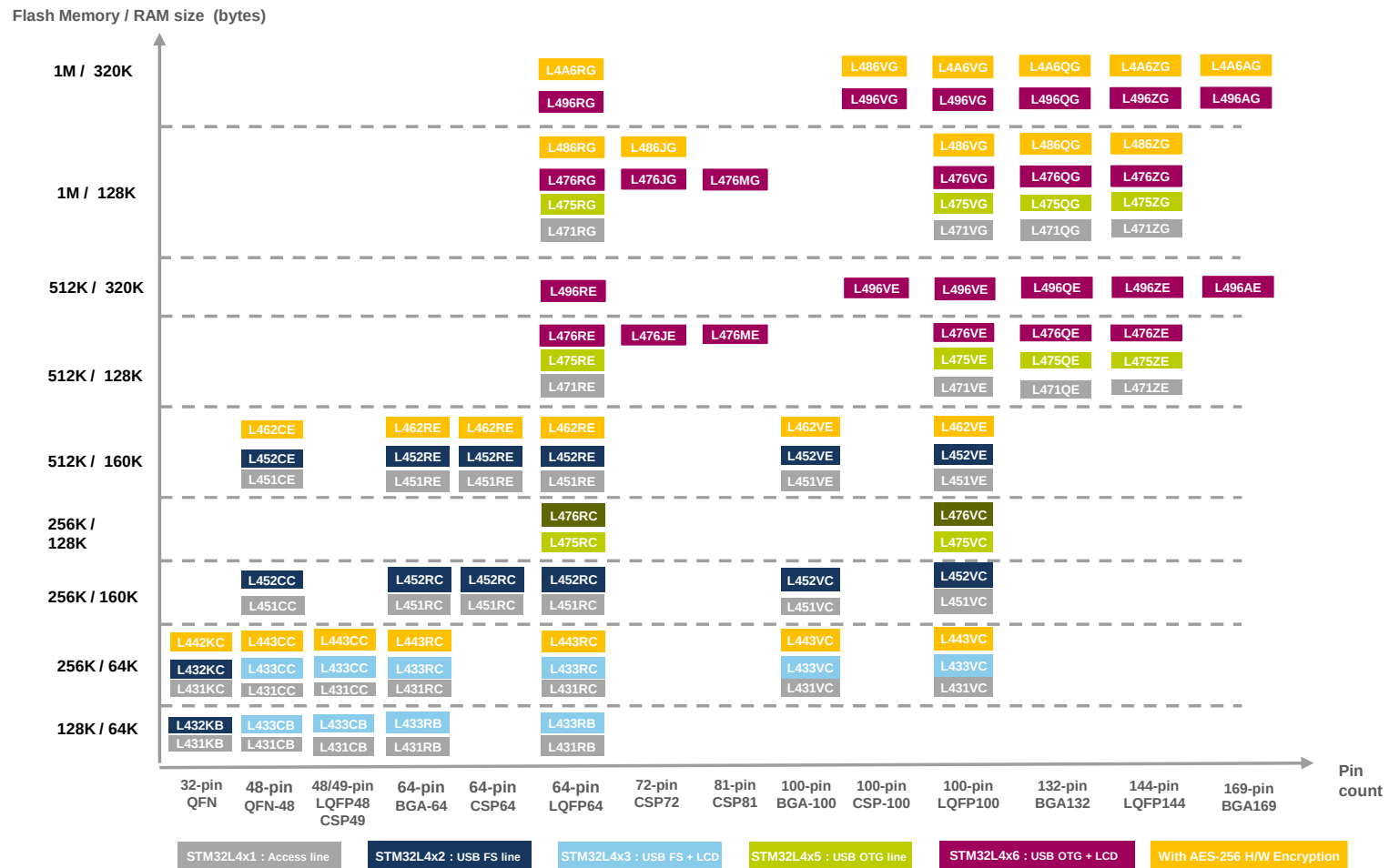
STM32 Euro 2017



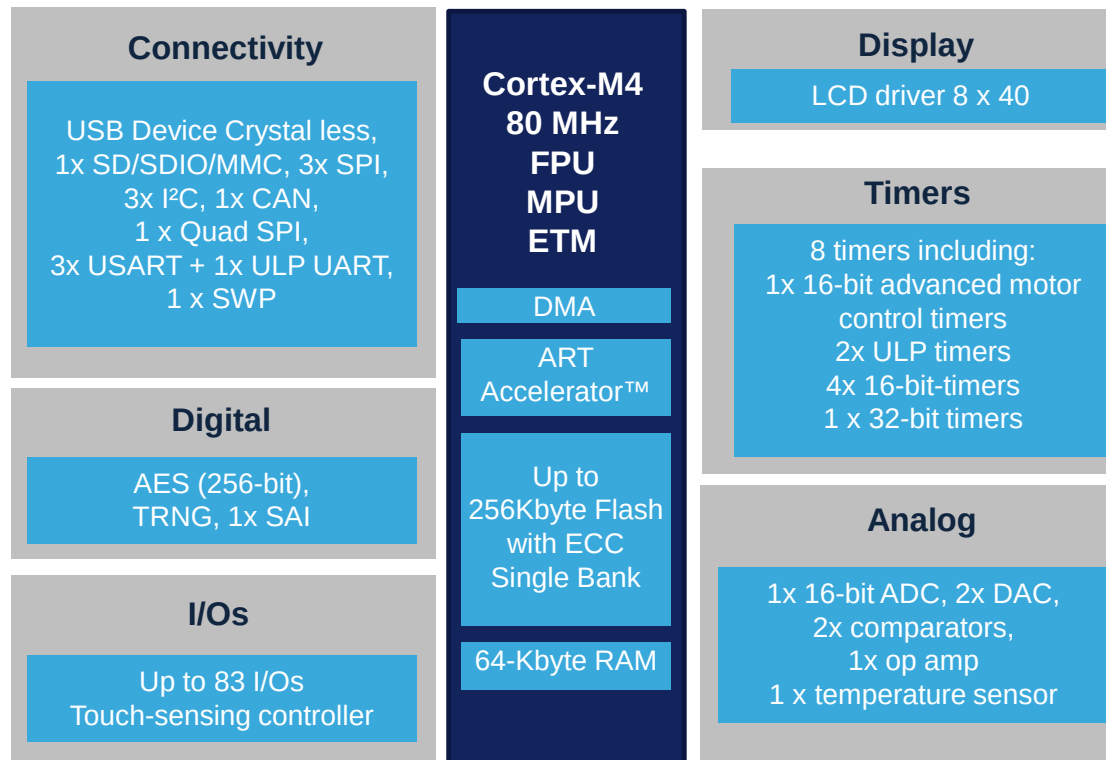
STM32L4 Road map – Now and next silicons

Unit parameters	STM32L47x	STM32L43x	STM32L49x	STM32L45x	STM32L412
Core, frequency	ARM Cortex-M4, 80 MHz, MPU, FPU, ART				
Flash (Up to) SRAM (Up to)	1MB (dual bank) 128KB	256KB (1x bank) 64KB	1MB (dual bank) 320KB	512K (1x bank) 160KB	128KB (1x bank) 40KB
FSMC	Yes	-	Yes	No	-
12-bit ADC SAR	3x 12-bit @ 5 MSPS	1x 12-bit @ 5 MSPS	3x 12-bit @ 5 MSPS	1x 12-bit @ 5 MSPS	1x 12-bit @ 5 MSPS
Comparator	2	2	2	2	1
Op amp	2	1	2	1	1
12-bit DAC	2	2	2	1	-
SAI	2	1	2	1	-
USB	OTG FS	Device Xtal-less	OTG Xtal-less	Device Xtal-less	Device Xtal-less
Sigma delta interface	8ch / 4 filters	-	8ch / 4 filters	4ch / 2 filters	-
USART	6	4	6	5	4
LCD	8x40/4x44	8x40/4x44	8x40/4x44	-	-
Camera Interface	-	-	Yes	-	-
Chrome ART	-	-	Yes	-	-
Other digital (except timers)	1x SDIO, 1x Quad SPI, 3 x SPI, 3 x I2C, 1x CAN, Charge-Transfer Touch sense, AES 256, TRNG, SWPMI	1x SDIO, 1x Quad SPI, 3x SPI, 3x I2C, 1x CAN, Charge-Transfer Touch sense, AES 256, TRNG, SWPMI	1x SDIO, 1x Quad SPI, 3x SPI, 4x I2C, 2x CAN, Charge-Transfer Touch sense, AES 256, TRNG, SWPMI, SHA-2	1x SDIO, 1x Quad SPI, 3x SPI, 4x I2C, Charge-Transfer Touch sense, AES 256, TRNG	1x Quad SPI, 2x SPI, 3x I2C, Charge-Transfer Touch sense, AES 256, TRNG
Package (pin number)	64/72/81/100/132/144	32/48/49/64/100	64/100/132/144/169	48/64/100	32/36/48/64
Schedule	Mass Prod	Mass Prod	Mass Prod	Mass Prod	Samples Jan 2018 Prod Sept 2018

STM32L4 portfolio



STM32L43x



**Package size down
to 3.1 x 3.1 mm**

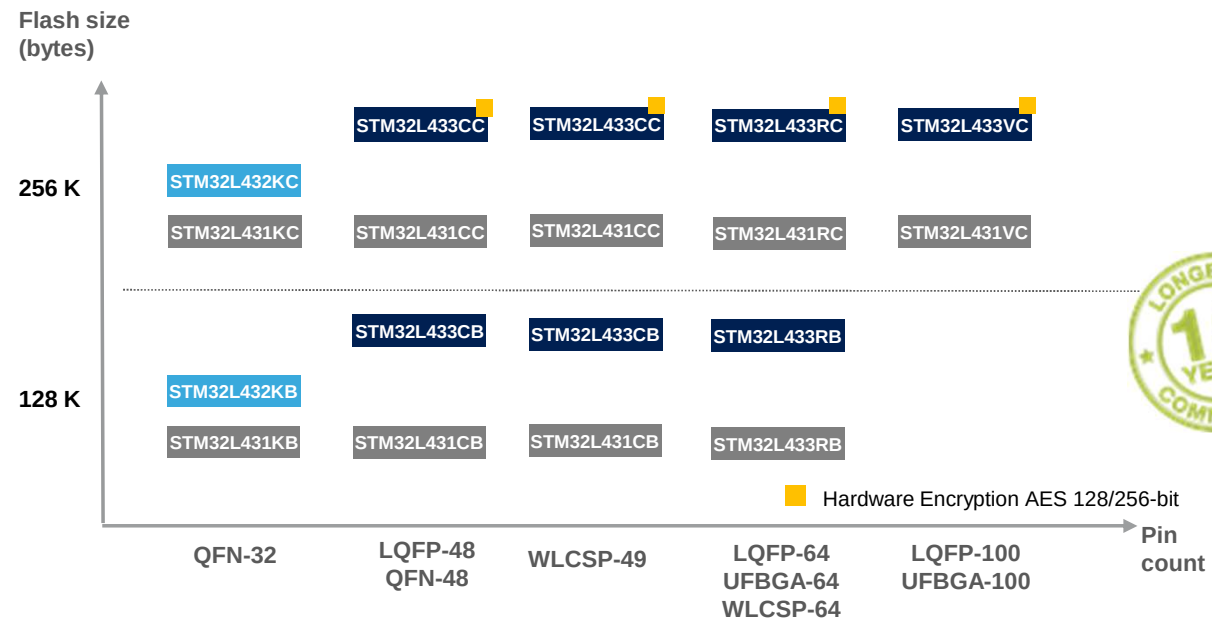


STM32L43x series

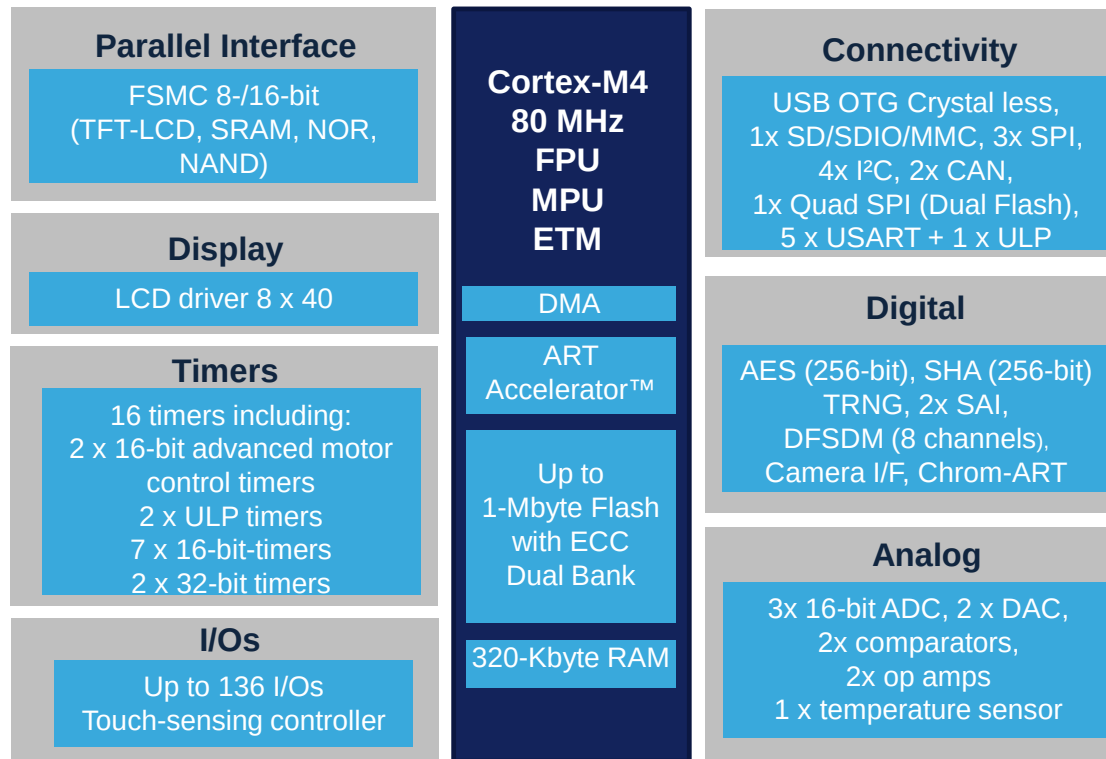
Cortex®-M4 (DSP + FPU) – 80 MHz	• ART Accelerator™ • USART, SPI, I²C • Quad SPI • 16 and 32-bit timers • SAI + audio PLL • SWP • 1x CAN • 1x 12-bit DAC • Temperature sensor • Low voltage 1.71 to 3.6 V • V_{BAT} Mode • Unique ID • Capacitive Touch sensing	Product line	Flash (KB)	RAM (KB)	Memory I/F	1 x Op amp	2 x Comp	16- bit ADC (5 Msps)	USB2.0 Device	Segment LCD Driver	AES 128/256-bit
		STM32L431 Access	128 to 256	64	SDIO	•	•	1			
		STM32L432 USB	128 to 256	64	SDIO	•	•	1	•		
		STM32L433 USB Device & LCD	128 to 256	64	SDIO	•	•	1	•	Up to 8x40	
		STM32L443 USB OTG & LCD & AES	256	64	SDIO	•	•	1	•	Up to 8x40	•

STM32L43x – portfolio

CortexTM-M4 @ 80MHz



STM32L49x



WLCSP100
4.62 x 4.14

STM32L4

Keeps moving on with

STM32L4 +

STM32L4+

STM32L4 scaling:

- Upgrade L4 performance
- Unique object reaching best-in-class ULP and performance



Main targets for STM32L4+

- Stretch STM32L4 portfolio by improving performance (horsepower)
 - Keep the ULP leadership (unique position) and attraction in the market
- Migration path for :
 - High performance customers who need better power consumption
 - Low-power customers who need more horsepower, memory, or peripherals



- Build a family (Flash from 128k to 2M) at a competitive cost



- Key targeted applications:

- Consumer market: audio accessories, wearables, fitness trackers, games and toys
- Industrial market: Gateway, alarm, sensors, POS, meter
- Home automation: high end thermostat, In Home display, camera



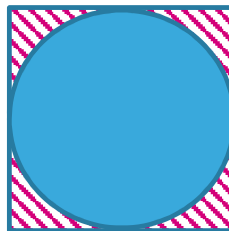
Main modifications versus STM32L4

- Increase frequency up to **120 MHz** while keeping state of the art Ultra-low power figures.
- Increase SRAM size up to **640KB**
- Add **MIPI DSI**
- Update architecture to optimize Graphics/ low power / CPU load:
Chrom- GRC allowing up to 20% of graphic resources optimization (RAM)
- Updated IPs including 2 x **Octo SPI** supporting Flash and SRAM

Chrom-GRC

- Graphic Ressources Cutter for non square displays
- No modification nor special management at SW level

➔ Saving up to 20% of RAM needs



Saved Memory

- For **360x360 round display**
 - @16bpp ~**205kBytes** (vs.253kBytes)
 - @24bpp ~**307kBytes** (vs.380kBytes)
- For **400x400 round display**
 - @16bpp: **250kBytes** (vs.312kBytes)
 - @24bpp: **372kBytes** (vs.469kBytes)

STM32L4 and STM32L4+ - comparison

Unit parameters	STM32L47x 1M/128k SRAM	STM32L49x 1M/320k SRAM	STM32L4Rx 2M/640k SRAM
Core, frequency	ARM Cortex-M4, 80 MHz, MPU, FPU, ART		ARM Cortex-M4, 120 MHz , MPU, FPU, ART
Flash (Up to)	1MB (dual bank)	1MB (dual bank)	2MB (dual bank)
SRAM (Up to)	128 KB	320 KB	640KB (64 KB backup + 384KB + 192 KB)
FMC	Yes	Yes	Yes
12-bit ADC SAR	3x 12-bit 5 MSPS	3x 12-bit 5 MSPS	1x 12-bit 5 MSPS
Op amp	2	2	2
SAI	2	2	2
USB	OTG FS	OTG FS Crystal less	OTG FS crystal less
Sigma delta interface	8ch / 4 filters	8ch / 4 filters	8ch / 4 filters
USART	6	6	6
I2C	3	4	4
LCD	8x40/4x44	8x40/4x44	-
CAN	1	2	1
QPI	Single Flash	Double Flash	2 x Octo SPI supporting Flash and RAM
SHA-256	-	1	1
Chrome ART	-	1	1 + Chrom GRC (rcircular display)
TFT Controller	-	-	1 x with MIPI DSI
Camera Interface	-	1	1
Other digital (except timers)	2x Comparator, 2x 12-bit DAC, 1x SDIO, 3 x SPI, Charge-Transfer Touch sense, AES 256, TRNG, SWPMI		2x Comparator, 2x 12-bit DAC, 1x SDIO, 3 x SPI, Charge-Transfer Touch sense, AES 256, TRNG
Package (pin number)	144/132/100/72/64	169/144/132/100/64	169/144 /132/100

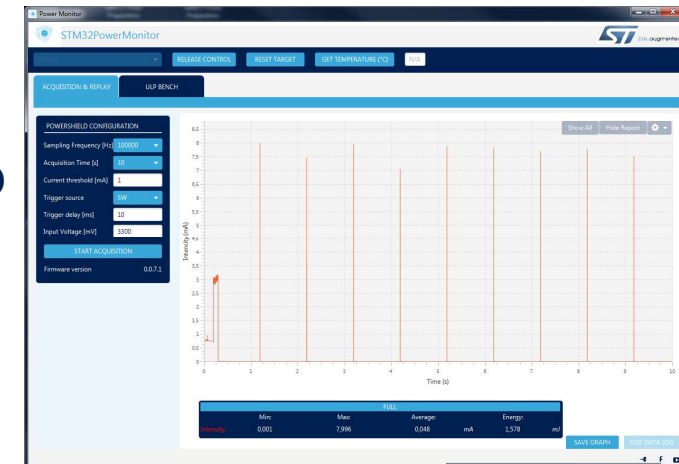
Introducing the Power shield



- Reference : [X-NUCLEO-LPM01A](#)
- Estimated price (RRP) 80\$
- Compatible with all Nucleo form factors (32, 64, 144)
- User-friendly graphical monitor tool: CubeMonitor-Power

Power shield & monitor tool...

- Measures power data of any target board with high sensitivity:
 - Power the board to test from 3.3V down to 1.8V
 - Measure static current from 500pA up to 200mA with 2% precision
 - Measure dynamic current from 100nA up to 50mA with 2% precision
 - Dynamic acquisition done at 750 Ksamples/s
- Easy to use: can do measurement autonomously & display on LCD
- Intuitive graphical analysis: can send the measurement to a PC through USB for analysis (zoom, measurement reports, remote control)



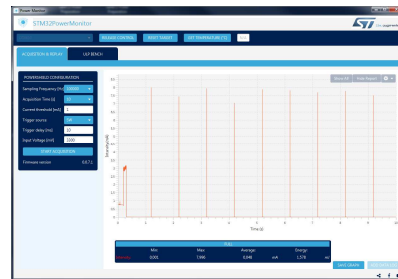
...adopted by EEMBC

- Autonomous ULPBench score computation
 - From 3.3V down to 1.8V power supply
 - Support for new Peripheral profile
- Intuitive user experience:
 - New SW Tool architecture with separate flexible graphical interface (JavaFX)
 - Modern look and feel



Leading ultra-low-power analysis

Benchmark reference



March'17

June'17

October'17

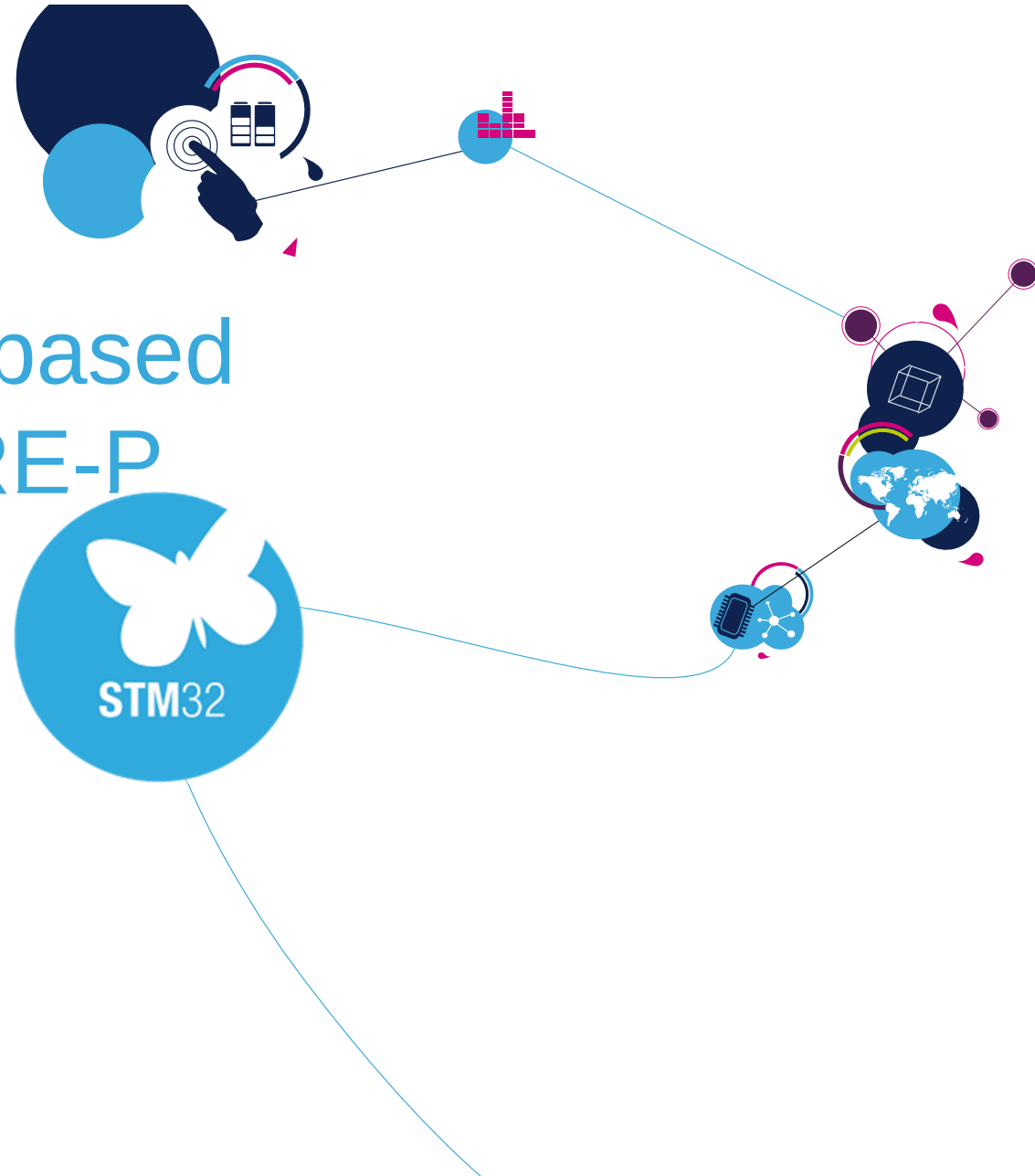
Rev.A samples
CubeMonitor release 1

Rev.B samples
CubeMonitor release 2

Launch
CubeMonitor release 3

Practical hands-on based on NUCLEO-L452RE-P

STM32 Euro 2017



Scope of this session

- Goal

- To become familiar with STM32L4 and its selected low power features

- Method

- Development of low-power applications step by step from scratch

- Steps

- STM32CubeMX

- Creating of a new project
 - Enabling and configuring of peripherals
 - Generating Keil project with code

- Keil IDE

- Adding user's source code into main.c (copy-paste from slides)
 - Compiling the application and flashing the memory of MCU

- Observing application's behavior and measuring the current consumption

Software tools

- **STM32CubeMX V4.23** – graphical code and project generator for STM32 MCUs
- **STM32L4 Cube V1.10** – software package containing peripheral libraries (HAL, LL) compatible with STM32CubeMX
- **ST-Link driver** – PC OS driver for programming/debugging probe
- **Keil MDK-ARM V5.x**– IDE for Cortex-M based MCUs
- **Keil STM32L4 pack** – STM32L4 device package for Keil MDK-ARM IDE

Hardware tools

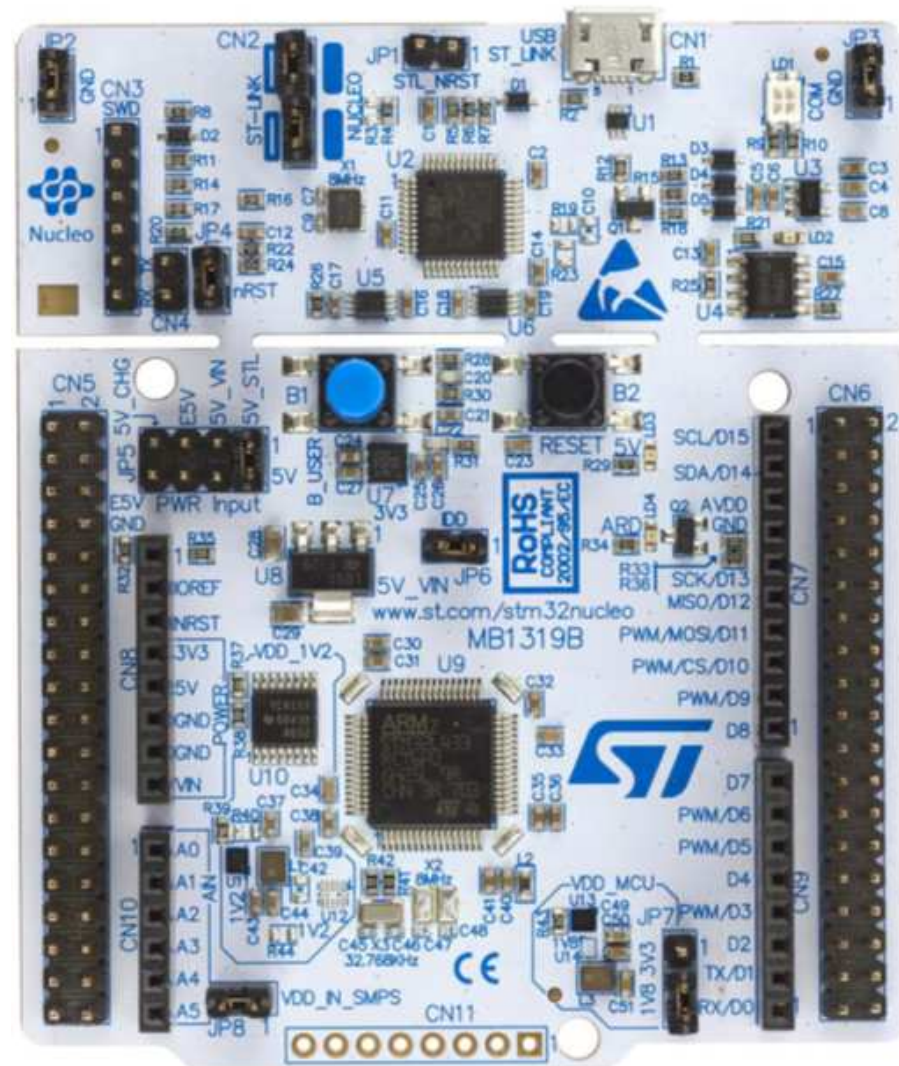
- STM32Nucleo kit with external SMPS support

- ST-Link board

- Programming/debugging probe
 - SWD interface to MCU
 - Micro USB interface to PC
 - Possibility to access MCU on the board or external MCU
 - LED to indicate status

- MCU board

- STM32L4 with external SMPS support to optimize current consumption
 - Two SMPS devices: for VDD and for VDD12
 - One analog switch
 - User LED (PB13)
 - User button (PC13)



STM32L452RET6(P) Specification

■ Key features:

■ Low-power

- Low-power manufacturing process
- Multiple low-power modes
- Low-power peripherals

■ Performance

- Cortex-M4 core
- High frequency of system clock
- ART Accelerator

■ Rich set of resources

- Memory
- Connectivity
- Timers
- Analog

Connectivity

USB Device Crystal less,
1x SD/SDIO/MMC, 3x SPI,
4x I²C, 1x CAN,
1 x Quad SPI (Dual Flash),
4x USART + 1x ULP UART,

Cortex-M4
80 MHz
FPU
MPU
ETM

DMA

ART
Accelerator™

Up to
512Kbyte Flash
with ECC
Single Bank

160-Kbyte RAM

Timers

8 timers including:
1x 16-bit advanced motor
control timers
2x ULP timers
4x 16-bit-timers
1 x 32-bit timers

Digital

AES (256-bit),
TRNG, 1x SAI,
DFSDM (4 channels)

Analog

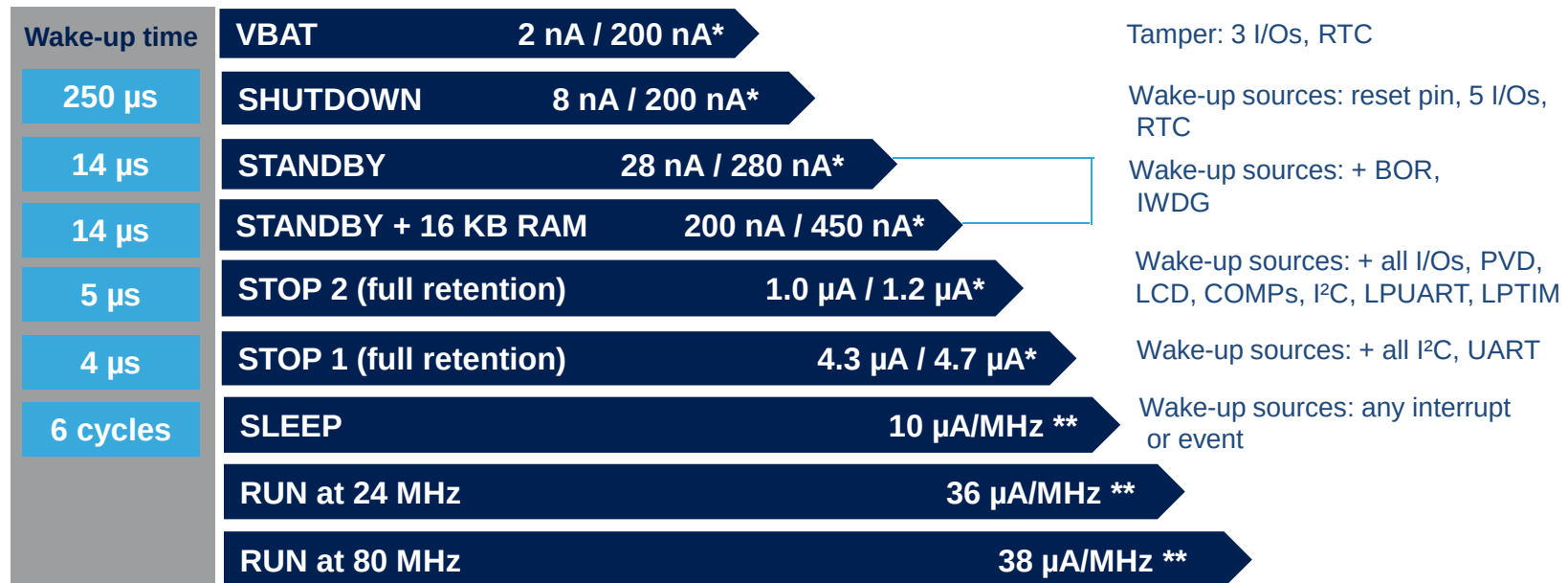
1x 16-bit ADC, 1x DAC,
2x comparators,
1x op amp
1 x temperature sensor

I/Os

Up to 83 I/Os
Touch-sensing controller

Ultra-low-power modes

Best power consumption numbers with full flexibility



Note : * without RTC / with RTC
 ** with external SMPS

ULPBENCH™ 347
 An EEMBC Benchmark

Low-power hands-on

RUN mode and external SMPS



SMPS overview

- Improves significantly MCU's power consumption by supplying some of MCU's resources from external SMPS (Switched-Mode Power Supply) through VDD12 pins
- Flexible design and wide range of existing SMPSs can be used (DC/DC with switch)

- STM32L4 devices with SMPS support are marked with ,P' suffix

Example:

Device family

STM32 = ARM[®] based 32-bit microcontroller

STM32

L

496

V

G

T

6

P

TR

SMPS Option

- Applications, which can benefit from combination of RUN mode and SMPS
 - All kind of applications, as in every application MCU uses RUN mode

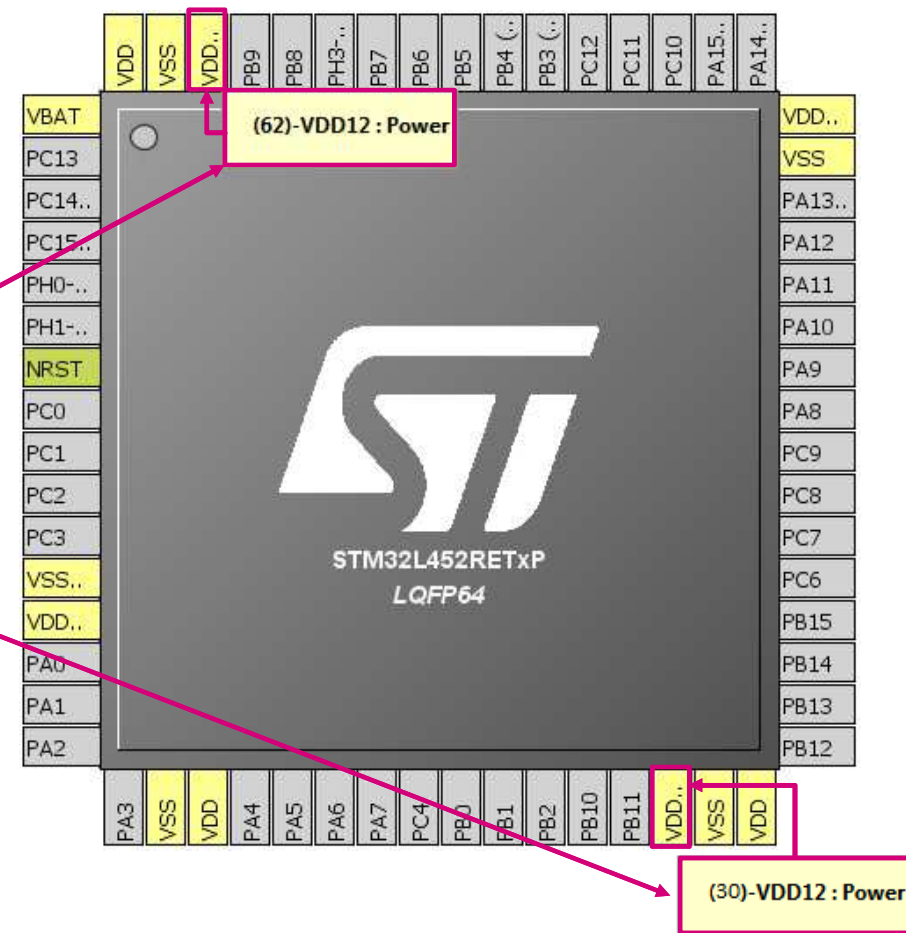
STM32L452RET6(P) - pinout

■ Variants of MCU:

- STM32L452RET6 – device without SMPS support
- STM32L452RET6**P** – device with SMPS support

■ Supply domains:

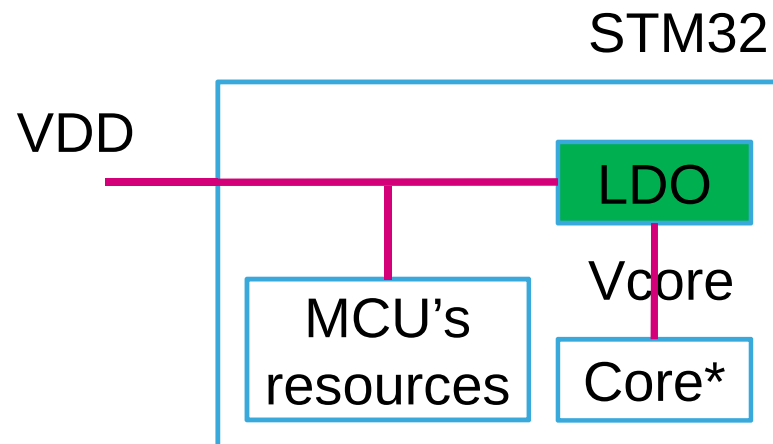
- VDD (1.71 – 3.6V) – 3 pins
- VDD12 (1.05 – 1.32V) – 2 pins
- VBAT (1.55 – 3.6V) – 1 pin
- VDDA (1.62 – 3.6V) – 1 pin
- VDDUSB (3.0 – 3.6V) – 1 pin



STM32L4 with external SMPS

The idea behind

- STM32 with embedded LDO – classical implementation



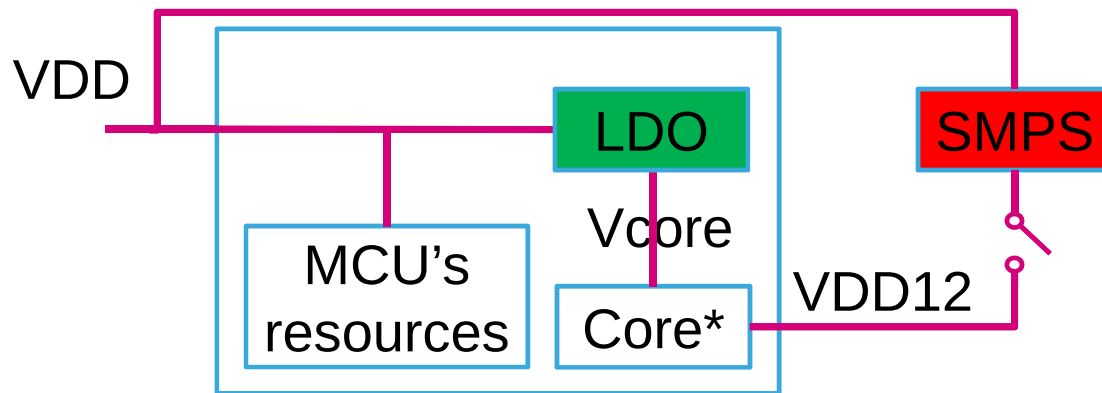
* Core represents Cortex-M, logic and memory

STM32L4 with external SMPS

The idea behind

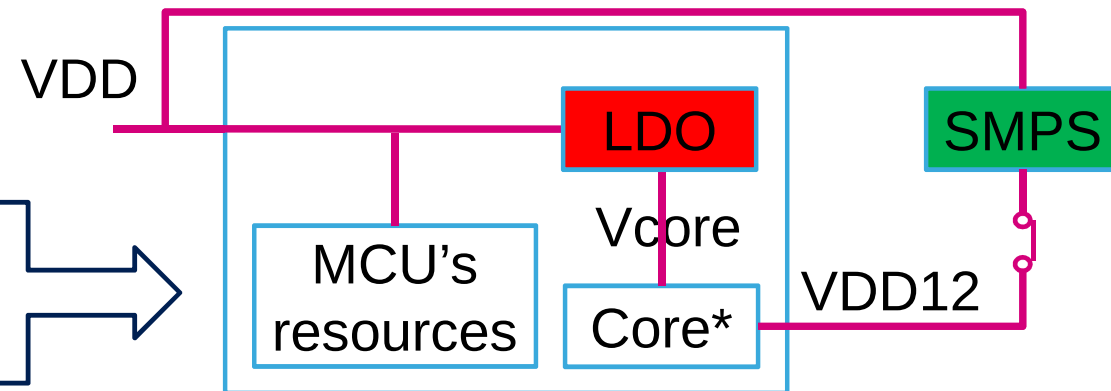
- STM32 with both embedded LDO and external SPMS – new implementation (two I/O pins are changed to VDD12 pins)

STM32...**P**



Variant 1:
LDO mode
used

STM32...**P**



Variant 2:
SMPS
mode used

* Core represents Cortex-M, logic and memory

STM32L4 with external SMPS

Advantages and disadvantages

- Advantages

- Significantly reduced MCU's current consumption in all active modes (RUN, SLEEP) and one inactive mode (STOP0)

- Disadvantages

- No reduction of MCU's current consumption in inactive modes (STOP1, STOP2, STANDBY, SHUTDOWN)
- Less MCU's I/O pins (consumed by VDD12)
- Larger PCB space
- Higher cost of PCB

SMPS example schematics

IC1 (SMPS examples):

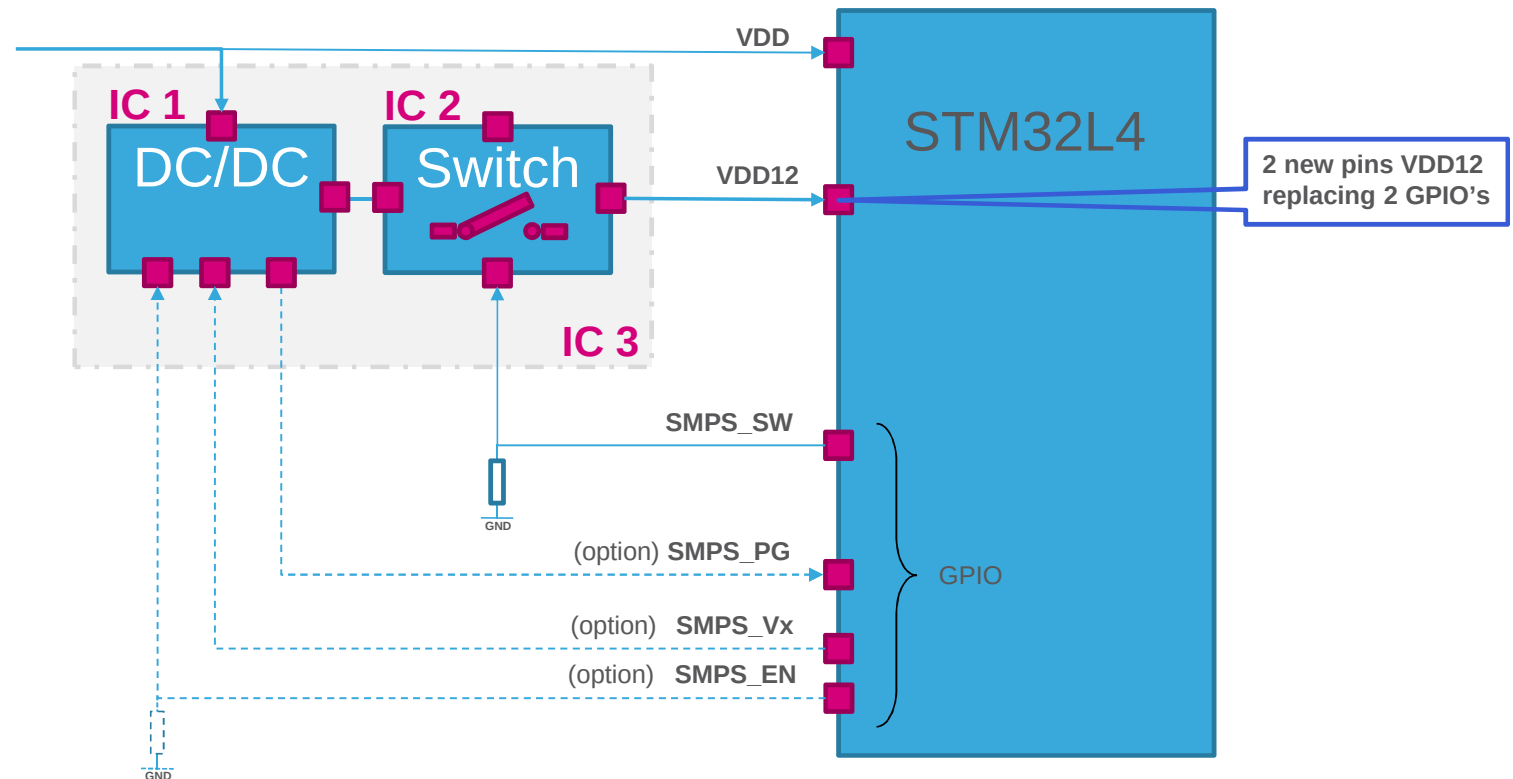
ADI ADP5301 (Nucleo 144/64)
TI TPS62748
ST ST1PS01

IC2 (Switch examples):

ST STG3684A (Nucleo 144)
TI TS3A44159 (Nucleo 64)

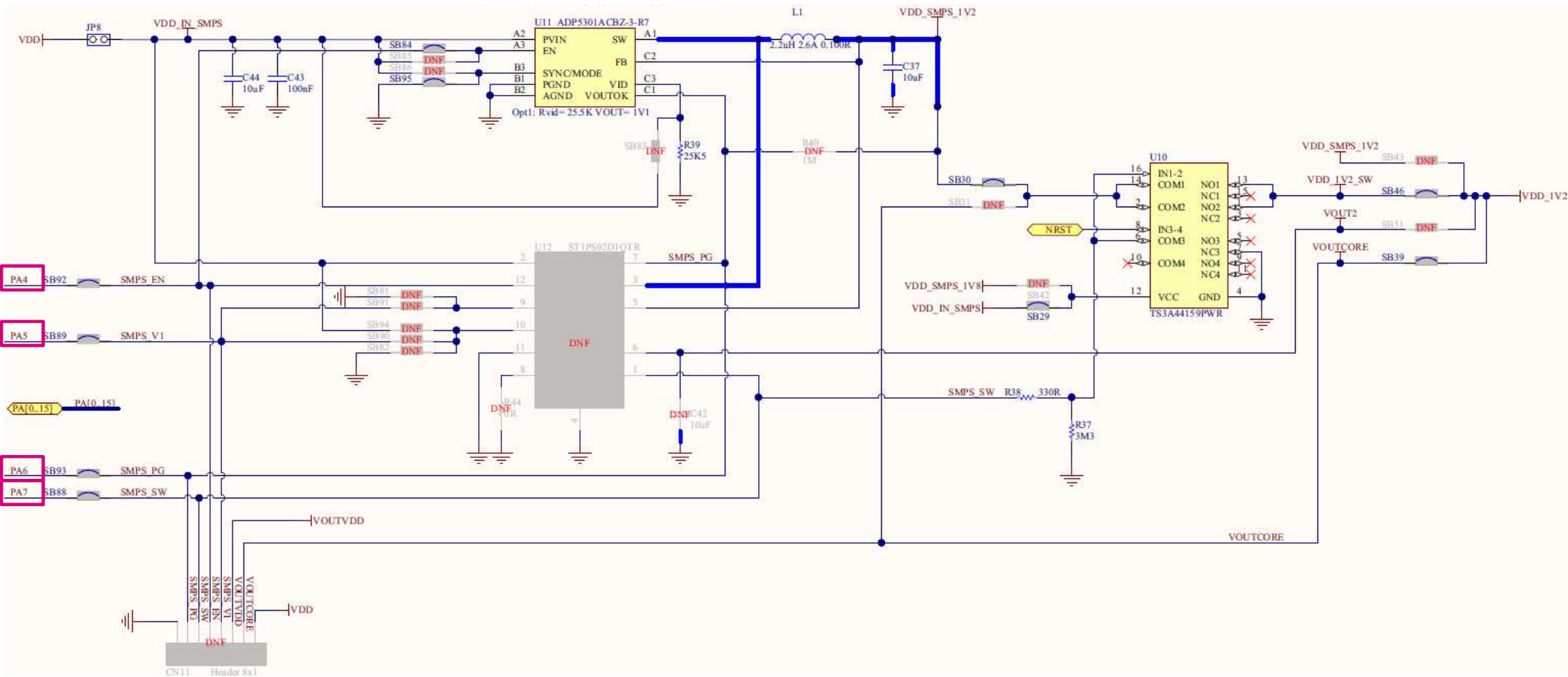
IC3 (SMPS + Switch examples):

ST ST1PS02 (year 2018)



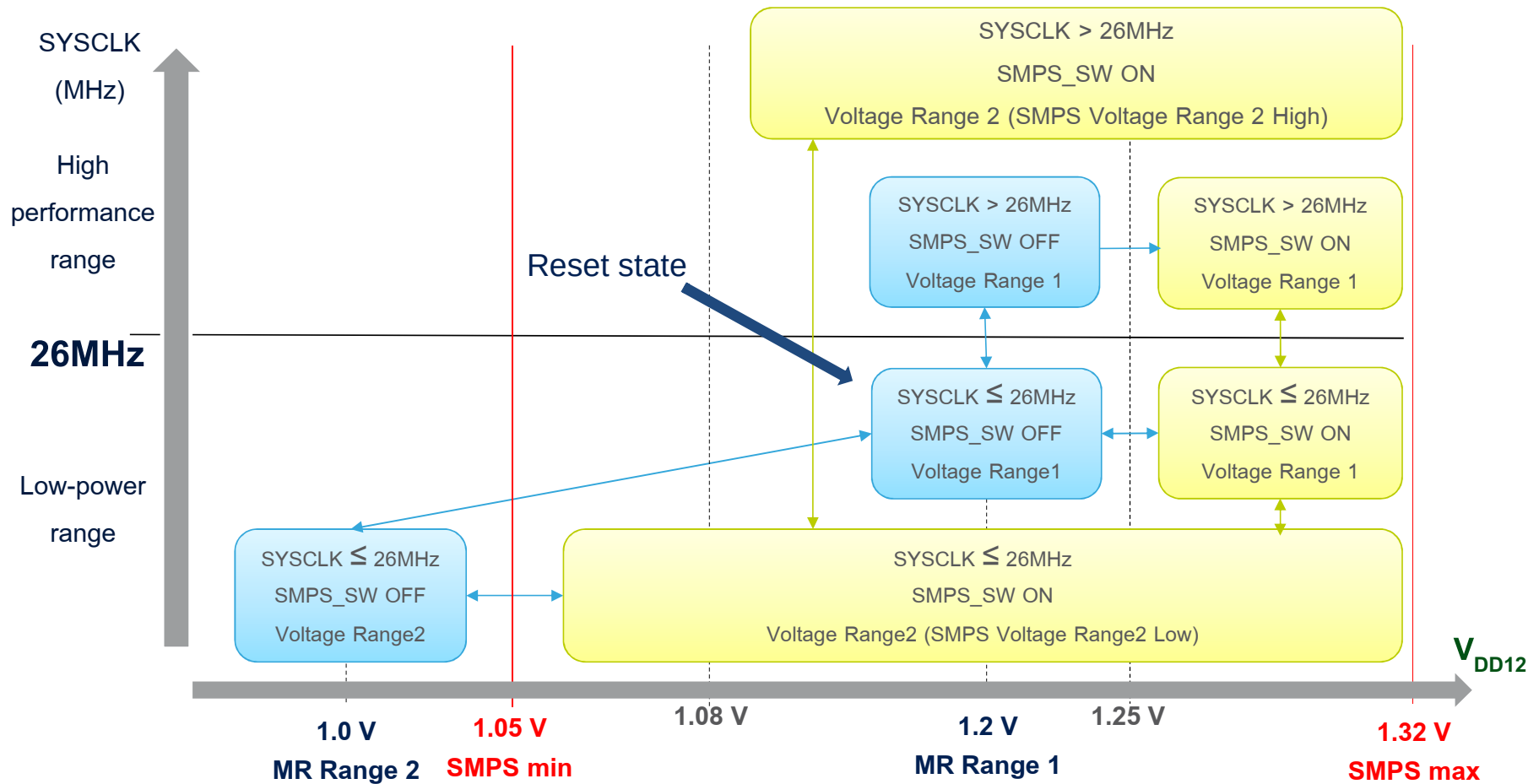
Pins	Mandatory	Type	Function
SMPS_SW	Yes	Out	Control switch to enable SMPS supply on VDD12 pins
SMPS_EN	No	Out	Control SMPS on/off
SMPS_PG	No	In	Check SMPS Power Good
SMPS_Vx	No	Out	Select SMPS voltage

SMPS schematics on STM32L452RET6P Nucleo kit



Transitions

- Only Run, Sleep and Stop 0 allowed
- All power mode transitions available



RUN mode and SMPS

- Key information about the application:

- MCU runs at maximum frequency – 80MHz
- Executed task is very simple – LED blinking in infinite loop
- Current consumption is evaluated in two scenarios:
 - when SMPS is switched off
 - When SMPS is switched on

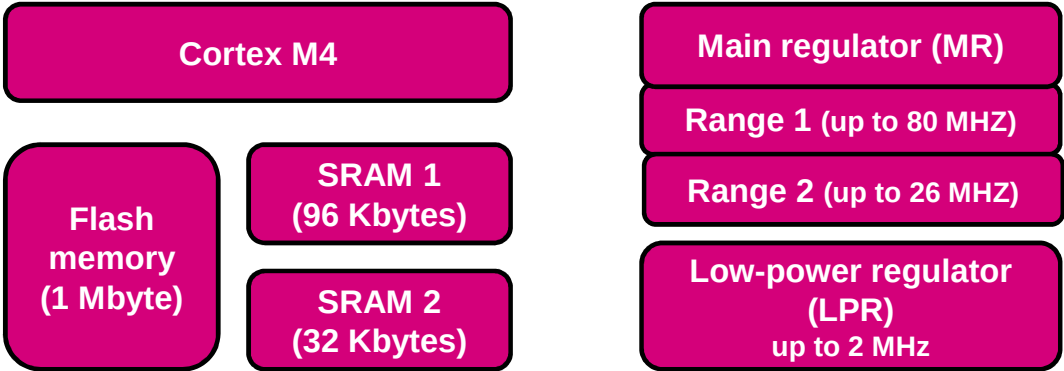
- Applications, which can benefit from combination of RUN mode and SMPS:

- All kind of applications, as in every application MCU uses RUN mode and reduction of current consumption is either a nice benefit or a must

Available peripherals

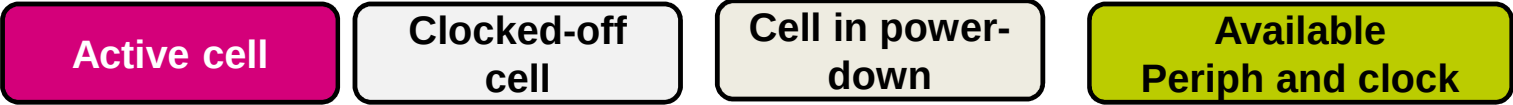
GPIO
DMA
FSMC
QUADSPI
BOR
PVD, PVM
LCD
USB OTG
USART
LP UART
I2C 1 / I2C 2
I2C 3
SPI
CAN
SDMMC
SWPMI
SAI
DFSDM
ADC
DAC
OPAMP
COMP
Temp Sensor
Timers
LPTIM 1
LPTIM 2
IWDG
WWDG
Systick Timer
Touch Sens
RNG
AES
CRC

Run mode



Available clocks

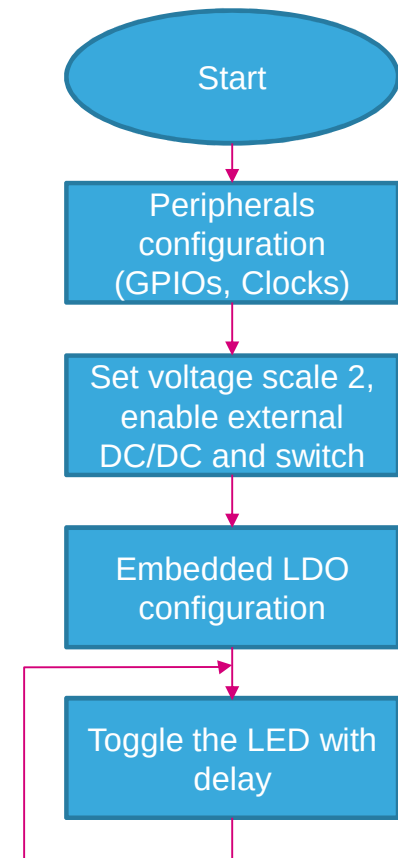
HSI
HSE
LSI
LSE
MSI



Application Description and block diagram

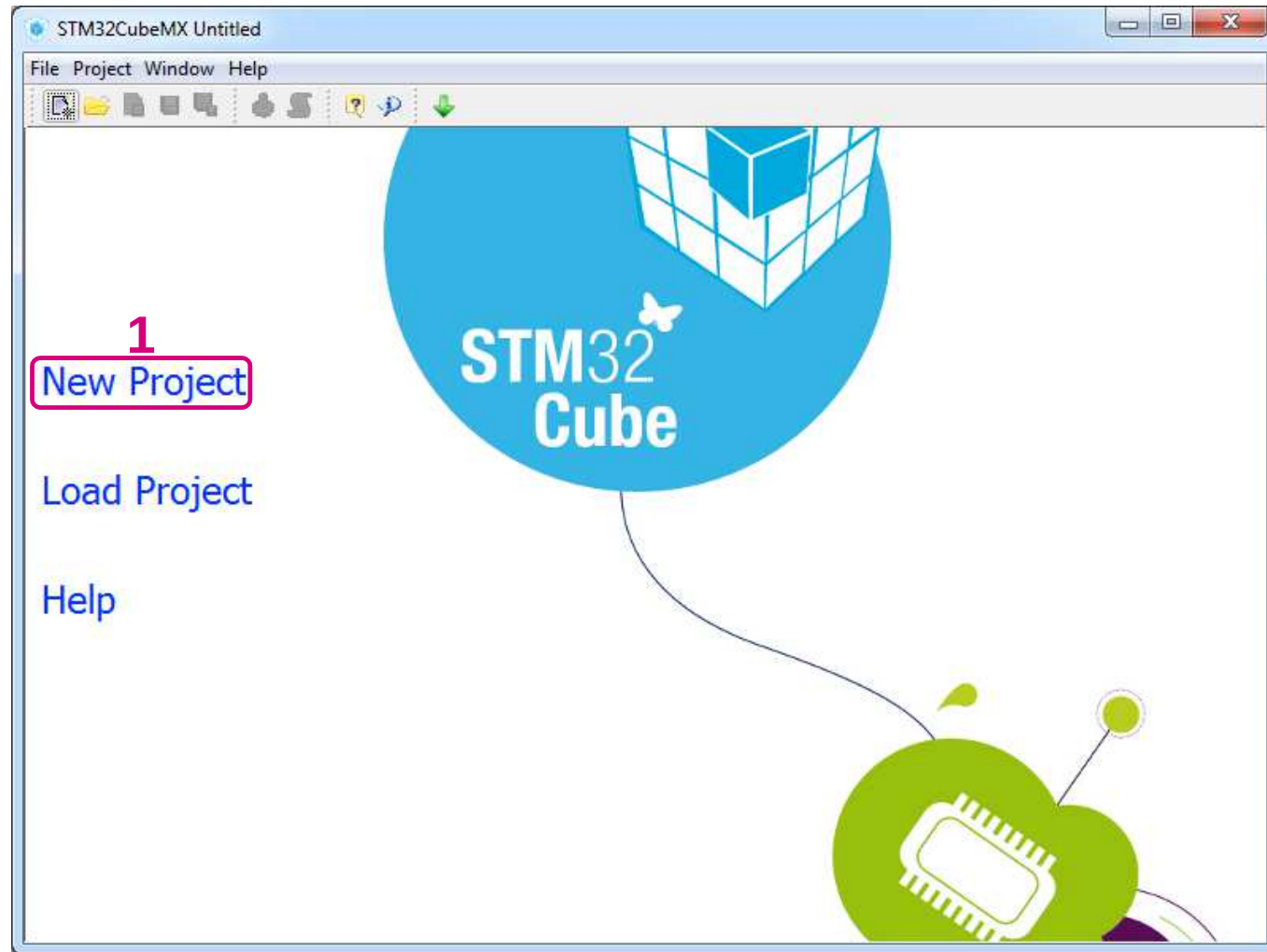
■ Application's behaviour:

1. Configuration of three GPIOs
 - a. PA4 and PA7 as outputs to control external DC/DC and switch
 - b. PB13 as output to control the LED
2. Change the embedded regulator voltage scale to 2
3. Enable DC/DC and switch (set high state on PA4 and PA7)
4. Configuration of MCU's system clock to 80MHz
5. Disable GPIOs clocks to optimize current consumption
6. Toggle the LED (PB13) with delay in the loop



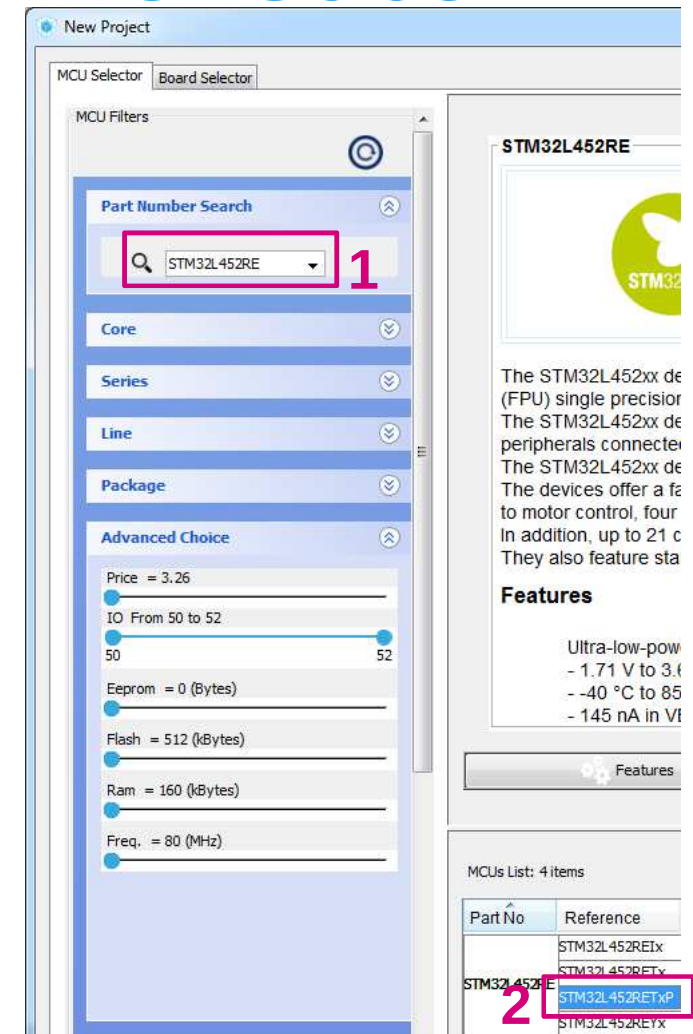
Creating of a new project in STM32CubeMX

- Open STM32CubeMX
- Create a new project by clicking on **New Project**



Creating of a new project in STM32CubeMX

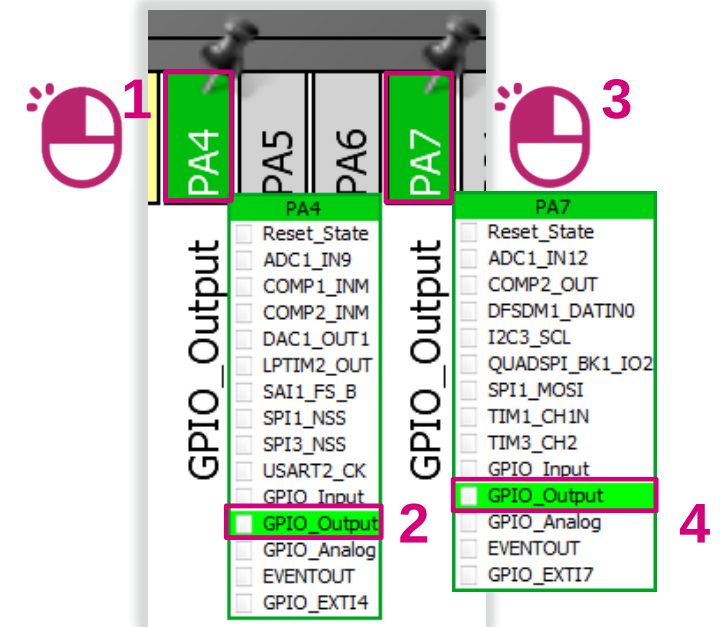
- Select proper device. To shrink the list type **STM32L452RE** in Part Number Search box
- From MCUs list select device: **STM32L452RETxP**
- Double click on selected MCU to create a project



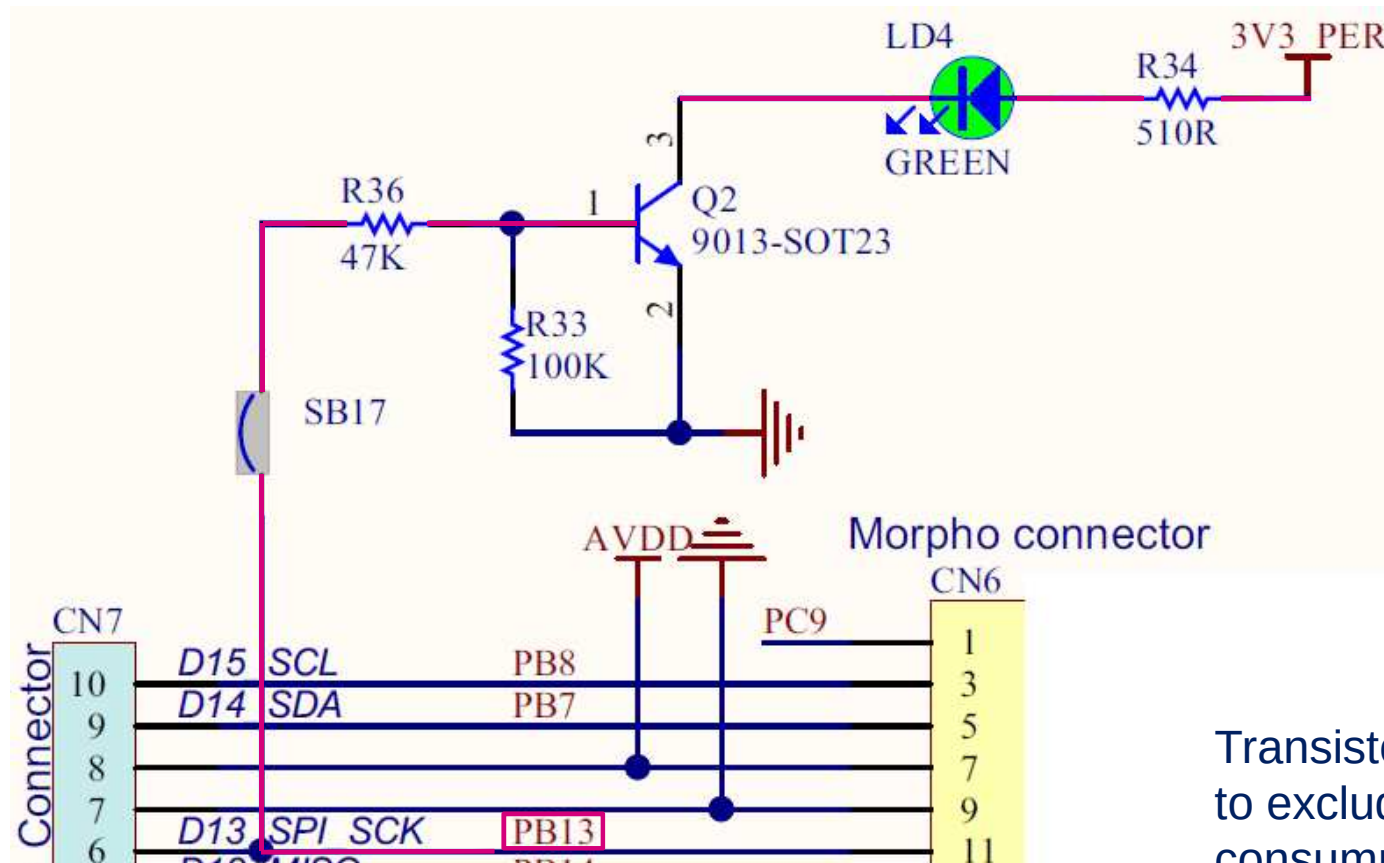
Configuration of pins and peripherals in STM32CubeMX

1. MCU's pins, which control DC/DC

- Configure PA4 (DC/DC enable) as output
 - Left mouse button click on **PA4** in Pinout tab
 - From menu select **GPIO_Output**
- Configure PA7 (switch enable) as output
 - Left mouse button click on **PA7** in Pinout tab
 - From menu select **GPIO_Output**



User LED on STM32L452RET6P Nucleo kit

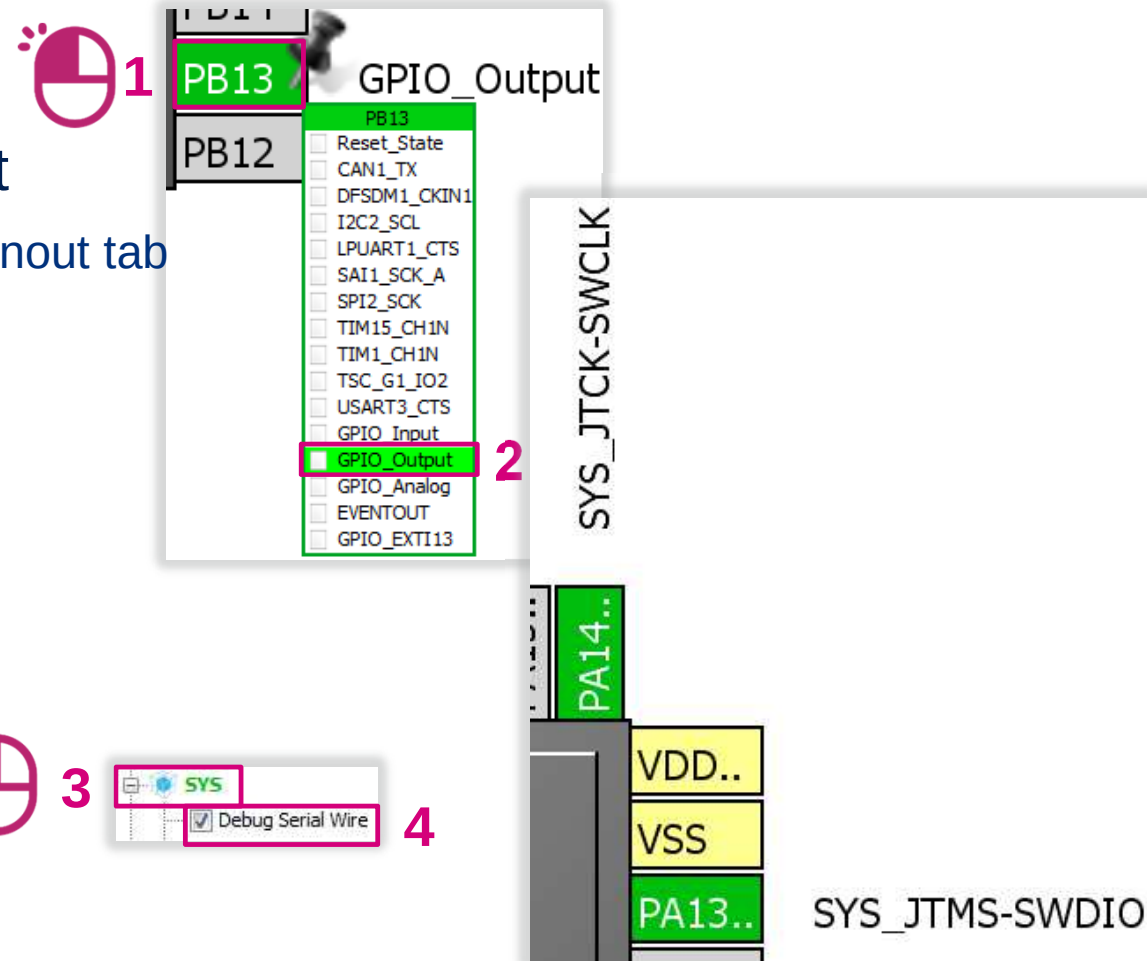


Transistor switch was used to exclude LED's current consumption from MCU current consumption

Configuration of pins and peripherals in STM32CubeMX

2. MCU's other resources

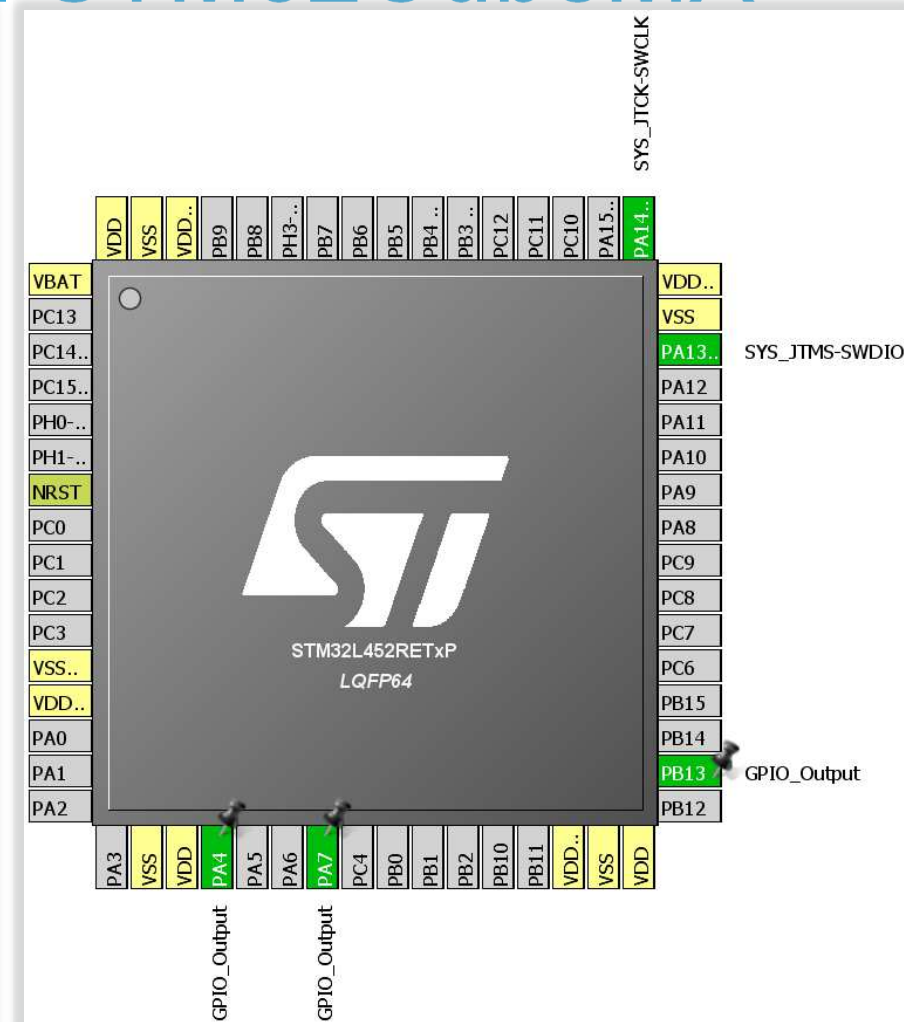
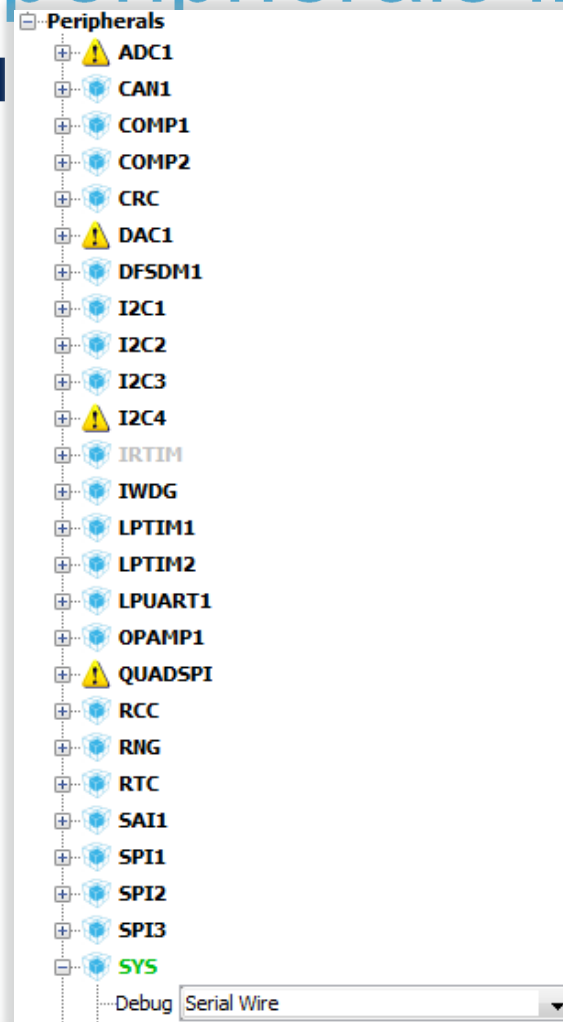
- Configure PB13 (LED) as output
 - Left mouse button click on **PB13** in Pinout tab
 - From menu select **GPIO_Output**
- Enable SWD debug interface
 - Click on **SYS** in Pinout tab
 - Select **Debug Serial Wire**



Configuration of pins and peripherals in STM32CubeMX

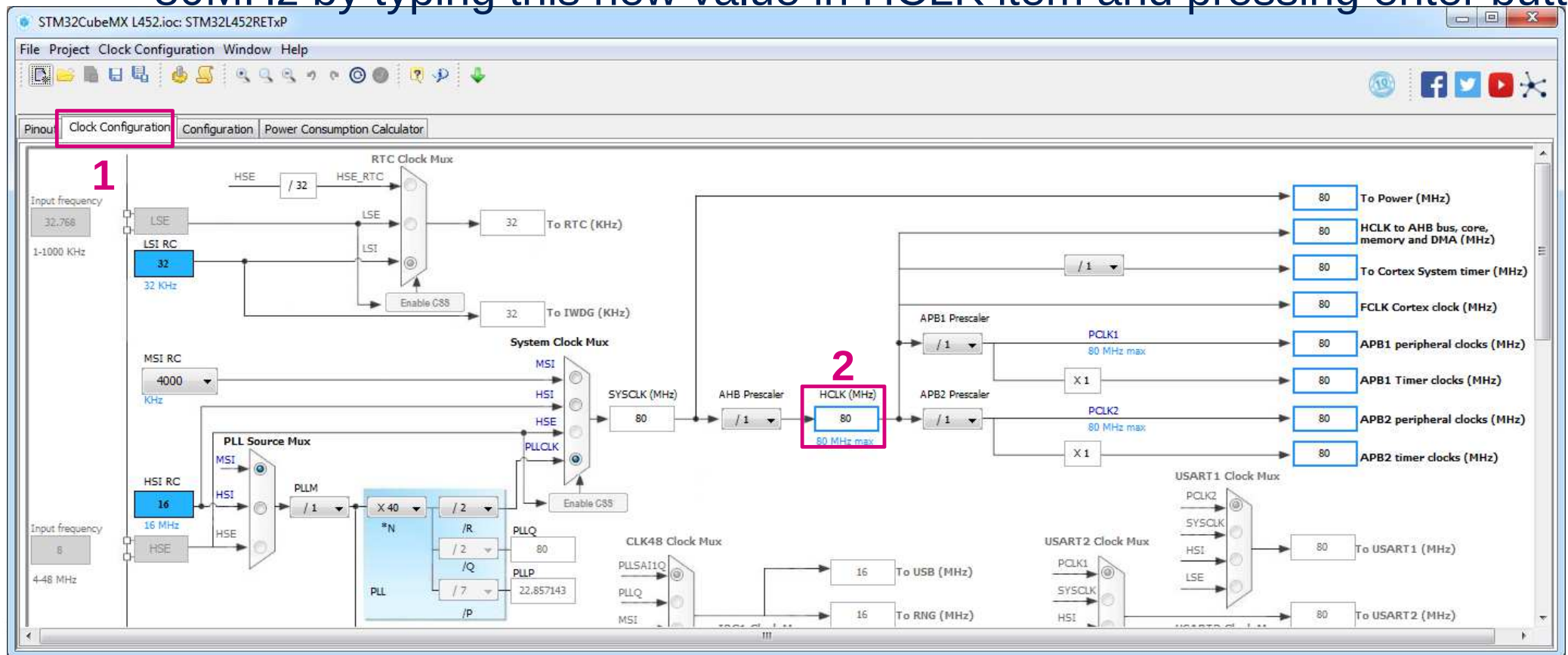
- Finally all needed peripherals are enabled:
 - 3x GPIOs
 - SWD

Checkpoint #1
Pinout configuration is finished



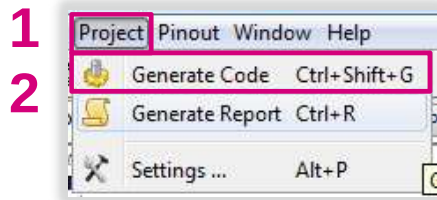
Configuration of pins and peripherals in STM32CubeMX

- Open **Clock Configuration** tab and change MCU's system clock to 80MHz by typing this new value in HCLK item and pressing enter button

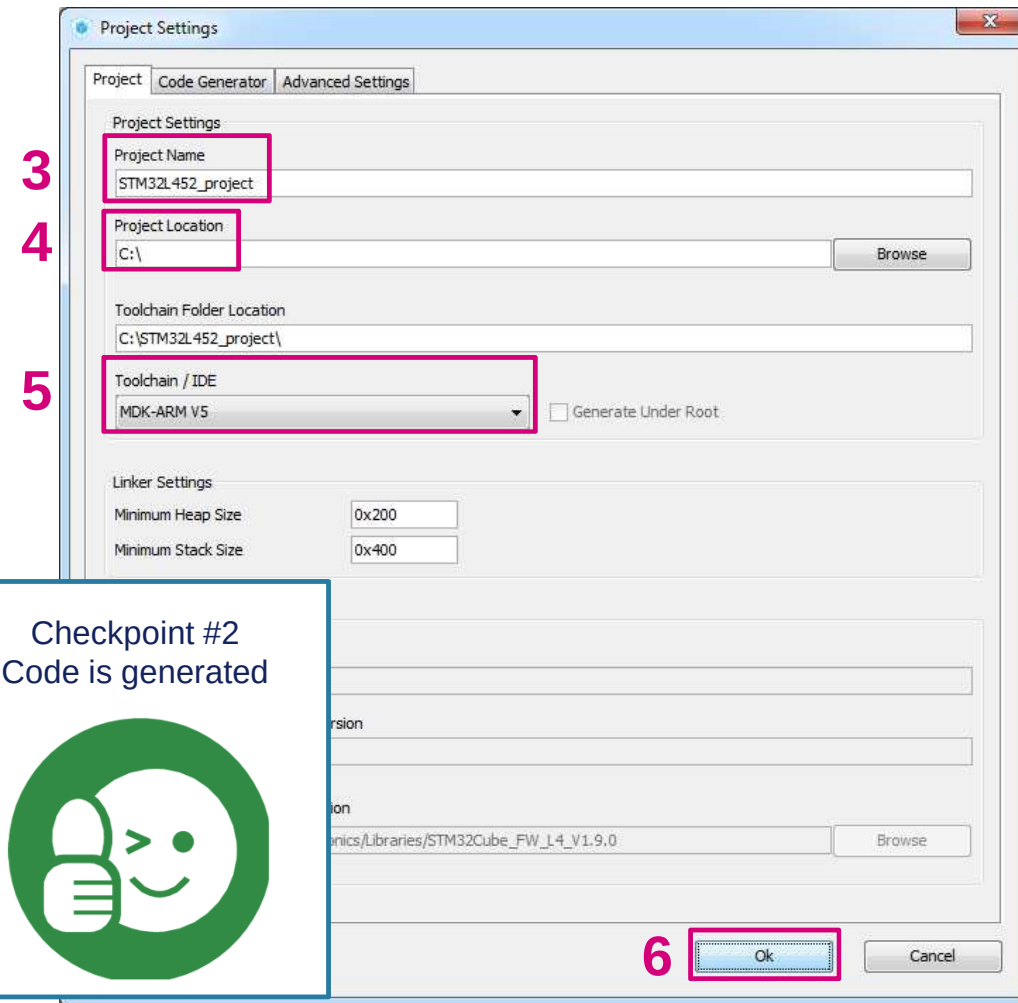


Generating IDE project with code in STM32CubeMX

- From main menu click on **Project -> Generate code**

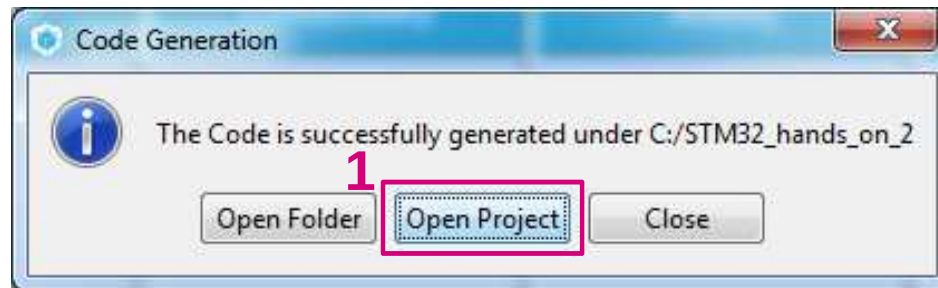


- Fill in **Project Name** and **Project Location** items
- Select preferred toolchain/IDE as **MDK-ARM V5**

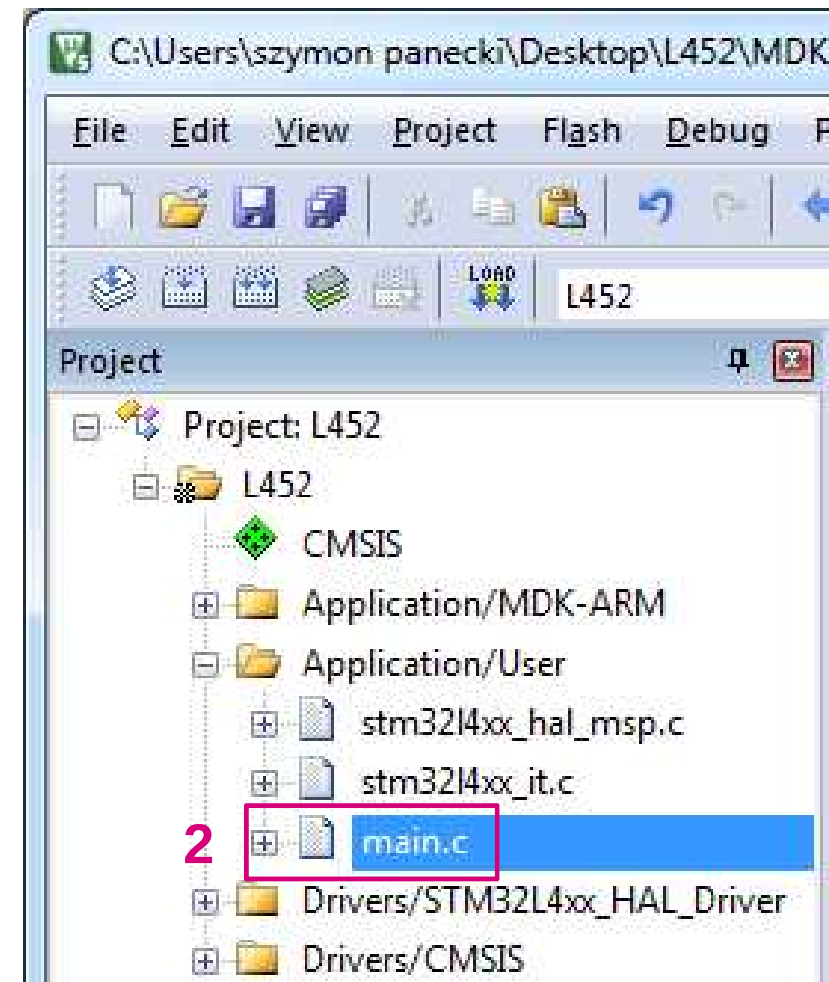


Moving from STM32CubeMX to IDE

- Click on **Open Project** to open it in IDE



- In IDE open main.c file



User code in main.c

- Code to be added in User Code Section 2:
 1. Set high pin PA4 (enable DC/DC)
 2. Set high pin PA7 (close switch)
 3. Disable GPIOs clock except GPIOB (reduce current consumption)

```
/* USER CODE BEGIN 2*/
```

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET);  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
```



```
__HAL_RCC_GPIOC_CLK_DISABLE();  
__HAL_RCC_GPIOH_CLK_DISABLE();  
__HAL_RCC_GPIOA_CLK_DISABLE();
```

```
/* USER CODE END 2*/
```

Let's evaluate the scenario with SMPS switched off first to have a reference point

User code in main.c

- Code to be added in User Code Section 3:

1. Toggle pin PB13
2. Create a short delay

```
/* USER CODE BEGIN 3 */  
HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_13);  
HAL_Delay(500);  
  
/* USER CODE END 3 */
```

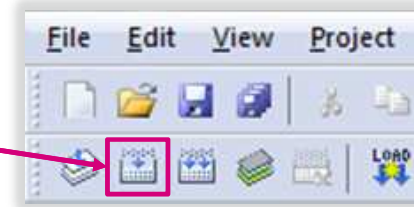
Checkpoint #3
Code development is
finished



Compiling the application, flashing of MCU

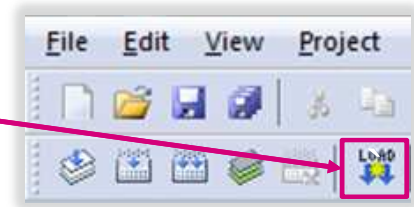
- Compile the code

- By clicking on the **Build** icon
- Or by pressing **F7** keyboard button
- Or by using main menu: **Project -> Build Target**



- Flashing

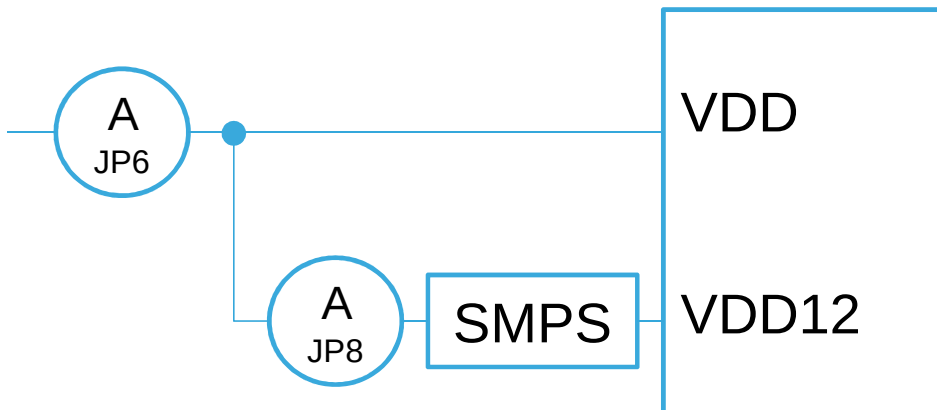
- By clicking **Load** icon
- Or by pressing **F8** keyboard button
- Or by using main menu: **Flash -> Download**



Evaluating of application

1. MCU supplied **without** support from SMPS

- Measure the current consumption on:
 - **JP6** (current for VDD, VDD12 and SMPS)
 - **JP8** (current for VDD12 and SMPS)



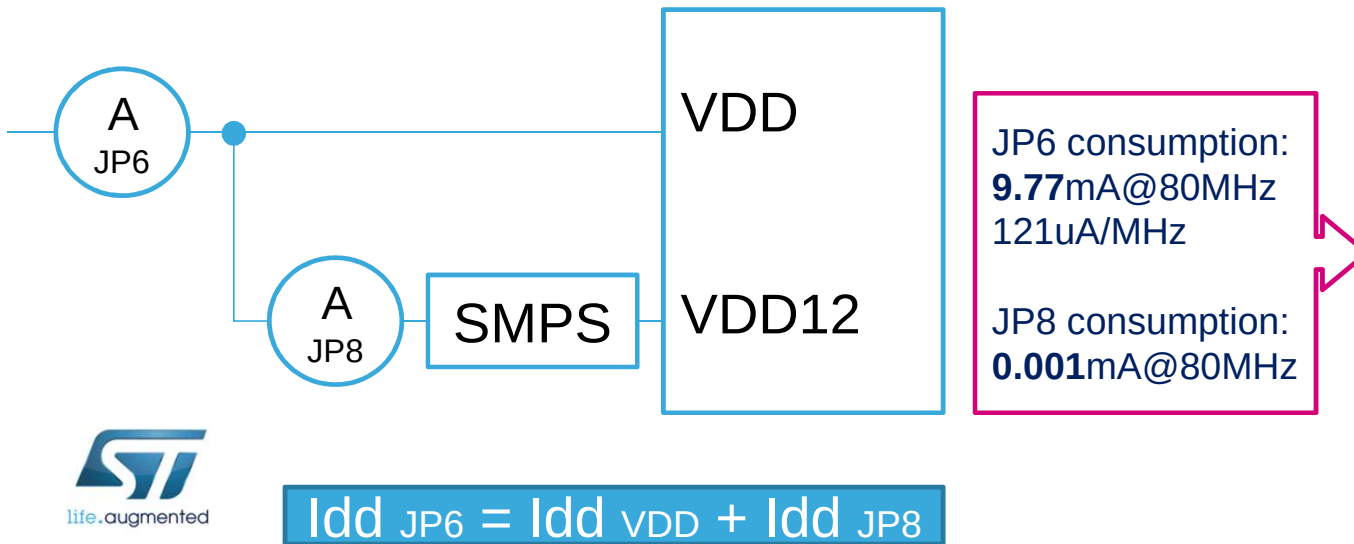
$$I_{dd\ JP6} = I_{dd\ VDD} + I_{dd\ JP8}$$



Evaluating of application

1. MCU supplied **without** support from SMPS

- Measure the current consumption on:
 - **JP6** (current for VDD, VDD12 and SMPS)
 - **JP8** (current for VDD12 and SMPS)



Evaluating of application

1. MCU supplied **with** support from SMPS

- Change the embedded regulator voltage scale to 2
- Change state of GPIO functions responsible for pins PA4 and PA7 from low to high (edit line 95 and 96) and add a short delay between these two function calls
- Move the call of SystemClock_Config() function after calls of HAL_GPIO_WritePin() functions

```
HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE2);
```

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);
```

```
HAL_Delay(100);
```

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
```

```
SystemClock_Config();
```

- Comment the code inside SystemClock_Config() function responsible for setting regulator voltage scale 1

```
//if(HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
```

```
{
```

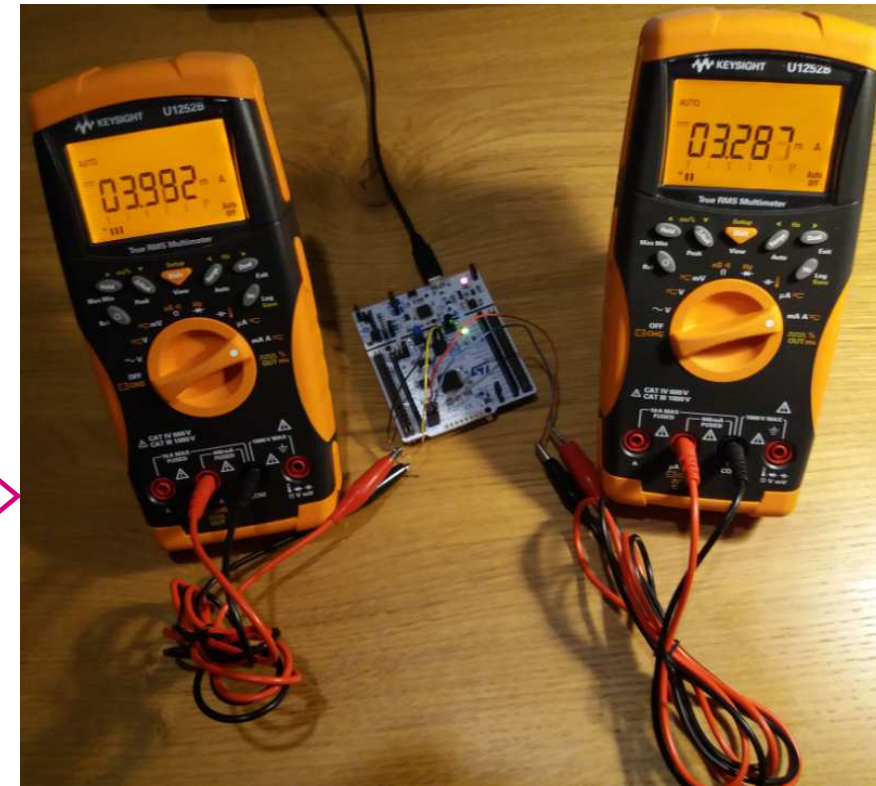
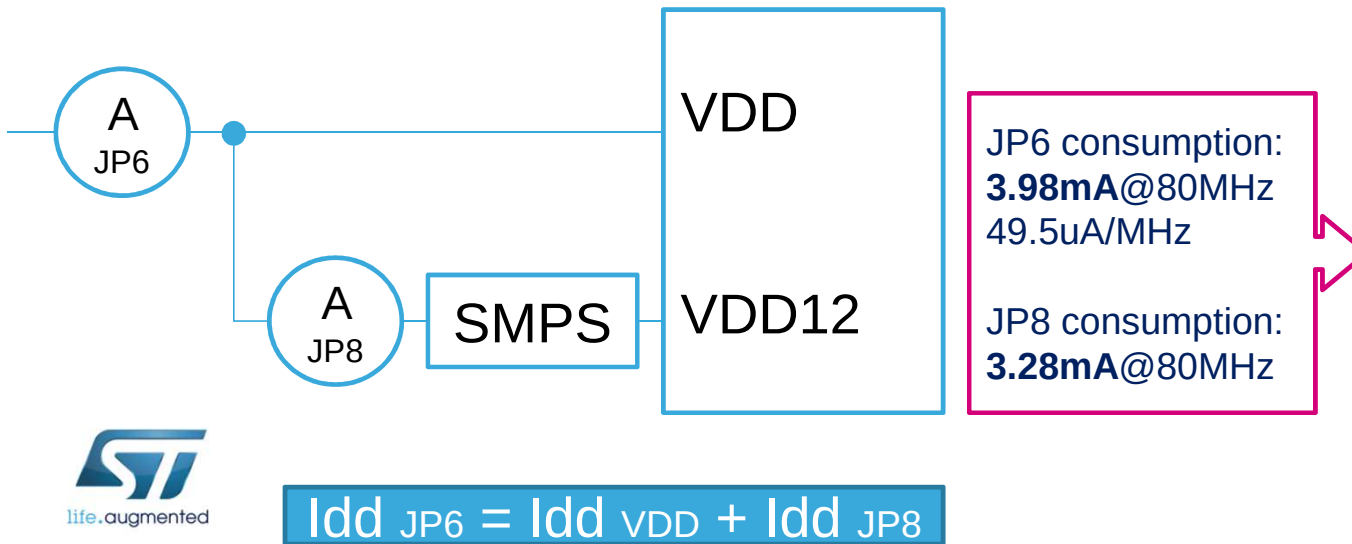
```
//  _Error_Handler(__FILE__, __LINE__);
```

```
}
```

Evaluating of application

1. MCU supplied **with** support from SMPS

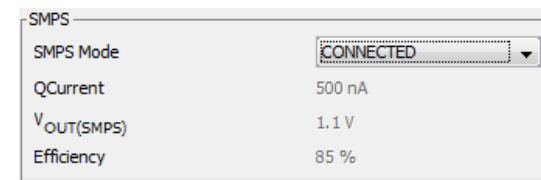
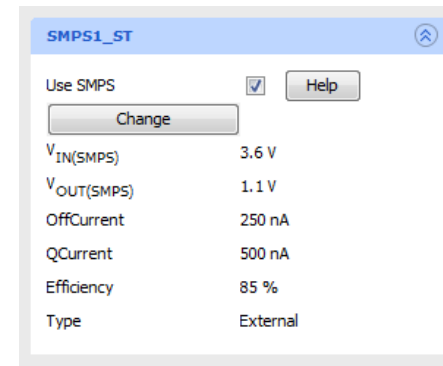
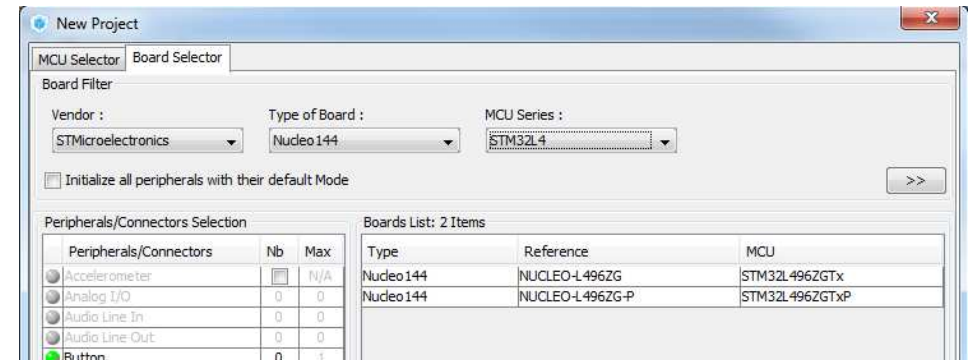
- Measure the current consumption on:
 - JP6
 - JP8



STM32L4 SMPS ecosystem: CubeMX

- **With latest version 4.23.0**

- Support SMPS version STM32L4 MCUs
- VDD12 POWER pins
- No specific code for SMPS
- PCC :
 - Current consumption computations with PCC
 - SMPS steps
 - SMPS transition checker
 - SMPS power consumption comparison (with and without)
- PCC SMPS states:
 - **OFF** : SMPS is OFF, switch is open
=> Offcurrent consumption only
 - **CONNECTED**: SMPS is ON, switch is closed
 - **DISCONNECTED**: SMPS is ON, switch is open
=> Qcurrent (Quiescent) consumption only



STM32L4 SMPS ecosystem: BSP Functions

- **Board Support Package** implements the following functions to manage SMPS

Function call	Description
BSP_SMPS_Init(uint32_t Voltage)	Initialize the external SMPS pins: SMPS_EN, SMPS_SW, SMPS_PG, SMPS_V1
BSP_SMPS_DeInit(void)	De-initialize the external SMPS component.
BSP_SMPS_Enable(uint32_t Delay, uint32_t Power_Good_Check)	Enable the external SMPS component by setting the SMPS_EN pin to '1'
BSP_SMPS_Disable(void)	Disable the external SMPS component by clearing the SMPS_EN pin to 0 only if SMPS_SW is already set to 0. Otherwise, the function returns an error code
BSP_SMPS_Supply_Enable (uint32_t Delay, uint32_t Power_Good_Check)	Close the switch to enable the power supply on the VDD12 pins by setting the SMPS_SW pin to '1'
BSP_SMPS_Supply_Disable(void)	Disable the SMPS power supply on the VDD12 pins by clearing the SMPS_SW pin to '0'

STM32L4 SMPS ecosystem: materials

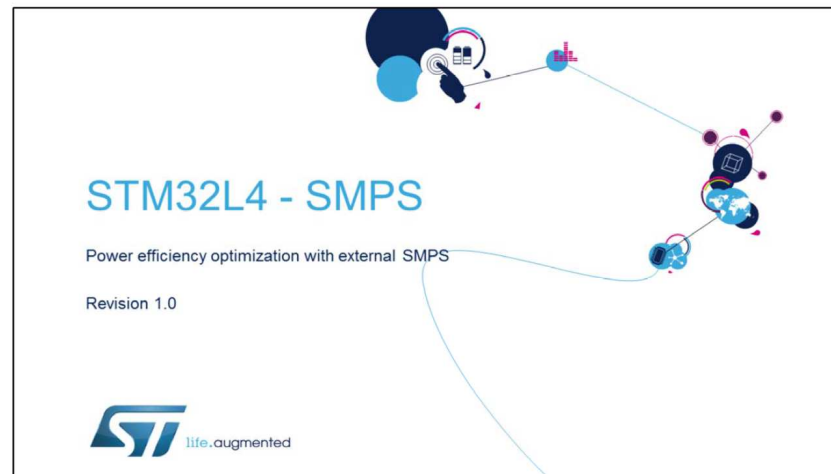
- AN4978: Design recommendations for STM32L4xxxx with external SMPS



AN4978 **Application note**

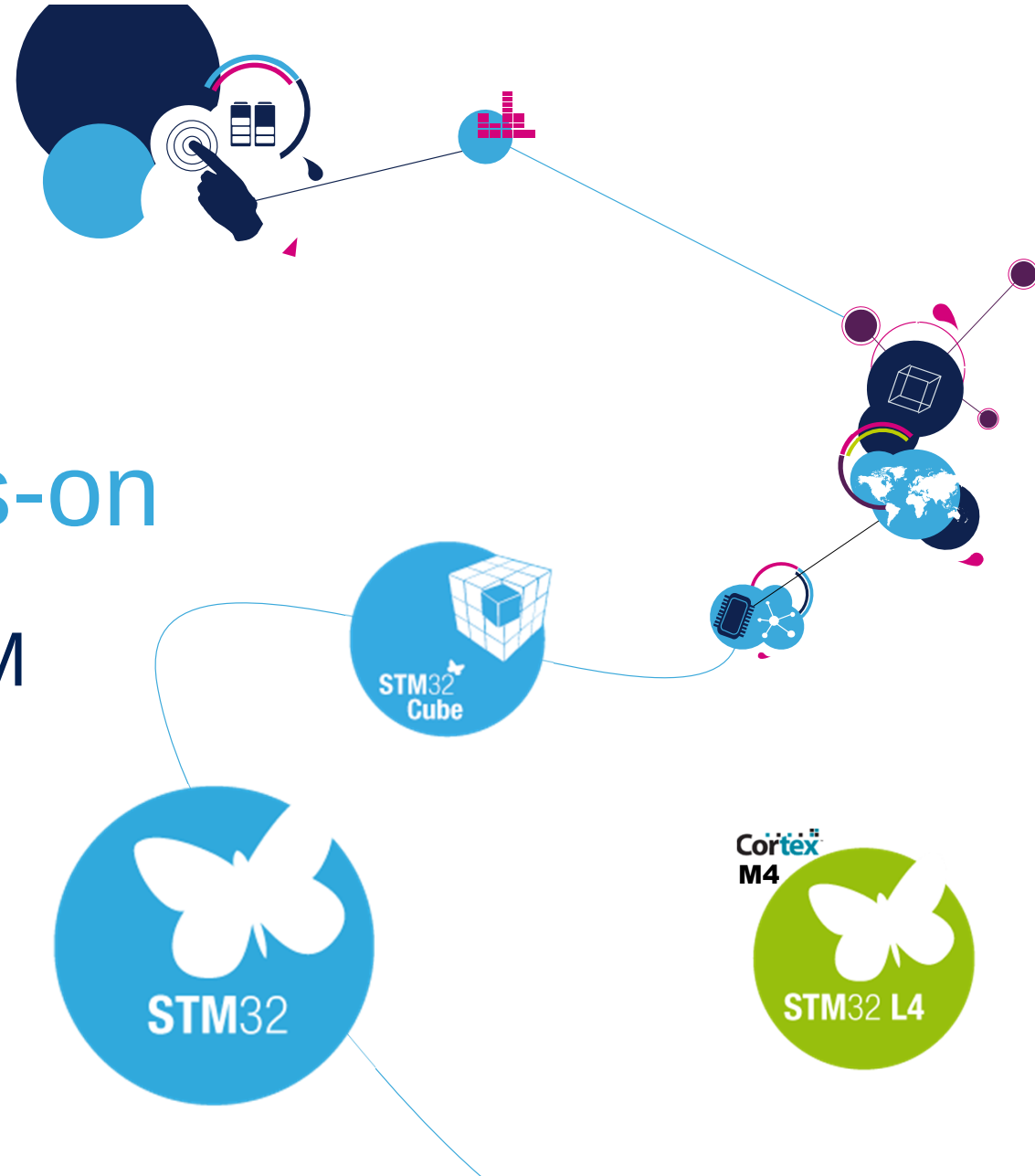
Design recommendations for STM32L4xxxx with external SMPS,
for ultra-low-power applications with high performance

- STM32L4 online training: presentation about STM32L4 SMPS



Low-power hands-on

STOP mode and LPTIM



STOP mode and LPTIM

- Key information about the application :
 - MCU starts in RUN mode with default clocks configuration (4MHz system clock)
 - MCU enters low-power mode (STOP)
 - LPTIM counts external pulses and wakes up the MCU after reaching defined number of pulses
 - After waking up MCU changes state of LED and comes back to low-power mode
- Applications, which can benefit from combination of STOP mode and LPTIM
 - Metering applications, especially water meters, which need to count pulses and consume as low as possible current

Available peripherals

GPIO
DMA
FSMC
OSPI
BOR
PVD, PVM
LCD
USB OTG
USART
LP UART
I2C 1 / I2C 2
I2C 3
SPI
CAN
SDMMC
SWPMI
SAI
DFSDM
ADC
DAC
OPAMP
COMP
Temp Sensor
Timers
LPTIM 1
LPTIM 2
IWDG
WWDG
Systick Timer
Touch Sens
RNG
AES
CRC

I/Os kept, and configurable

Zzz

Cortex M4

Flash memory
(1 Mbyte)

SRAM 1
(96 Kbytes)

SRAM 2
(32 Kbytes)

Main regulator (MR)

Low Power regulator
(LPR)

Backup domain

Backup Register
(32x32-bits)

RTC

Wake-up event

NRST
BOR
PVD
PVM
RTC + Tamper
LCD
USB OTG
USART
LP UART
I2C 1 / I2C 2
I2C 3
SWPMI
COMP
LPTIM 1
LPTIM 2
IWDG
GPIOs

Available clocks

HSI
HSE
LSI
LSE
MSI

Active cell

Clocked-off cell

Cell in power-down

Available cell

Stop 1 mode

Available peripherals

GPIO
DMA
FSMC
OSPI
BOR
PVD, PVM
LCD
USB OTG
USART
LP UART
I2C 1 / I2C 2
I2C 3
SPI
CAN
SDMMC
SWPMI
SAI
DFSDM
ADC
DAC
OPAMP
COMP
Temp Sensor
Timers
LPTIM 1
LPTIM 2
IWDG
WWDG
Systick Timer
Touch Sens
RNG
AES
CRC

I/Os kept, and configurable

Zzz

Cortex M4

Flash
memory
(1 Mbyte)

SRAM 1
(96 Kbytes)

SRAM 2
(32 Kbytes)

Main regulator (MR)

Low Power regulator
(LPR)

Backup domain

Backup Register
(32x32-bits)

RTC

Wake-up
event

NRST

BOR

PVD

PVM

RTC + Tamper

LCD

LP UART

I2C 3

COMP

LPTIM 1

IWDG

GPIOs

Available
clocks

HSI
HSE
LSI
LSE
MSI

Active cell

Clocked-off cell

Cell in power-down

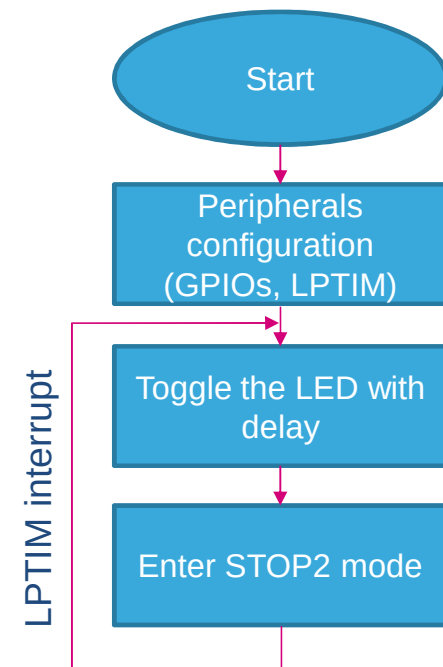
Available cell

Stop 2 mode

Application Description and block diagram

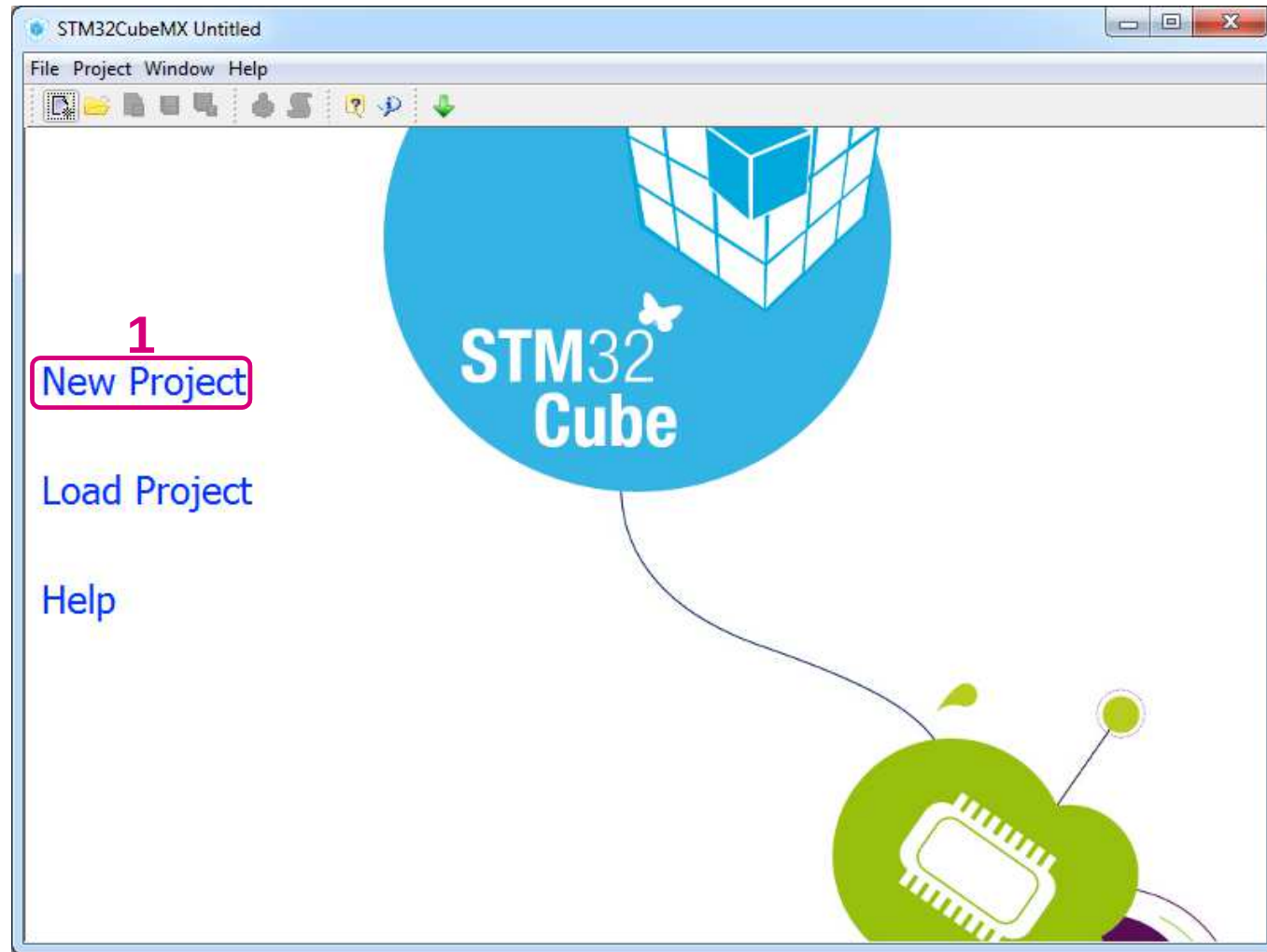
■ Application's behaviour:

1. Configuration of PB13 GPIO as output to control LED
2. Configuration of LPTIM to count external pulses and generate interrupt
3. Enable pull-ups on PA2 and PB3 to decrease current consumption (action is specific for Nucleo-P)
4. Enter STOP2 mode
5. Count external pulses (PC0) and wake up after reaching defined number of pulses
6. Toggle the LED (PB13), then back to point number 4



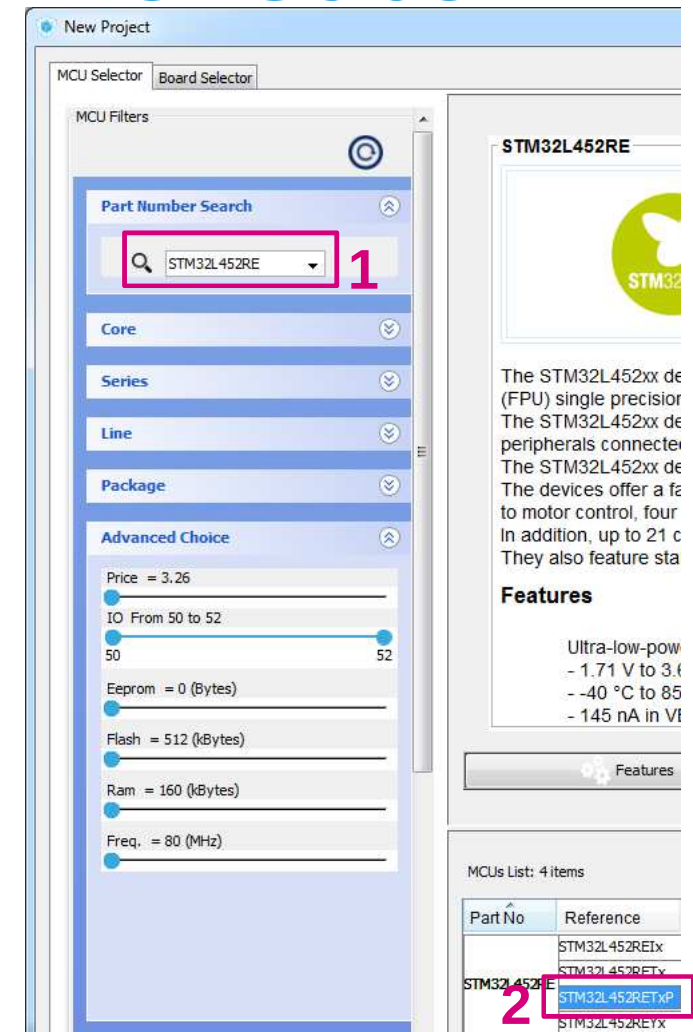
Creating of a new project in STM32CubeMX

- Open STM32CubeMX
- Create a new project by clicking on **New Project**

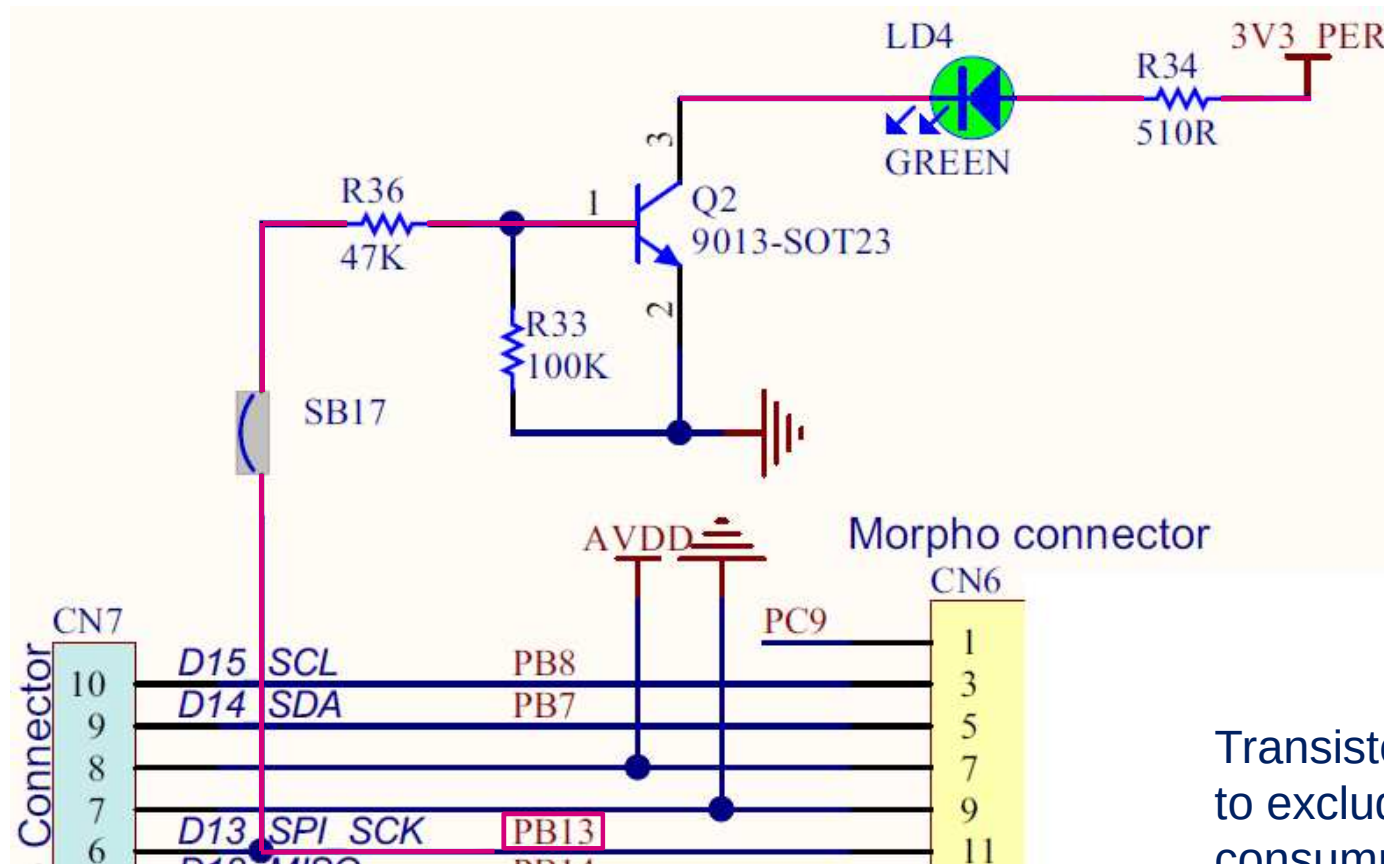


Creating of a new project in STM32CubeMX

- Select proper device. To shrink the list type **STM32L452RE** in Part Number Search box
- From MCUs list select device: **STM32L452RETxP**
- Double click on selected MCU to create a project



User LED on STM32L452RET6P Nucleo kit

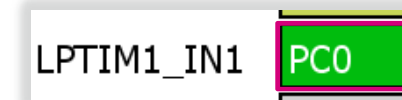
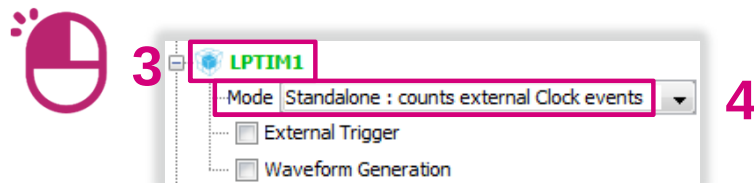
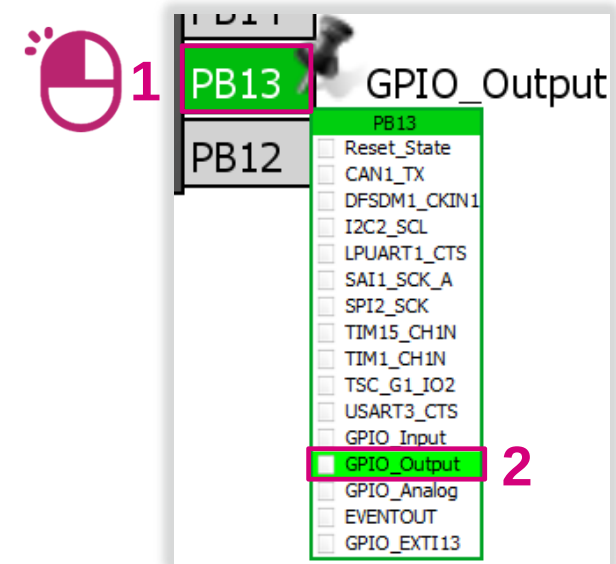


Transistor switch was used to exclude LED's current consumption from MCU current consumption

Configuration of pins and peripherals in STM32CubeMX

1. MCU's resources

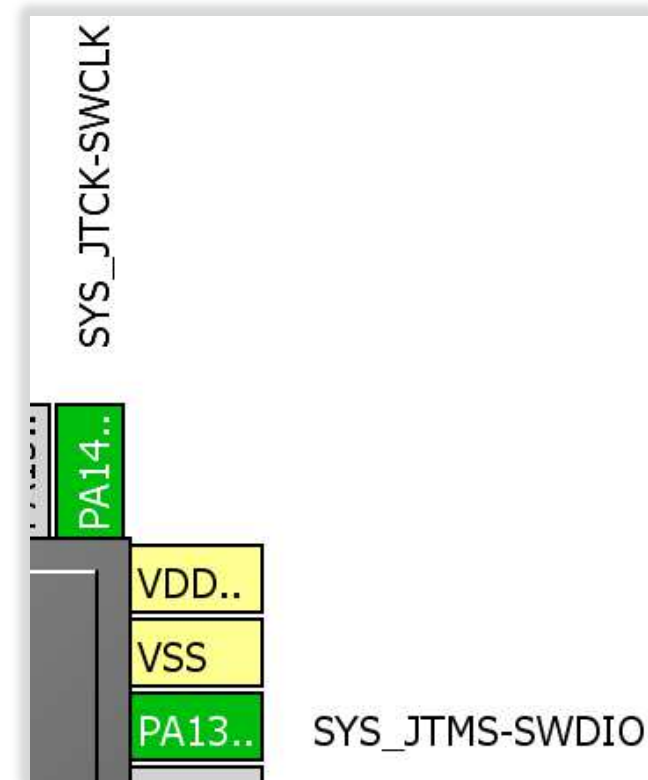
- Configure PB13 (LED) as output
 - Left mouse button click on **PB13** in Pinout tab
 - From menu select **GPIO_Output**
- Enable LPTIM1
 - Click on **LPTIM1** in Pinout tab
 - Change mode to **Standalone: count external clock events**



Configuration of pins and peripherals in STM32CubeMX

2. MCU's resources

- Enable SWD debug interface
 - Click on **SYS** in Pinout tab
 - Select **Debug Serial Wire**

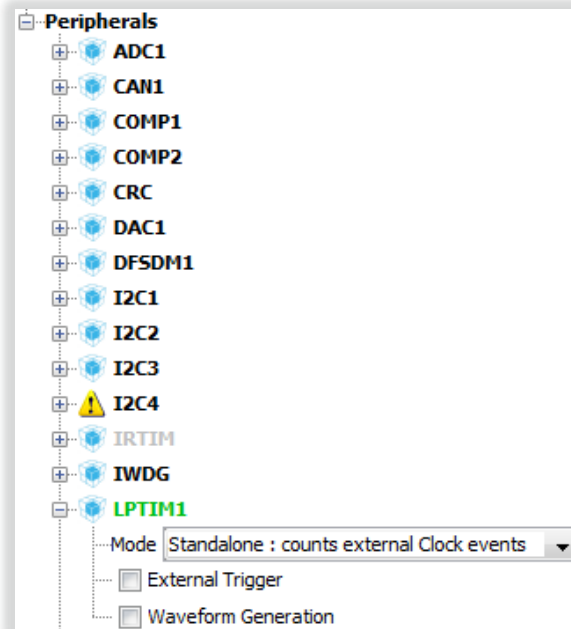


Configuration of pins and peripherals in STM32CubeMX

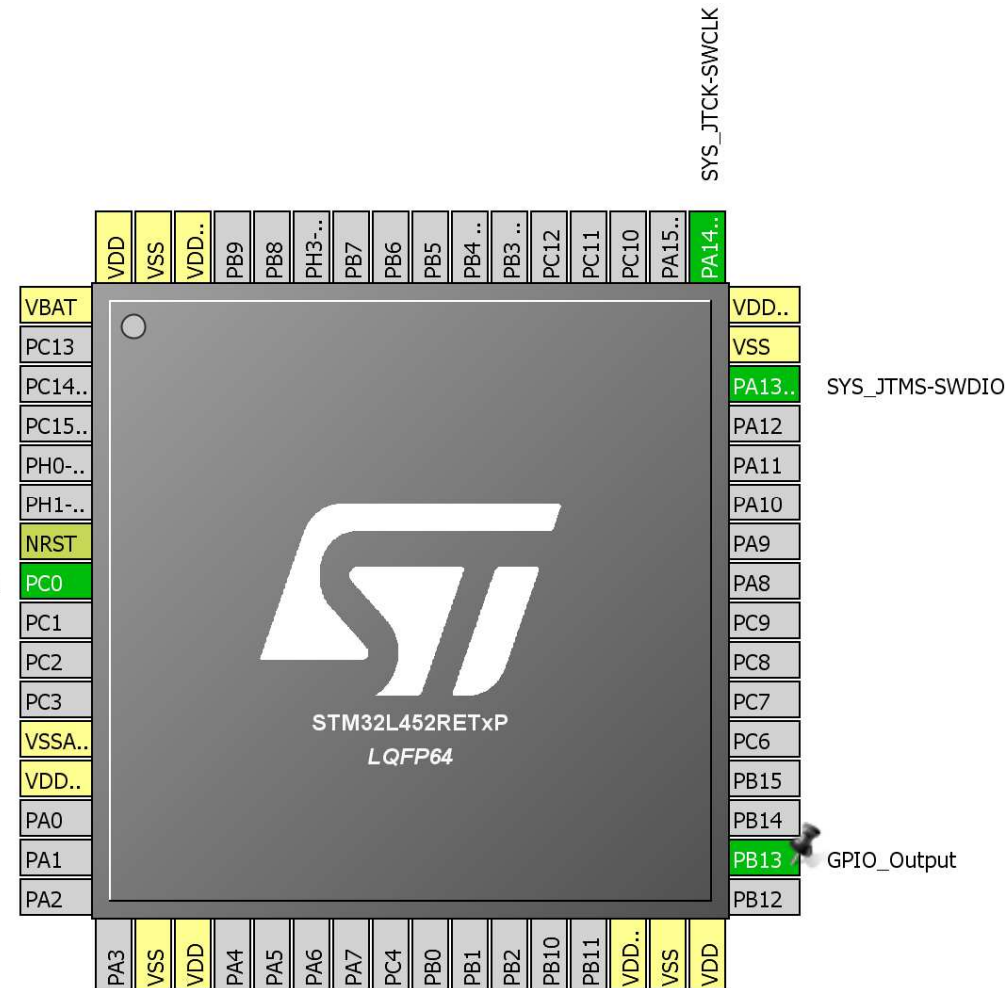
- Finally all needed peripherals are enabled:

- GPIO
- LPTIM1
- SWD

Checkpoint #1
Pinout configuration is finished

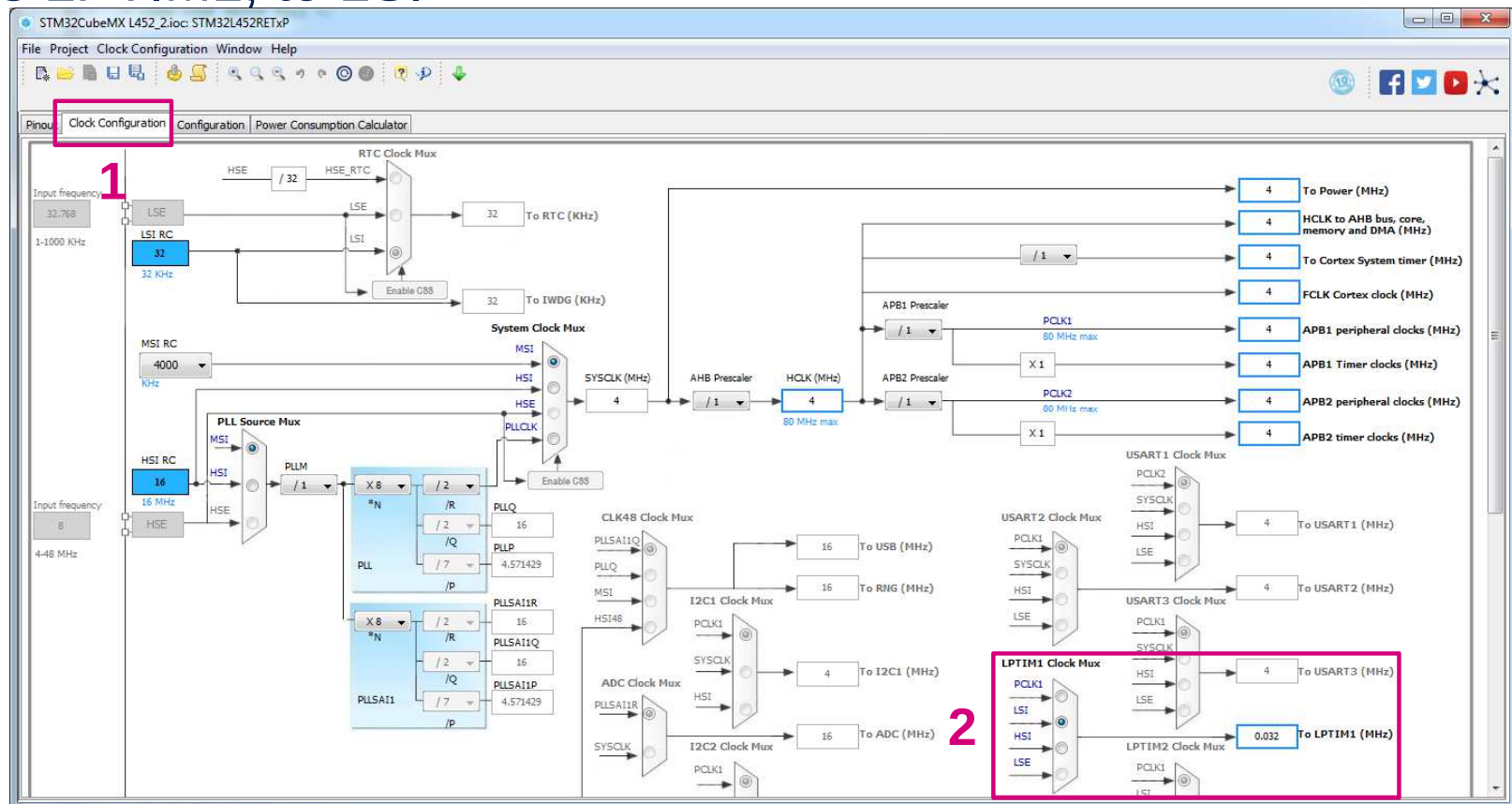


LPTIM1_IN1



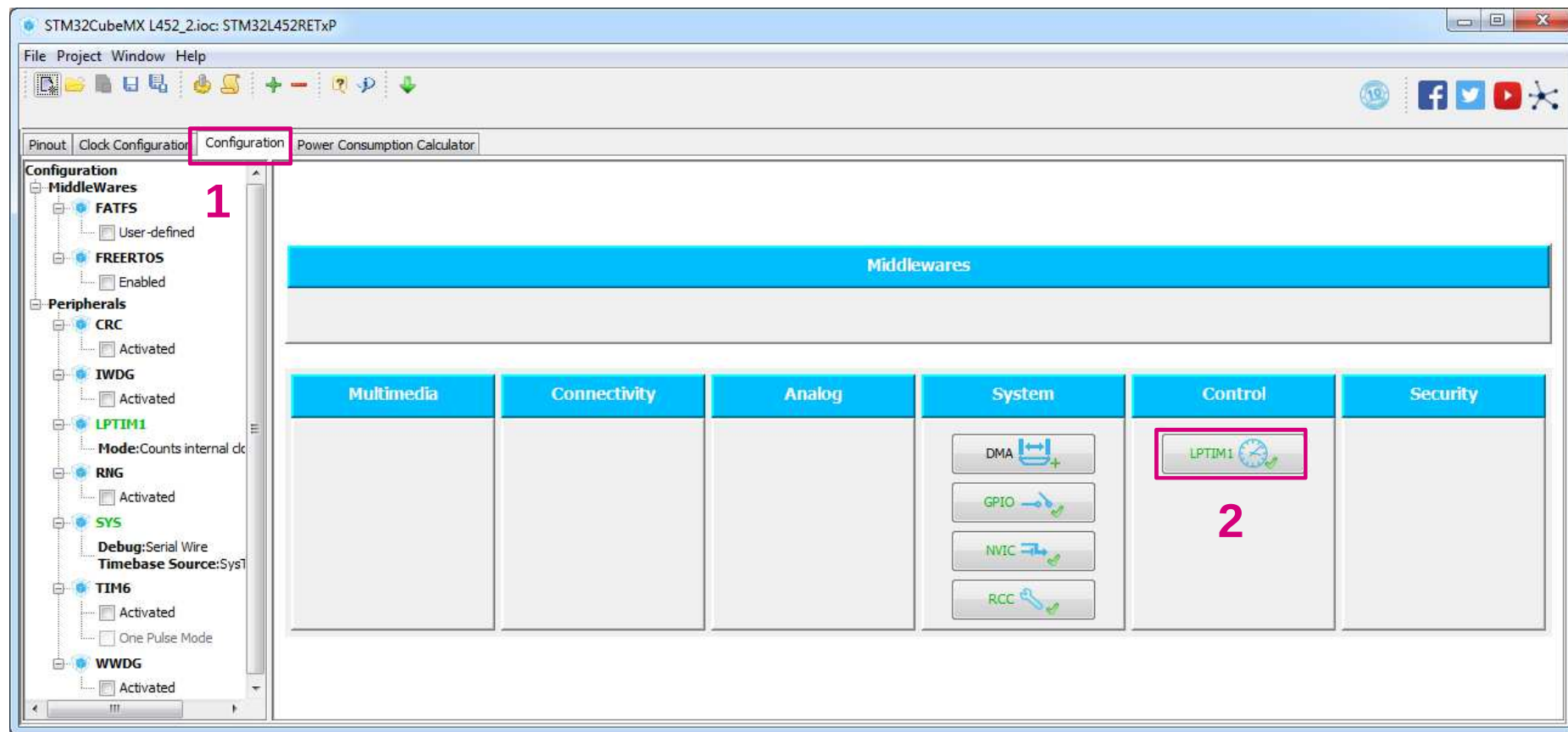
Configuration of pins and peripherals in STM32CubeMX

- Open **Clock Configuration** tab and change clock source, which feeds LPTIM1, to LSI



Configuration of pins and peripherals in STM32CubeMX

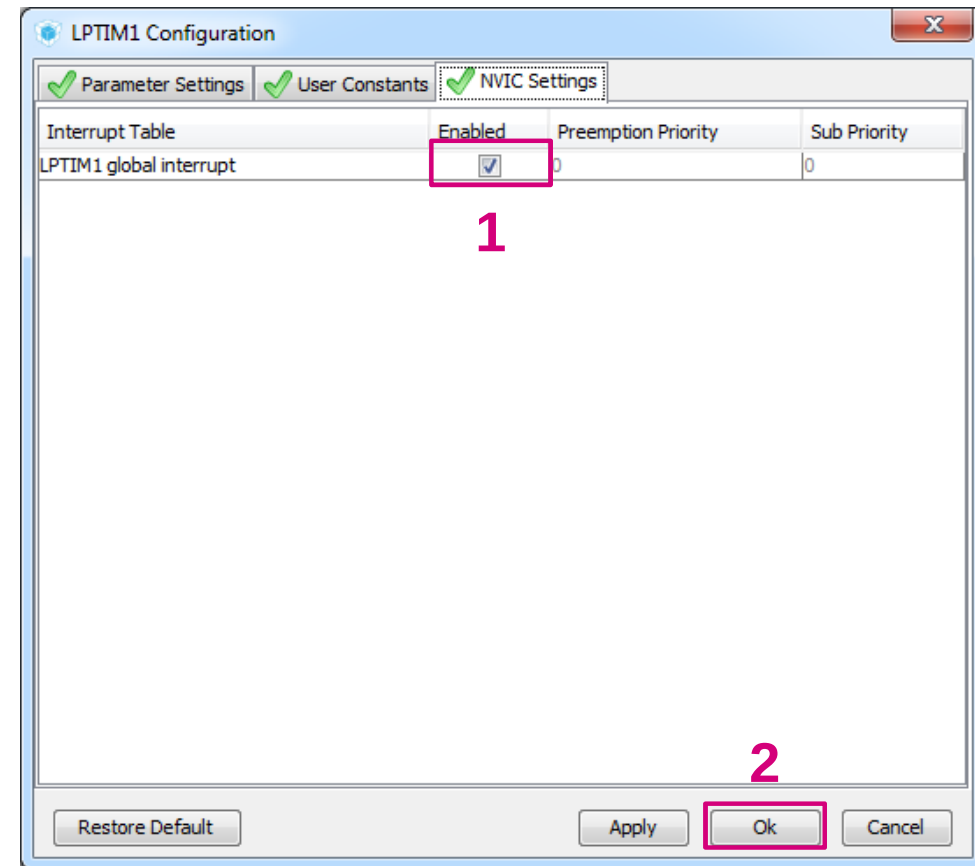
- Open **Configuration** tab and click on **LPTIM1** to configure its details



Configuration of pins and peripherals in STM32CubeMX

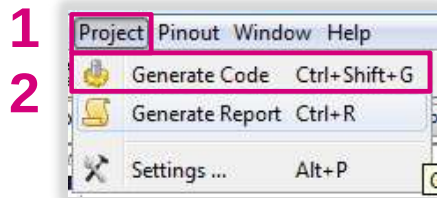
- In **RTC Configuration** window:
 - Open NVIC Settings tab
 - Enable **LPTIM1 global interrupt**
 - Click **Apply** and **OK**

Checkpoint #2
Peripherals
configuration is finished



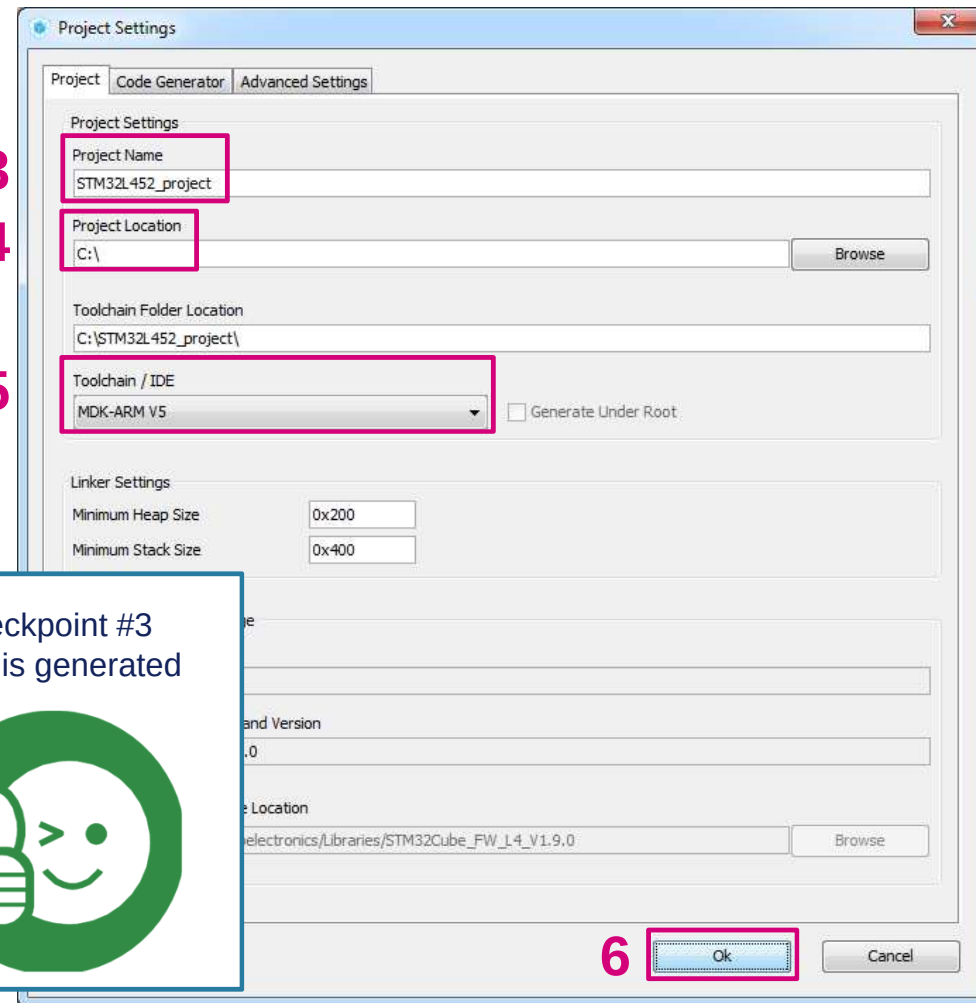
Generating IDE project with code in STM32CubeMX

- From main menu click on **Project -> Generate code**



- Fill in **Project Name** and **Project Location** items
- Select preferred toolchain/IDE as **MDK-ARM V5**

3
4
5

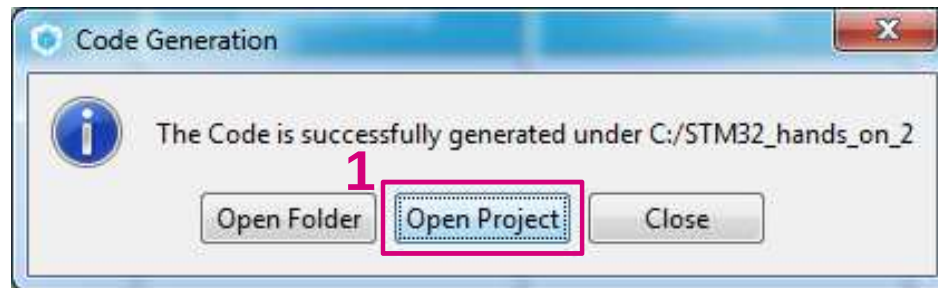


Checkpoint #3
Code is generated

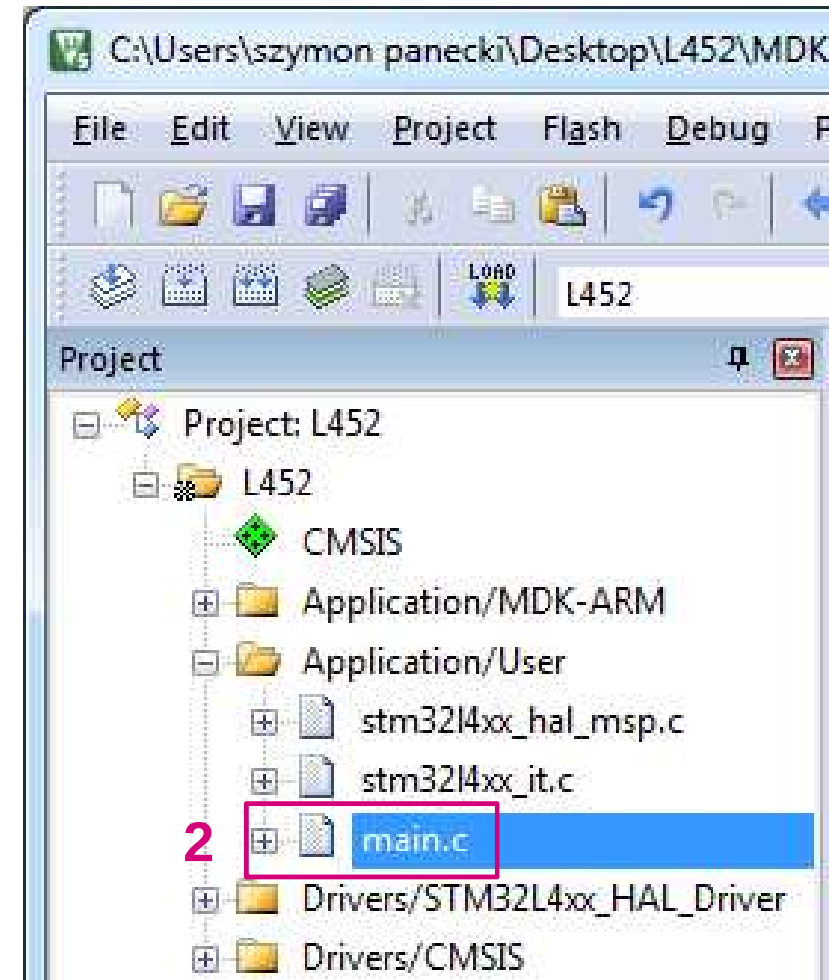


Moving from STM32CubeMX to IDE

- Click on **Open Project** to open it in IDE

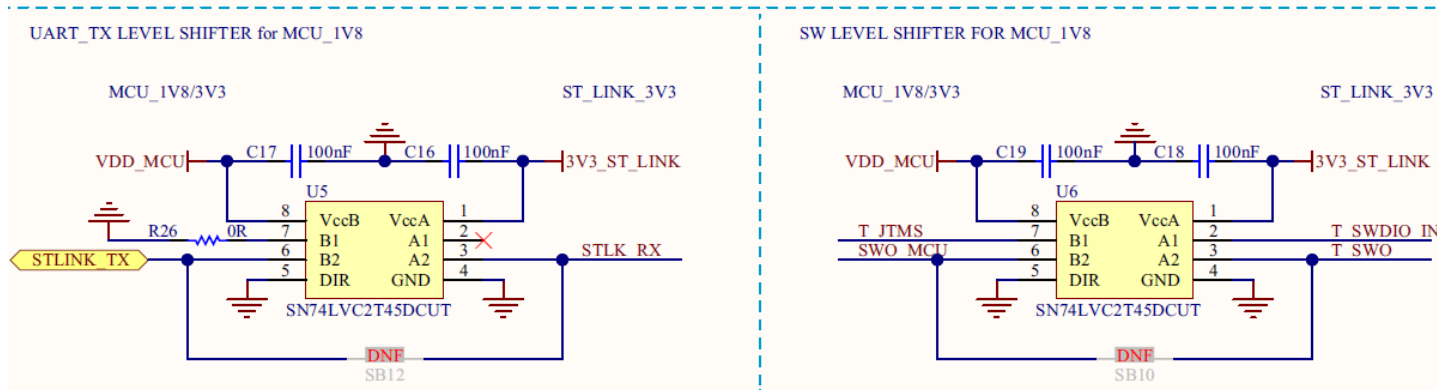


- In IDE open main.c file



User code in main.c

- Code to be added in User Code Section 2:
 1. Enable pull-up resistors on PA2 and PB3 to reduce consumption (from level shifters)



2. Disable GPIOs clock except GPIOB (reduce current consumption)
3. Put Flash memory in power down mode (reduce current consumption)
4. Start LPTIM counter

User code in main.c

- Code to be added in User Code Section 2:

```
/* USER CODE BEGIN 2 */
HAL_PWREx_EnableGPIOPullUp(PWR_GPIO_A, GPIO_PIN_2);
HAL_PWREx_EnableGPIOPullUp(PWR_GPIO_B, GPIO_PIN_3);
HAL_PWREx_EnablePullUpPullDownConfig();

__HAL_RCC_GPIOC_CLK_DISABLE();
__HAL_RCC_GPIOH_CLK_DISABLE();
__HAL_RCC_GPIOA_CLK_DISABLE();
__HAL_RCC_GPIOE_CLK_DISABLE();
__HAL_RCC_GPIOD_CLK_DISABLE();

__HAL_FLASH_POWER_DOWN_ENABLE();
HAL_LPTIM_Counter_Start_IT(&hlptim1, 4);

/* USER CODE END 2 */
```

User code in main.c

- Code to be added in User Code Section 3:

1. Enter STOP2 mode
2. Toggle pin PB13

```
/* USER CODE BEGIN 3 */  
HAL_PWREx_EnterSTOP2Mode(PWR_STOPENTRY_WFI);  
HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_13);  
  
/* USER CODE END 3 */
```

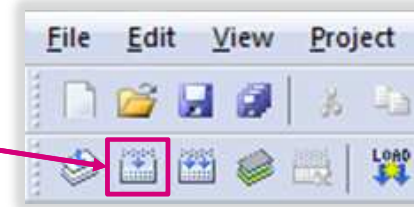
Checkpoint #3
Code development is
finished



Compiling the application, flashing of MCU

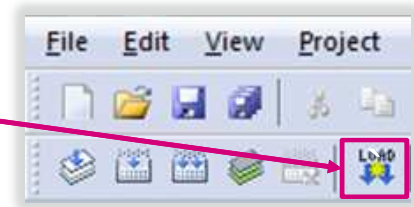
- Compile the code

- By clicking on the **Build** icon
- Or by pressing **F7** keyboard button
- Or by using main menu: **Project -> Build Target**



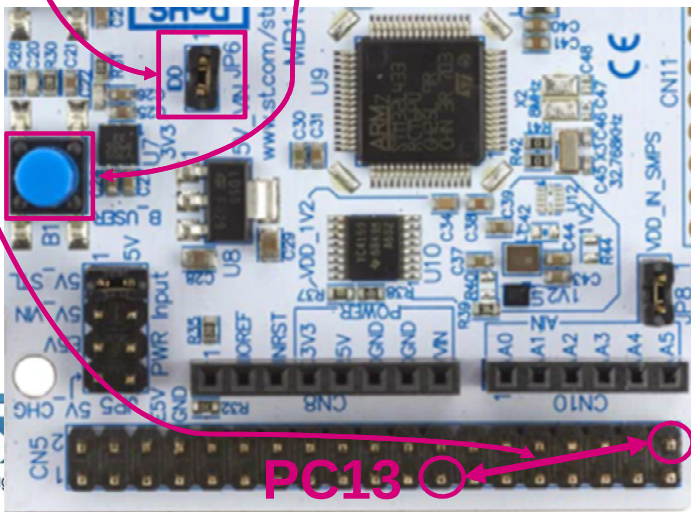
- Flashing

- By clicking **Load** icon
- Or by pressing **F8** keyboard button
- Or by using main menu: **Flash -> Download**

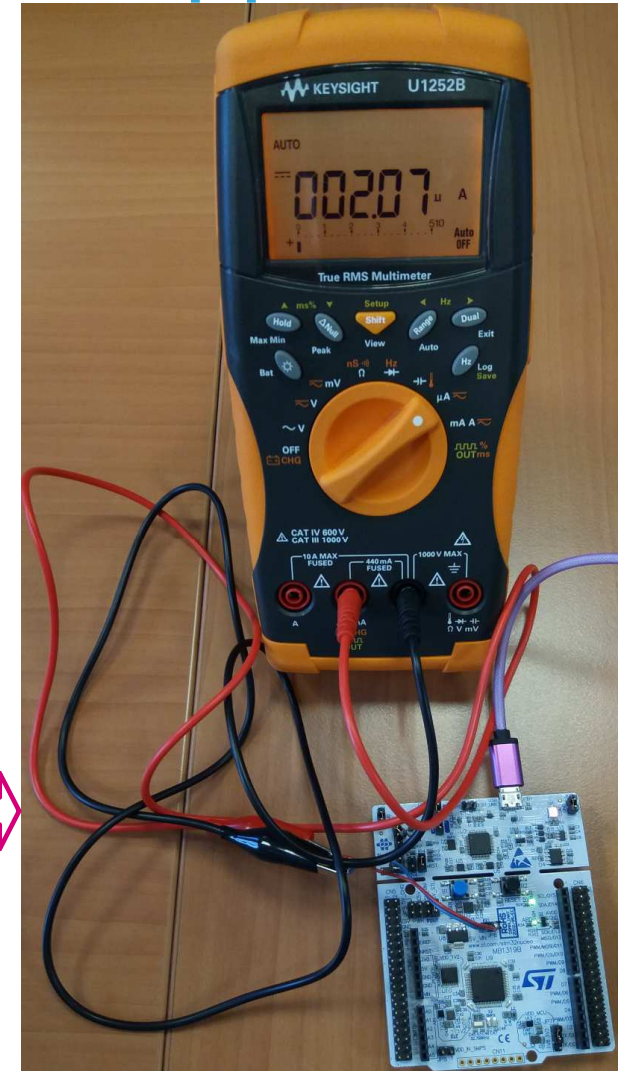


Evaluating of application

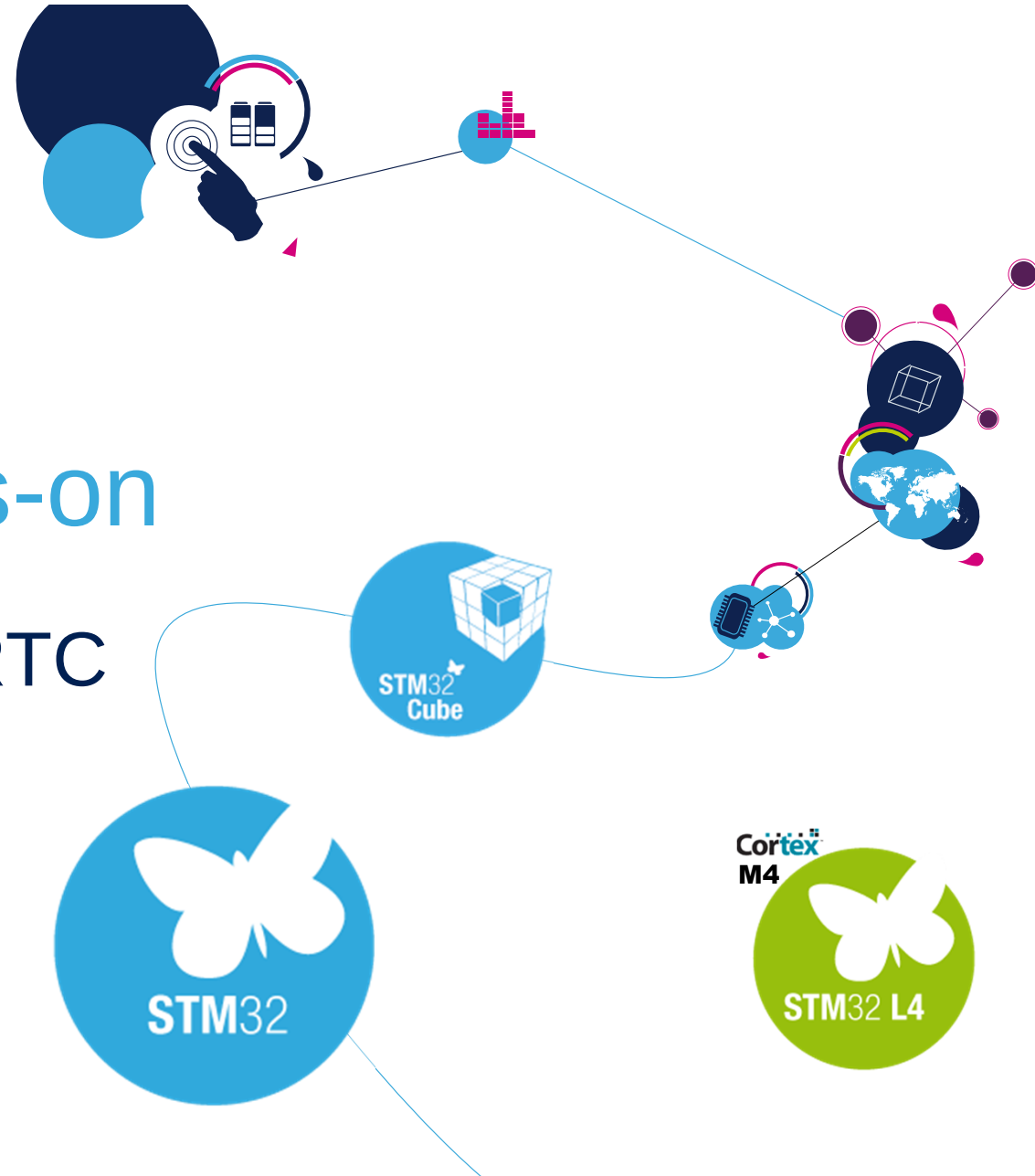
- Connect PC13 (button) to PC0 (LPTIM) for purpose of generating impulses
- Measure the current consumption on JP6
- Press user button 5 times to wake up the MCU for a while (LED will change the state)



JP6 consumption:
2.07uA in STOP2 mode
when LED is switched off



Low-power hands-on STANDBY mode and RTC



STANDBY mode and RTC

- Key information about the application:
 - MCU starts in RUN mode with default clocks configuration (4MHz system clock)
 - MCU enters low-power mode (STANDBY) and periodically wakes for a very short time only to change LED state
- Applications, which can benefit from combination of STANDBY mode and RTC:
 - All applications, which stays in low power mode for a long time and require extremely low-current consumption with wake-up capability on a regular time base, for example sensor devices and data loggers

Available peripherals

GPIO
DMA
FSMC
OSPI
BOR
PVD, PVM
LCD
USB OTG
USART
LP UART
I2C 1 / I2C 2
I2C 3
SPI
CAN
SDMMC
SWPMI
SAI
DFSDM
ADC
DAC
OPAMP
COMP
Temp Sensor
Timers
LPTIM 1
LPTIM 2
IWDG
WWDG
Systick Timer
Touch Sens
RNG
AES
CRC

I/Os can be configured
w/ or w/o pull-up
w/ or w/o pull-down

Zzz

Cortex M4

Main regulator (MR)

Flash
memory
(1 Mbyte)

SRAM 1
(96 Kbytes)

SRAM 2
(32 Kbytes)

Low Power regulator
(LPR)

Wake-up
event

NRST
BOR
RTC + Tamper
IWDG
5 WKUP pins

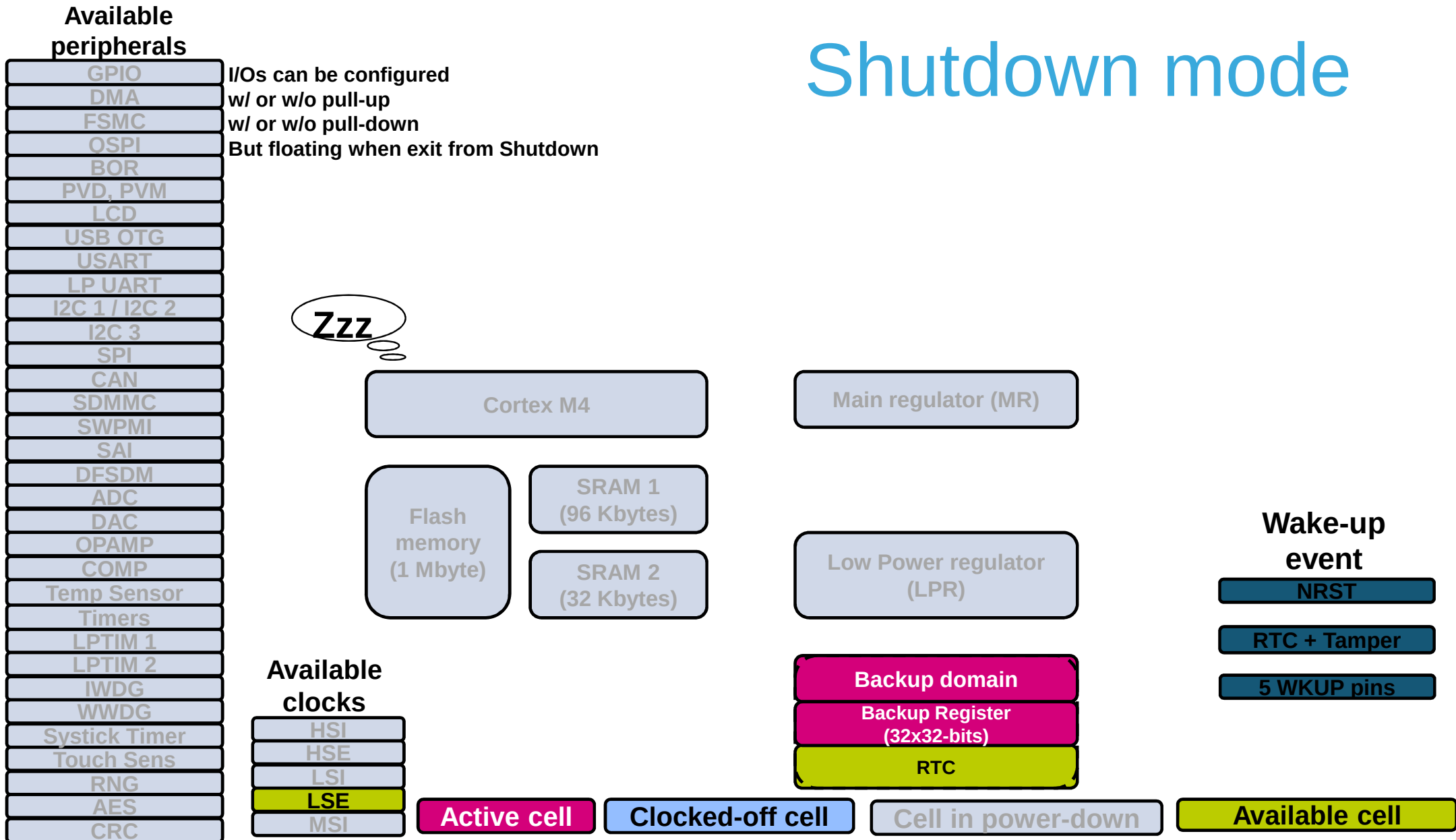
Available
clocks

HSI
HSE
LSI
LSE
MSI

Backup domain
Backup Register (32x32-bits)
RTC

Active cell	Clocked-off cell	Cell in power-down	Available cell
--------------------	-------------------------	---------------------------	-----------------------

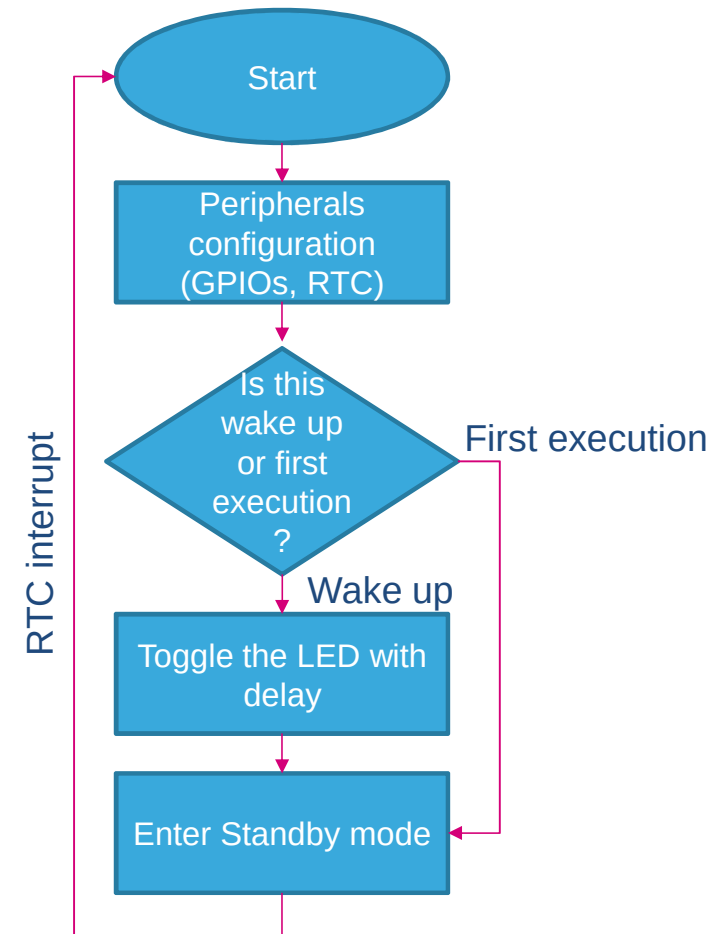
Shutdown mode



Application Description and block diagram

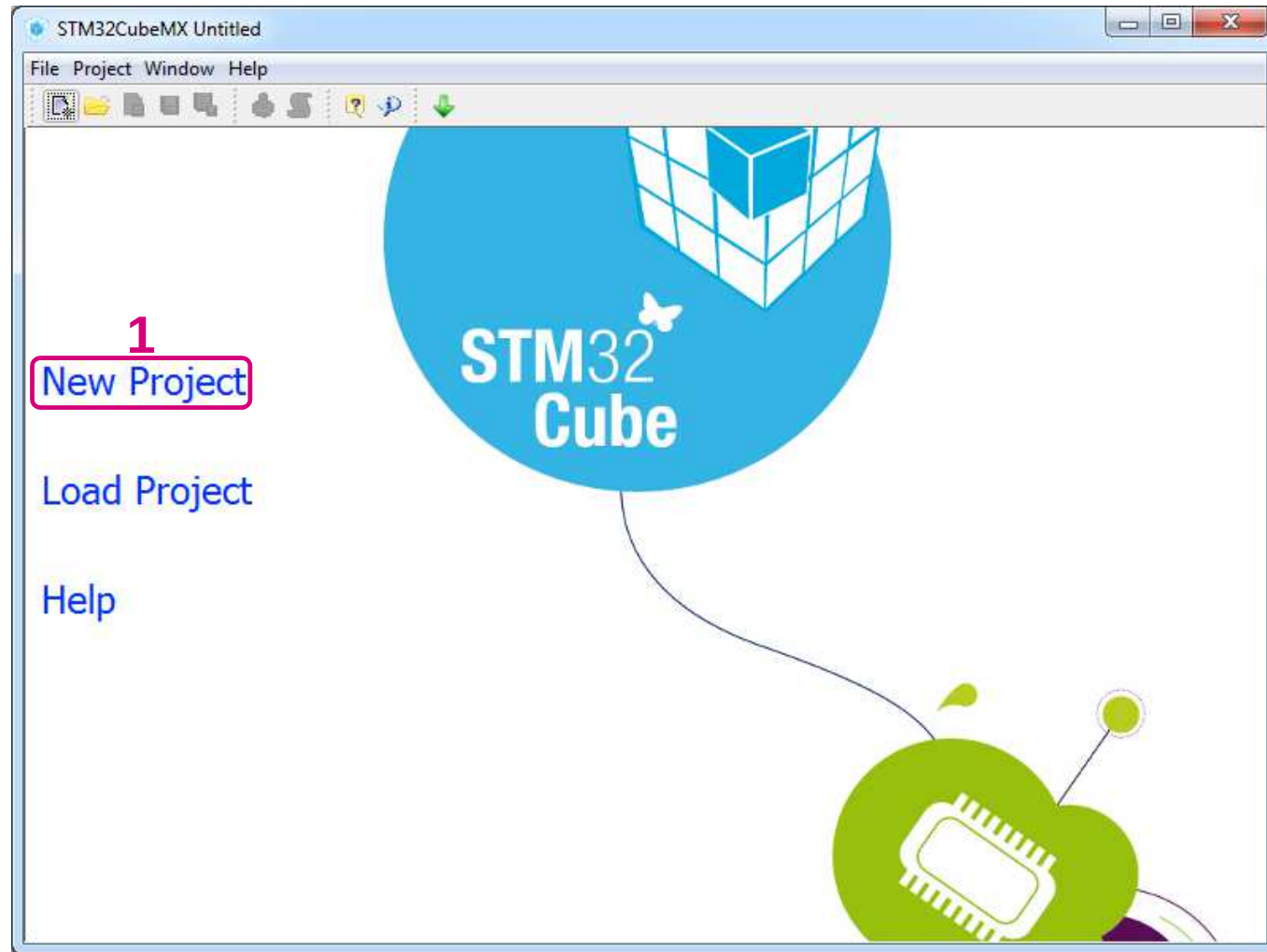
■ Application's behaviour:

1. Configuration of PB13 GPIO as output to control LED
2. Configuration of RTC to generate periodic interrupt
3. Enable pull-ups on PA2 and PB3 to decrease current consumption (action is specific for Nucleo-P)
4. Identification of wake-up or first application execution
5. Toggle the LED (PB13)
6. Enter Standby mode (without SRAM retention)
7. Wait for RTC interrupt, then reset and back to point 1



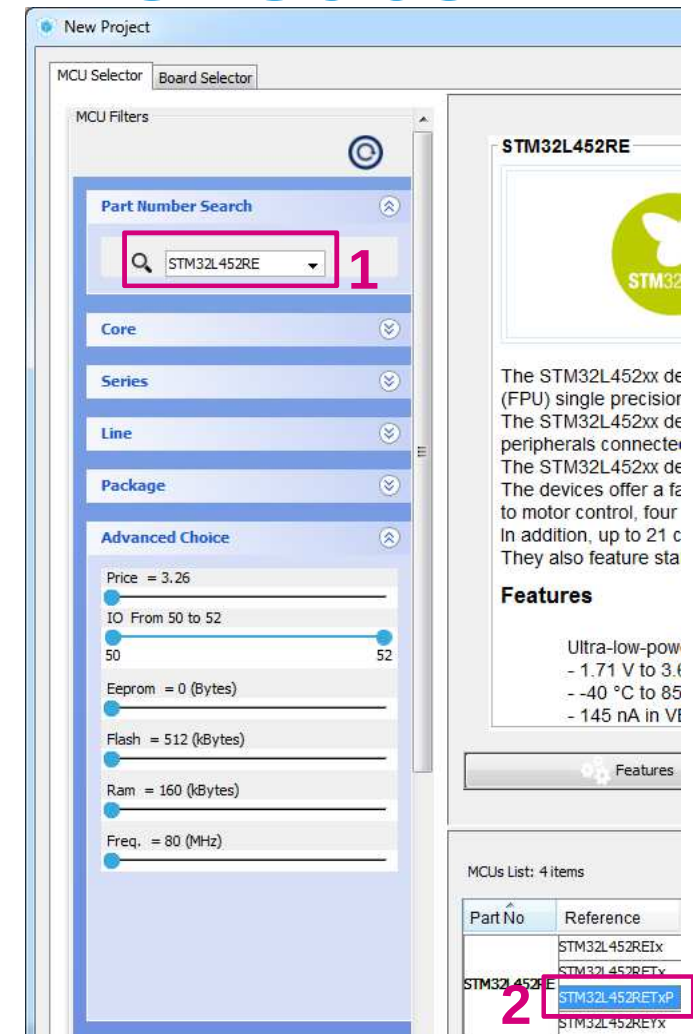
Creating of a new project in STM32CubeMX

- Open STM32CubeMX
- Create a new project by clicking on **New Project**

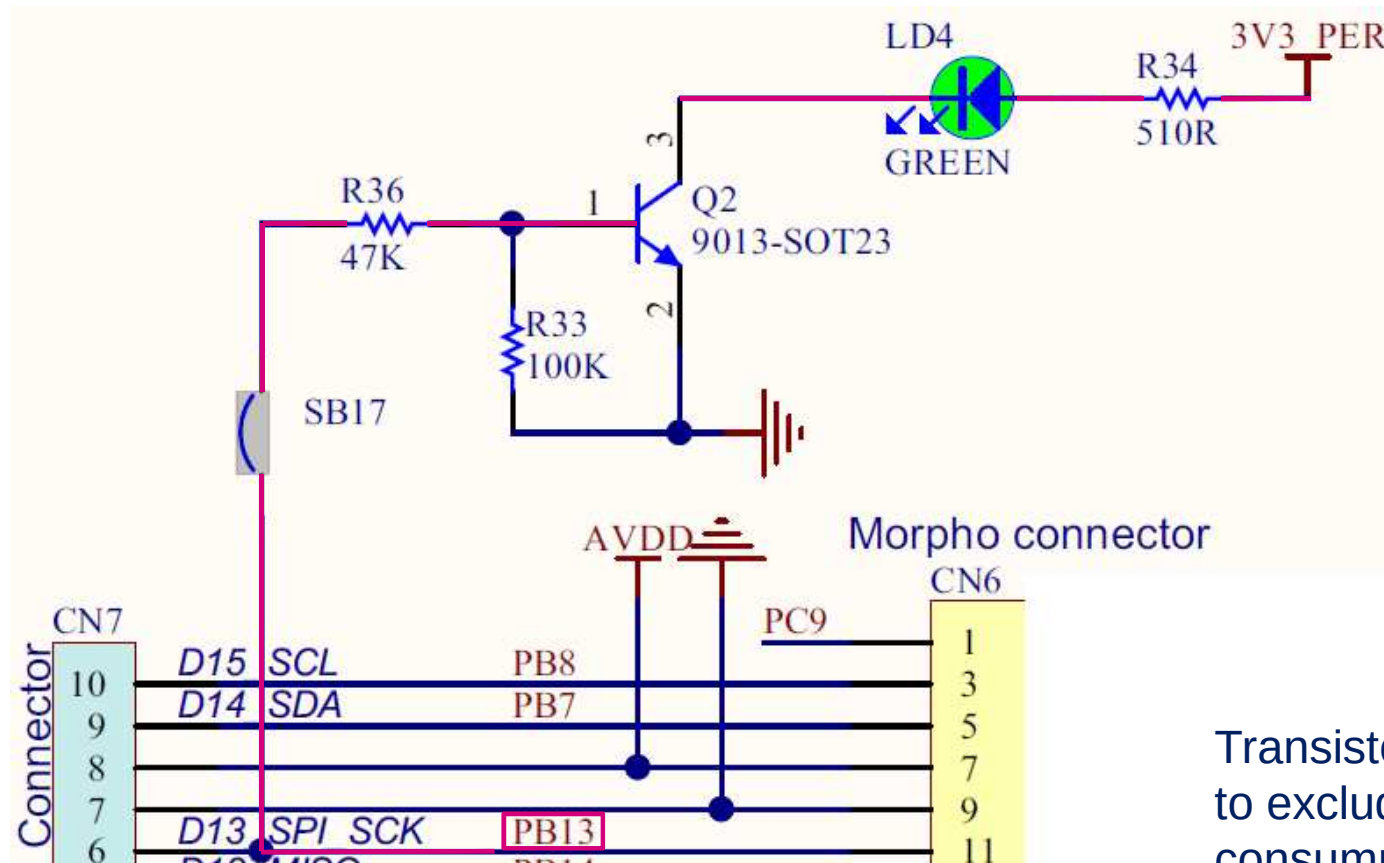


Creating of a new project in STM32CubeMX

- Select proper device. To shrink the list type **STM32L452RE** in Part Number Search box
- From MCUs list select device: **STM32L452RETxP**
- Double click on selected MCU to create a project



User LED on STM32L452RET6P Nucleo kit

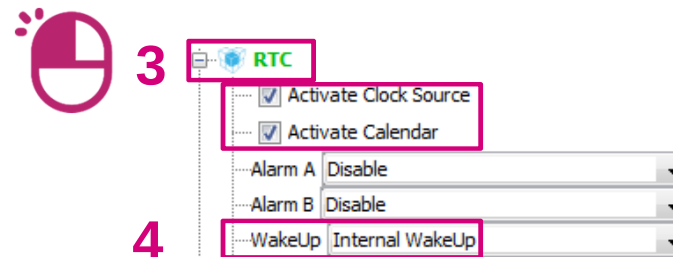
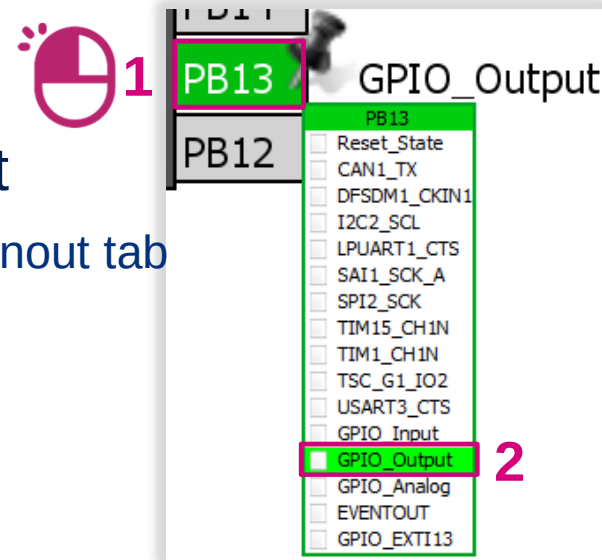


Transistor switch was used to exclude LED's current consumption from MCU current consumption

Configuration of pins and peripherals in STM32CubeMX

1. MCU's resources

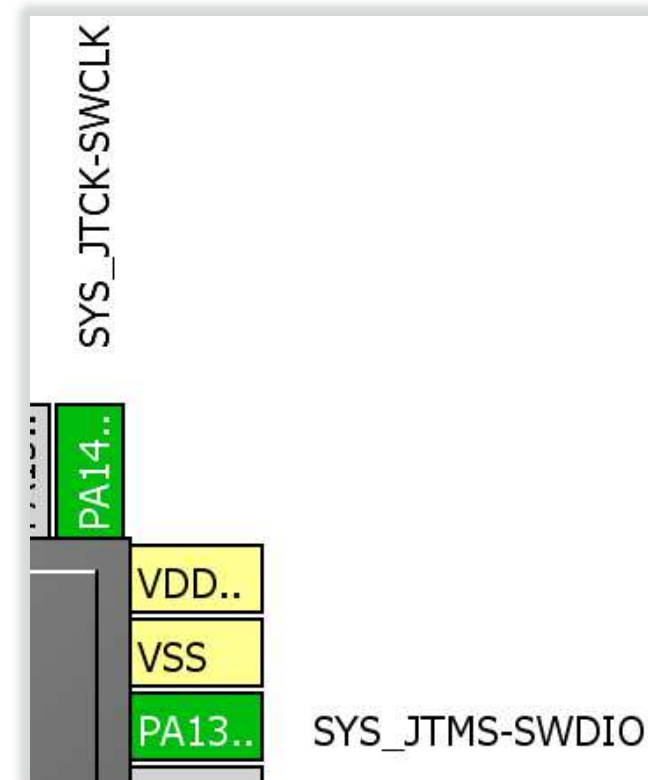
- Configure PB13 (LED) as output
 - Left mouse button click on **PB13** in Pinout tab
 - From menu select **GPIO_Output**
- Enable RTC
 - Click on **RTC** in Pinout tab
 - Select **Activate Clock Source**
 - Select **Activate Calendar**
 - Select **Internal Wakeup**



Configuration of pins and peripherals in STM32CubeMX

2. MCU's resources

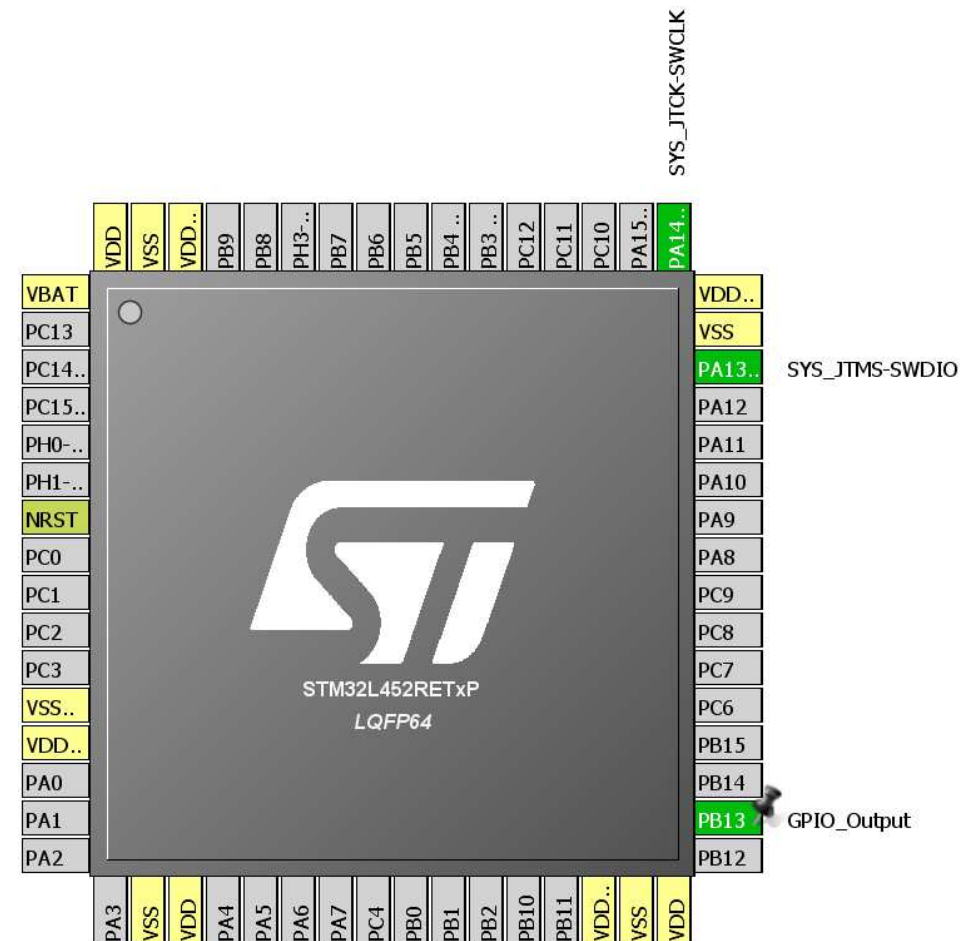
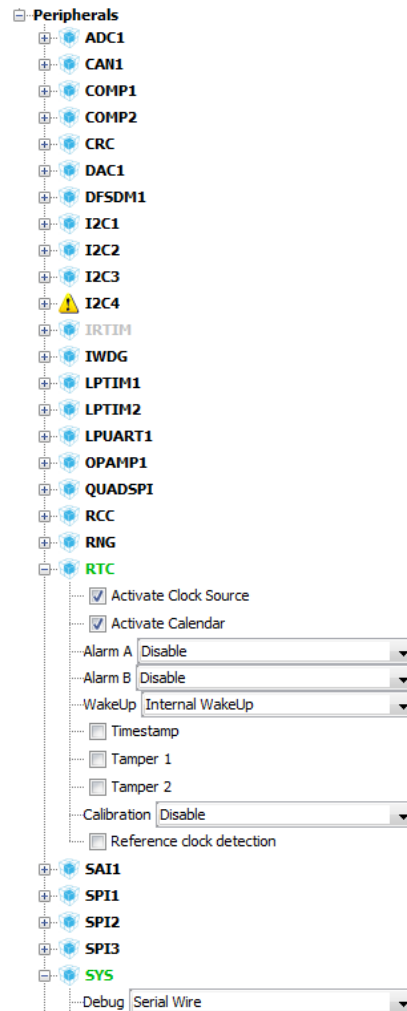
- Enable SWD debug interface
 - Click on **SYS** in Pinout tab
 - Select **Debug Serial Wire**



Configuration of pins and peripherals in STM32CubeMX

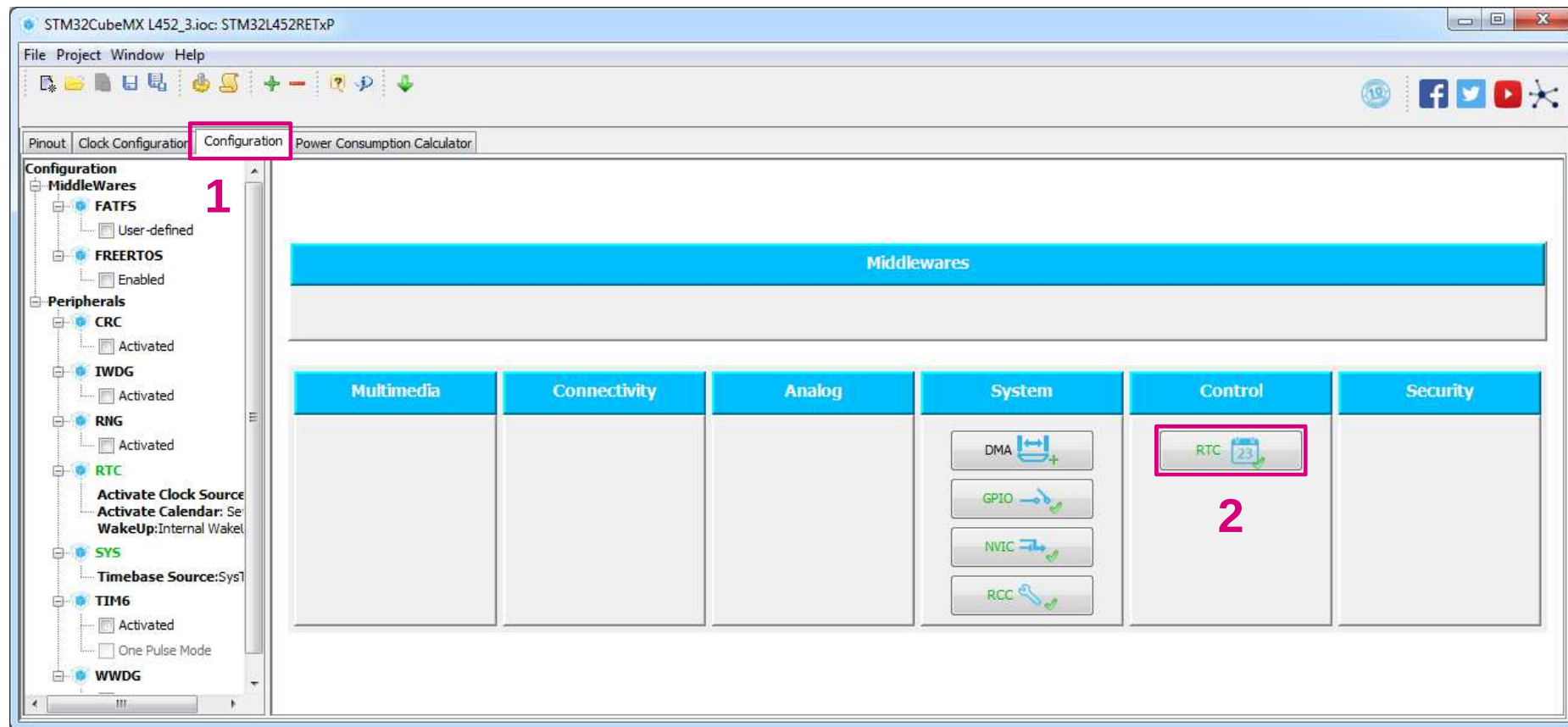
- Finally all needed peripherals are enabled:
 - GPIO
 - RTC
 - SWD

Checkpoint #1
Pinout configuration is finished



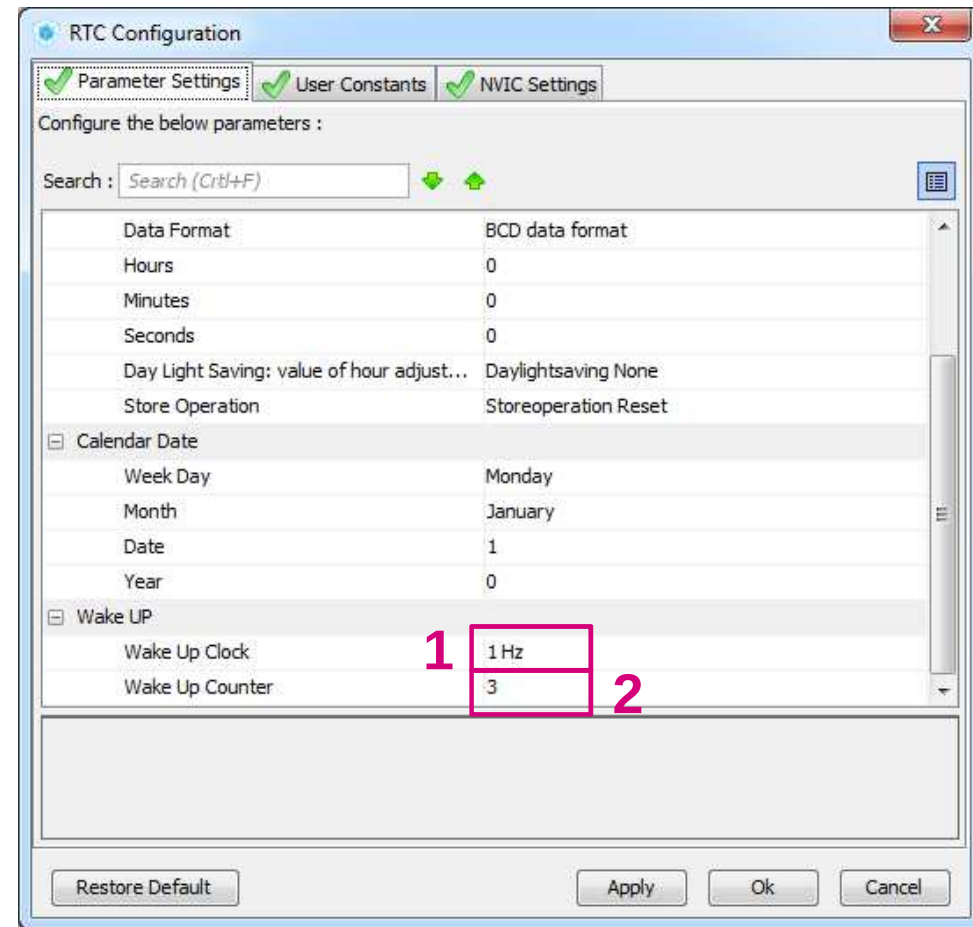
Configuration of pins and peripherals in STM32CubeMX

- Open **Configuration** tab and click on **RTC** to configure its details



Configuration of pins and peripherals in STM32CubeMX

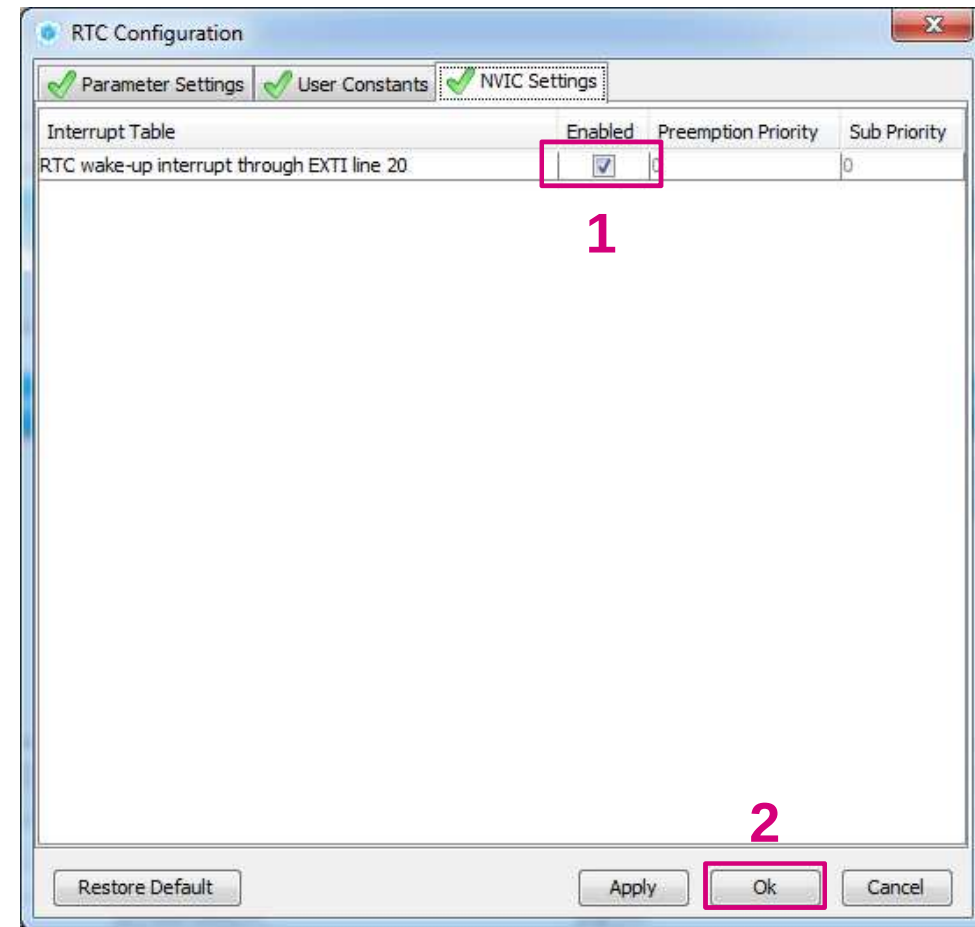
- In **RTC Configuration** window:
 - Change **Wake Up Clock** to **1Hz**
 - Change **Wake Up Counter** to **3**



Configuration of pins and peripherals in STM32CubeMX

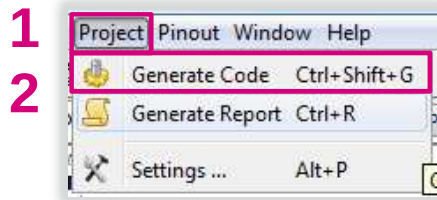
- In **RTC Configuration** window:
 - Open NVIC Settings tab
 - Enable **RTC wake-up interrupt**
 - Click **Apply** and **OK**

Checkpoint #2
Peripherals
configuration is finished



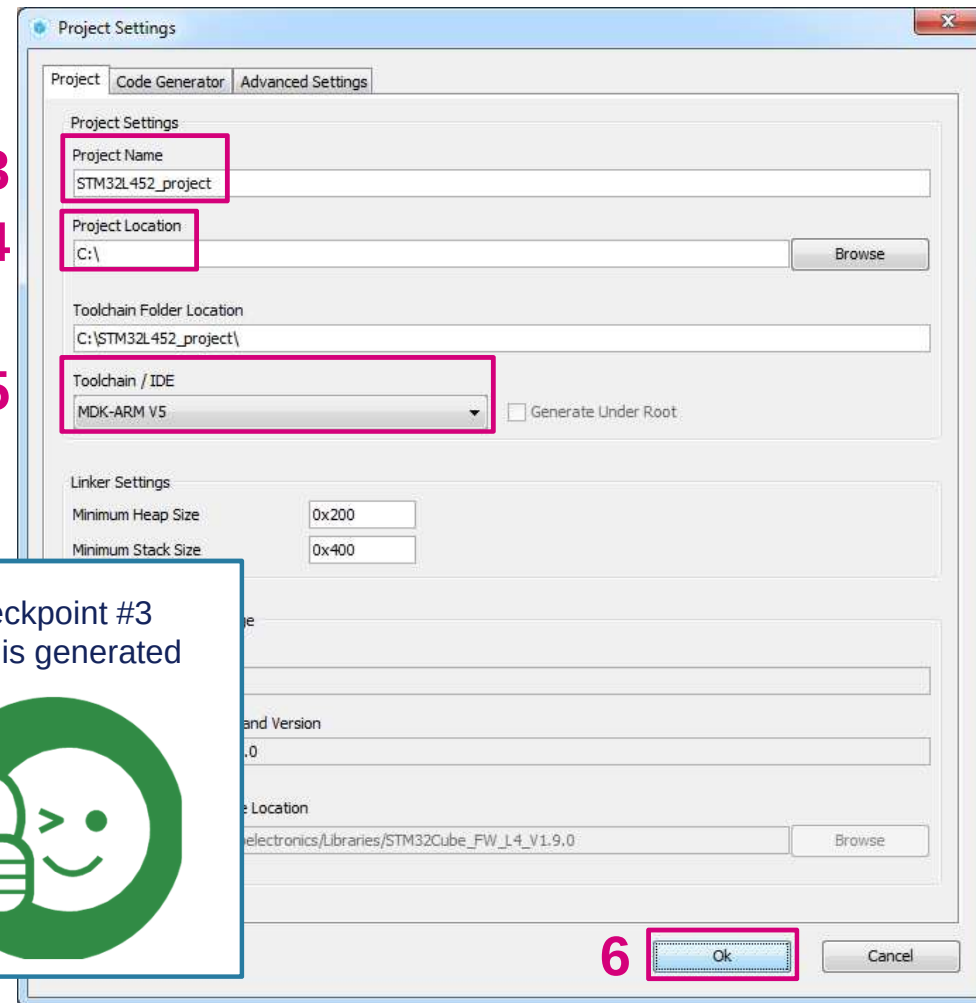
Generating IDE project with code in STM32CubeMX

- From main menu click on **Project -> Generate code**



- Fill in **Project Name** and **Project Location** items
- Select preferred toolchain/IDE as **MDK-ARM V5**

3
4
5

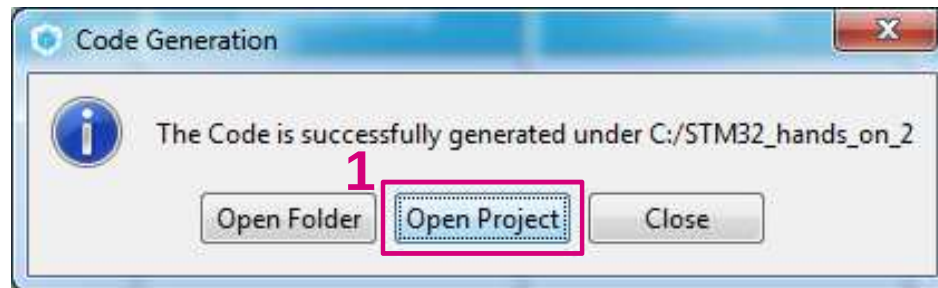


Checkpoint #3
Code is generated

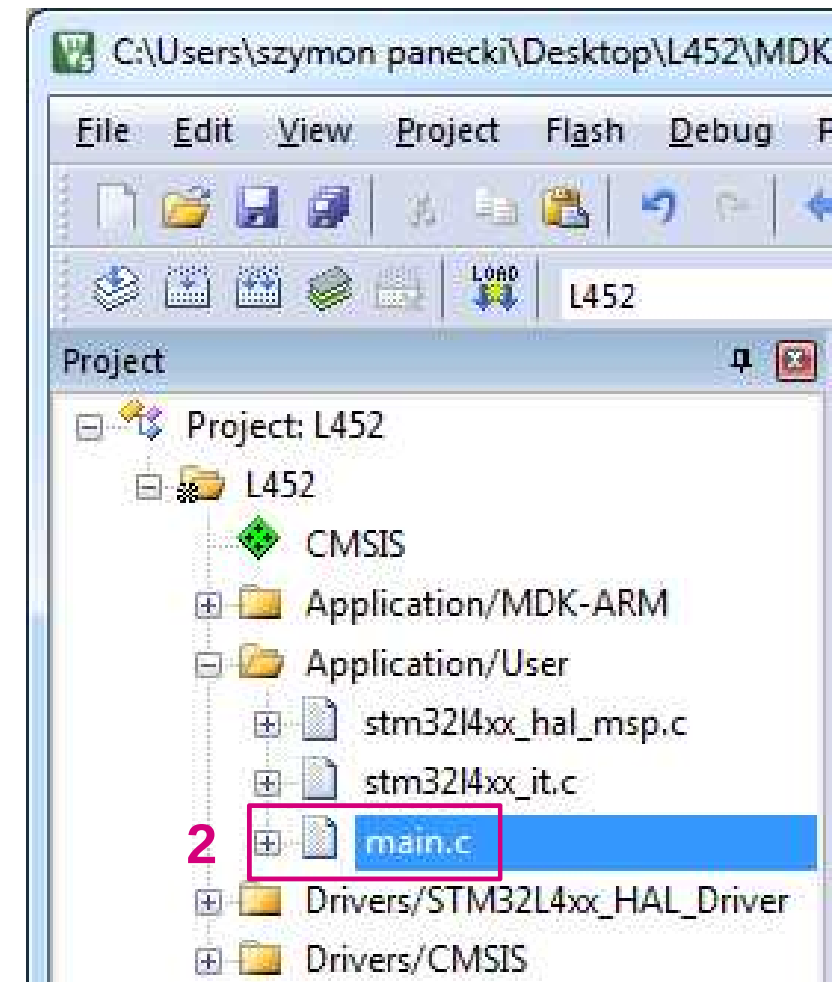


Moving from STM32CubeMX to IDE

- Click on **Open Project** to open it in IDE

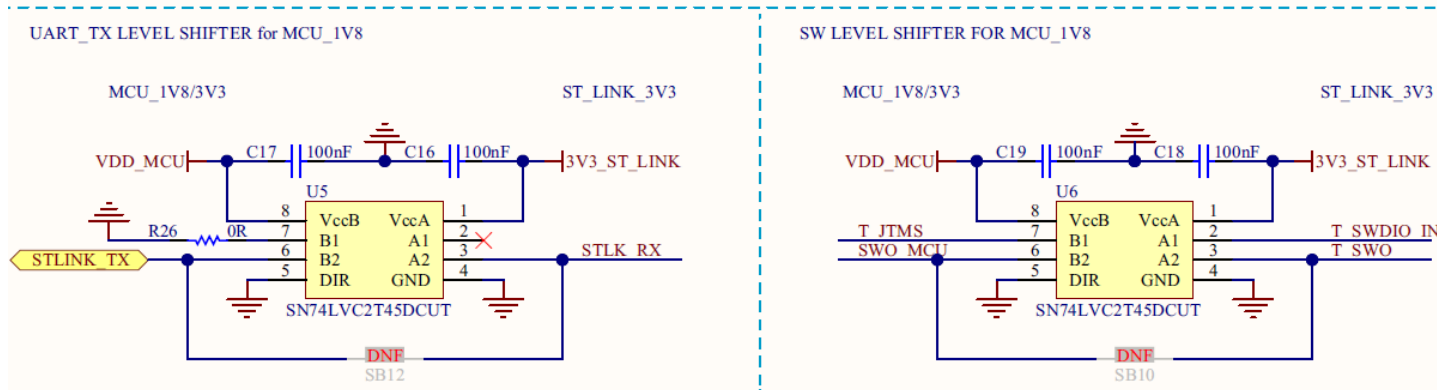


- In IDE open main.c file



User code in main.c

- Code to be added in User Code Section 2:
 1. Enable pull-up resistors on PA2 and PB3 to reduce consumption (from level shifters)



2. Check Standby flag to identify if MCU waking up or if it is executing the application for a first time (in both cases MCU starts from reset state)
 1. Clear Standby flag
 2. Toggle PB13 to change LED state
3. Make a delay to keep MCU in RUN mode for a while
4. Enter Standby mode

User code in main.c

- Code to be added in User Code Section 2:

```
/* USER CODE BEGIN 2 */
HAL_PWREx_EnableGPIOPullUp(PWR_GPIO_A, GPIO_PIN_2);
HAL_PWREx_EnableGPIOPullUp(PWR_GPIO_B, GPIO_PIN_3);
HAL_PWREx_EnablePullUpPullDownConfig();

if (__HAL_PWR_GET_FLAG(PWR_FLAG_SB) != RESET)
{
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_SB);
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13, GPIO_PIN_SET);
}

HAL_Delay(500);
HAL_PWR_EnterSTANDBYMode();

/* USER CODE END 2 */
```

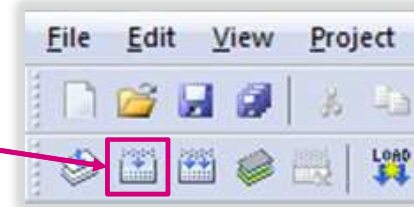
Checkpoint #4
Code development is
finished



Compiling the application, flashing of MCU

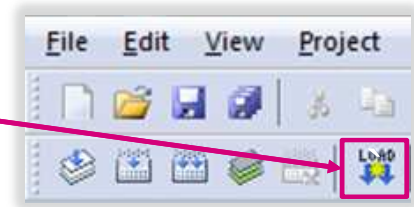
- Compile the code

- By clicking on the **Build** icon
- Or by pressing **F7** keyboard button
- Or by using main menu: **Project -> Build Target**



- Flashing

- By clicking **Load** icon
- Or by pressing **F8** keyboard button
- Or by using main menu: **Flash -> Download**



Evaluating of application

- Measure the current consumption on JP6

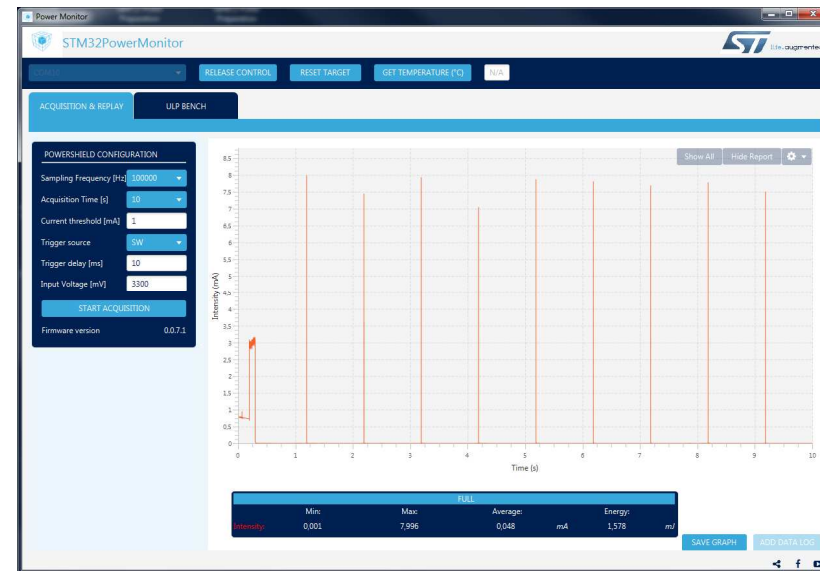


JP6 consumption:
0.9uA in Standby mode
when LED is switched off



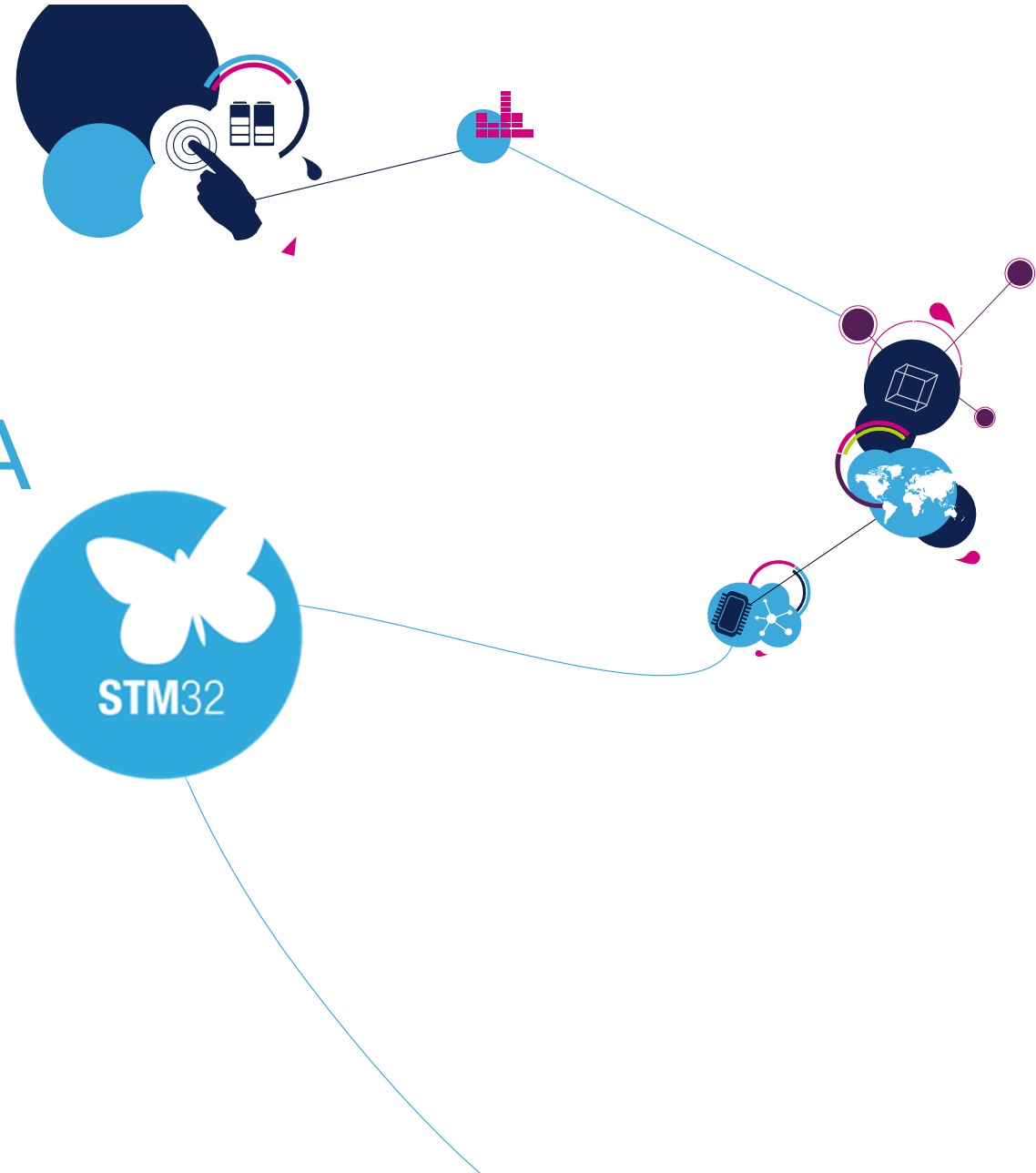
Evaluating of application with STM32 Power Shield

1. Remove CN2, JP6 and JP4, close JP8
2. Conect STM32Nucleo kit with STM32 Power Shield
3. Open STM32CubeMonitor-Power application



Conclusions, Q&A

STM32 Euro 2017



What we have learnt

- **Portfolio** and **roadmap** of STM32L4/STM32L4+ and tools
- Developing from scratch an application, which uses **RUN** mode and **SMPS** (applicable for all kind of low-power applications)
- Developing from scratch an application, which uses **STOP** mode and **LPTIM** (applicable for water metering applications)
- Developing from scratch an application, which uses **STANDBY** mode and **RTC** (applicable for sensor device or data logger applications)

Survey

https://www.surveymonkey.com/r/STM32Euro2017_STM32L4_ULP

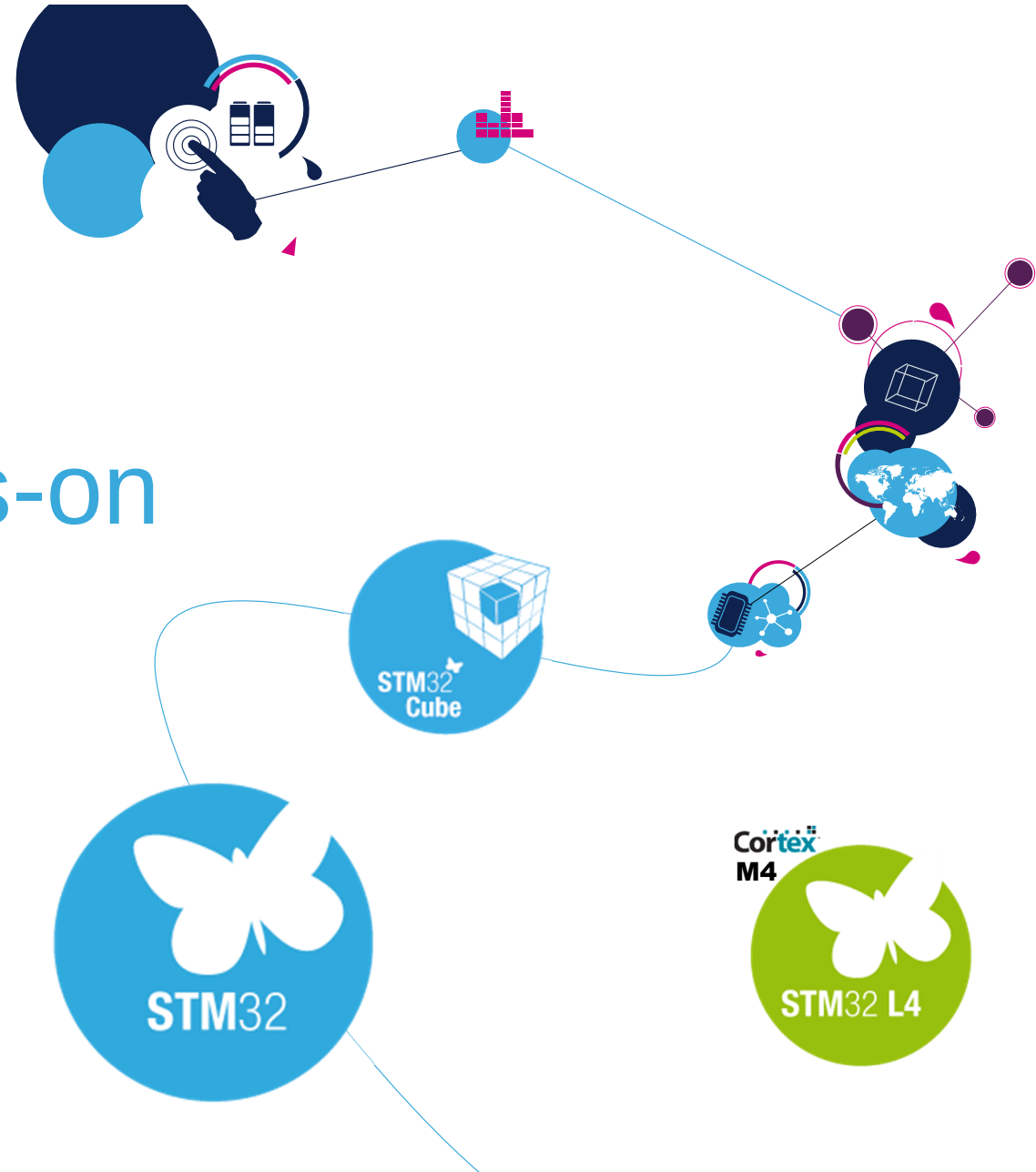
Thank you



Low-power hands-on

Extra material

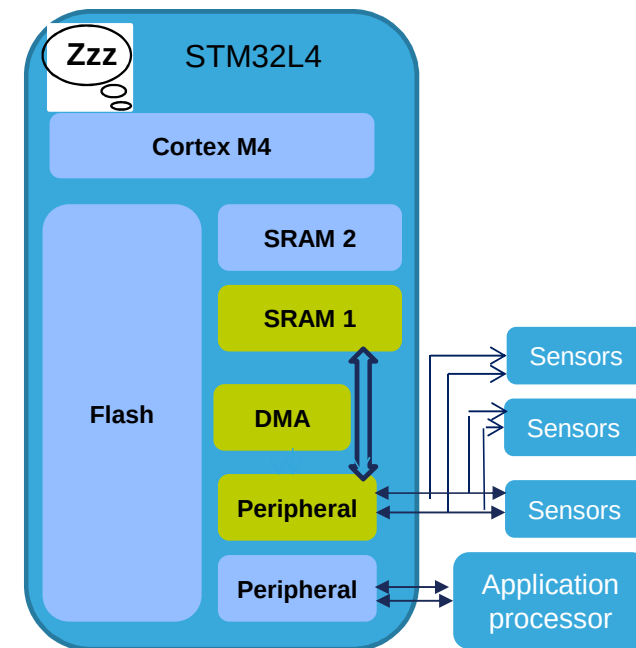
SLEEP mode and BAM



Batch Acquisition mode (BAM) 104

Optimized mode for transferring data with peripherals,
while the rest of the device is in low power.

1. Only the needed peripheral + 1 DMA + 1 SRAM are configured with clock enabled in Sleep mode
2. Flash memory is put in Power-down mode and Flash clock is gated off during Sleep mode
3. Enter either Sleep or Low-power sleep mode



SLEEP mode and BAM

- Key information about the application:
 - MCU starts in RUN mode with default clocks configuration (4MHz system clock)
 - MCU enters low-power mode (SLEEP) with Flash switched off. ADC is triggered by TIM to acquire data from internal temperature sensor and DMA transfers it to SRAM memory without waking up the MCU.
- Applications, which can benefit from combination of STANDBY mode and RTC:
 - All applications, which stays in active mode for a long period of time (to constantly get data from outside world like sensors) and need to consume as low as possible current, for example fitness trackers

Available peripherals

GPIO
DMA
FSMC
QUADSPI
BOR
PVD, PVM
LCD
USB OTG
USART
LP UART
I2C 1 / I2C 2
I2C 3
SPI
CAN
SDMMC
SWPMI
SAI
DFSDM
ADC
DAC
OPAMP
COMP
Temp Sensor
Timers
LPTIM 1
LPTIM 2
IWDG
WWDG
Systick Timer
Touch Sens
RNG
AES
CRC

Zzz

Cortex M4

Flash
memory
(1 Mbyte)

SRAM 1
(96 Kbytes)

SRAM 2
(32 Kbytes)

Main regulator (MR)

Range 1 (up to 80 MHz)

Range 2 (up to 26 MHz)

Low Power regulator
(LPR)
up to 2 MHz

Available clocks

HSI
HSE
LSI
LSE
MSI

Active cell

Clocked-off
cell

Cell in power-
down

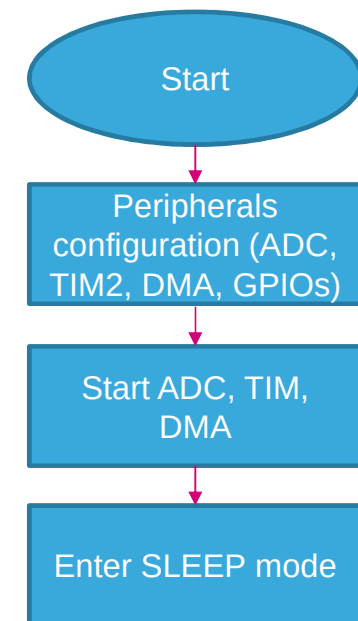
Available
Periph and clock

Sleep mode

Application Description and block diagram

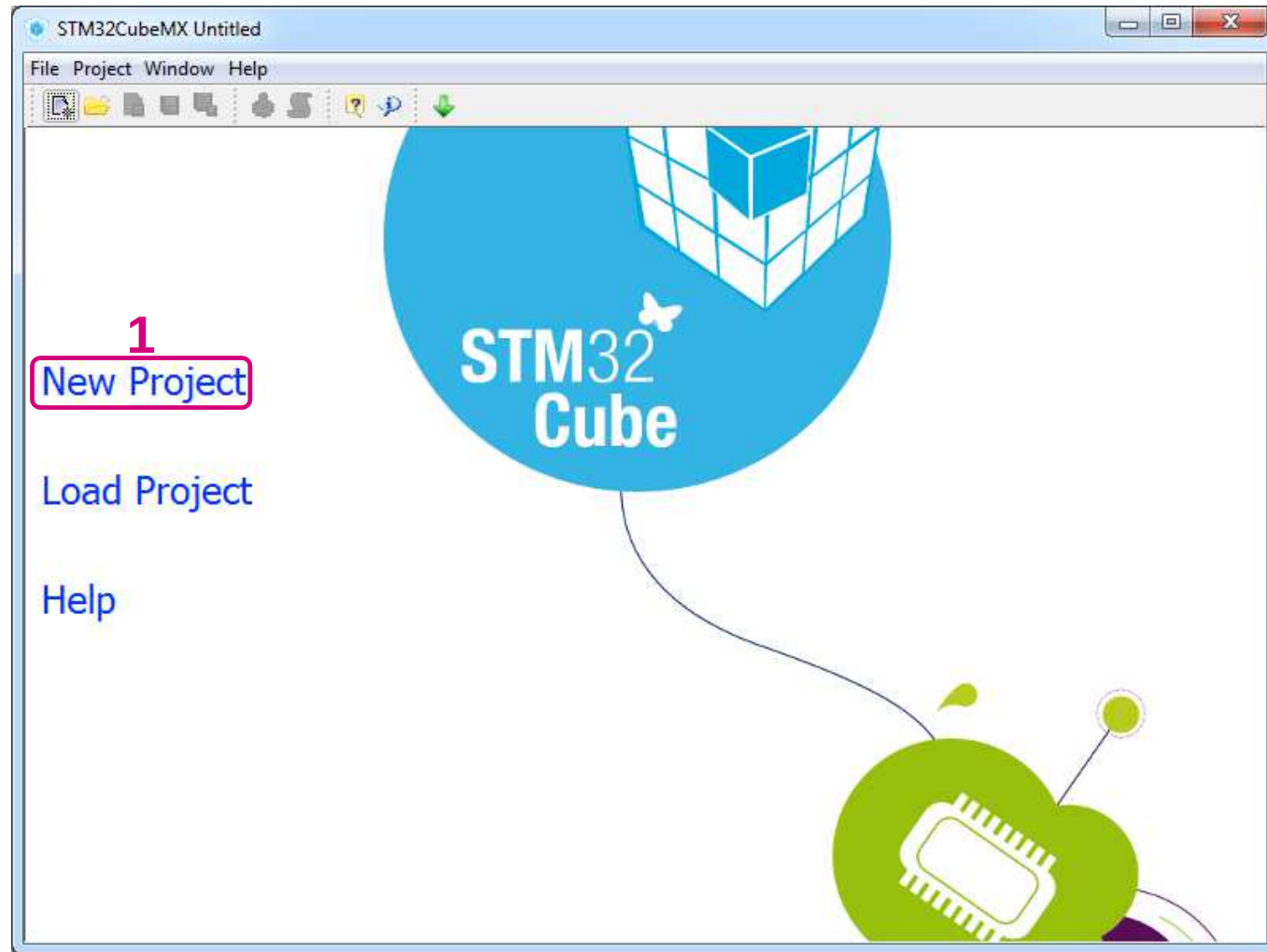
■ Application's behaviour:

1. Configuration of peripherals
 - a. ADC (internal temperature sensor)
 - b. TIM2 (to trigger ADC)
 - c. DMA (to transfer ADC data to SRAM)
2. Enable pull-ups on PA2 and PB3 to decrease current consumption (action is specific for Nucleo-P)
3. Disable clocks for GPIOs and put Flash in power down mode
4. Calibrate ADC, start TIM2, Start ADC with DMA support
5. Enter SLEEP mode. Peripherals (DMA, TIM, ADC) continues to operate



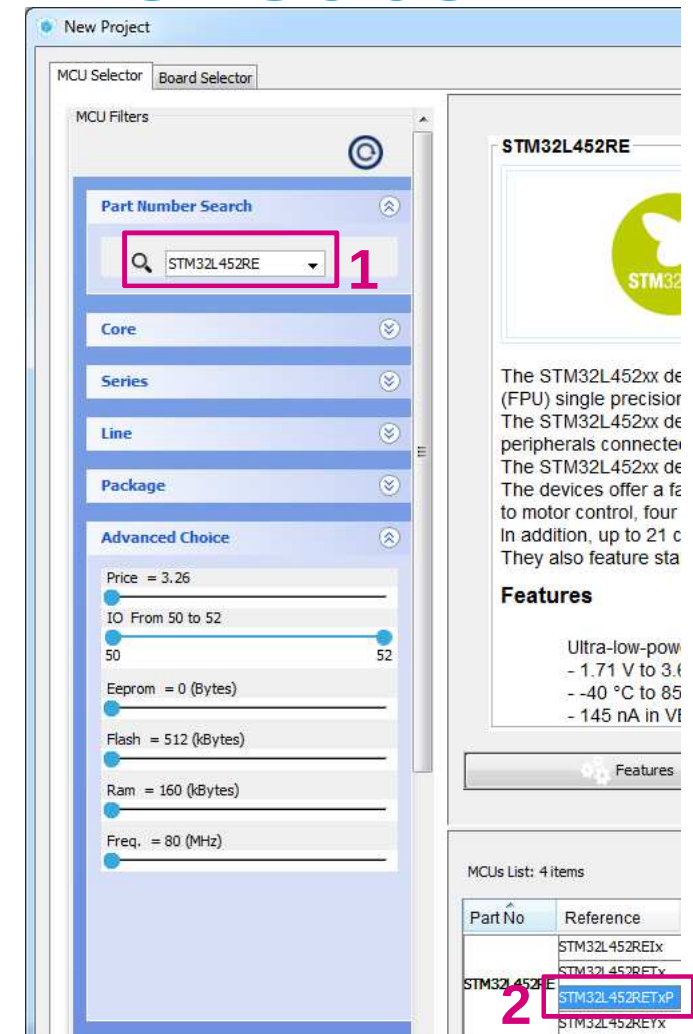
Creating of a new project in STM32CubeMX

- Open STM32CubeMX
- Create a new project by clicking on **New Project**



Creating of a new project in STM32CubeMX

- Select proper device. To shrink the list type **STM32L452RE** in Part Number Search box
- From MCUs list select device: **STM32L452RETxP**
- Double click on selected MCU to create a project

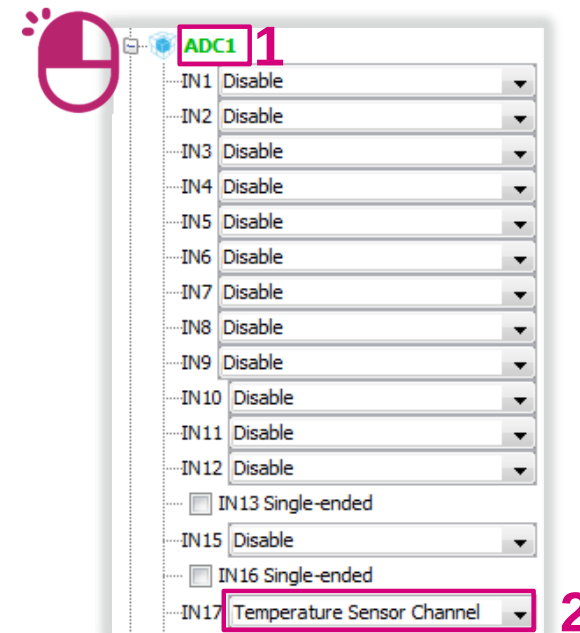


Configuration of pins and peripherals in STM32CubeMX

1. MCU's resources

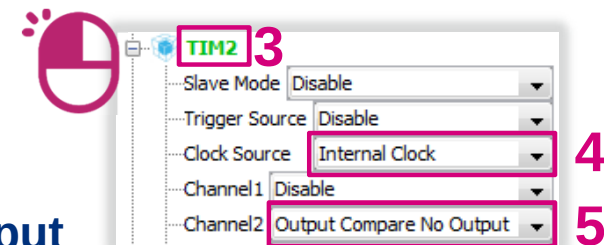
- Configure ADC1

- Click on **ADC1** in Pinout tab
- Enable **IN17** as **Temperature Sensor Channel**



- Configure TIM2

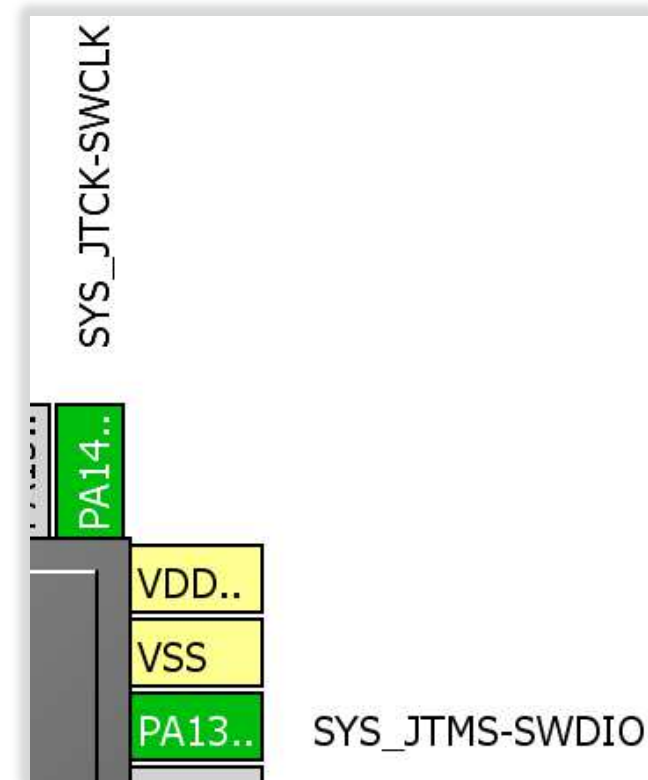
- Click on **TIM2** in Pinout tab
- Change **Clock source** to **Internal Clock**
- Enable **Channel2** as **Output Compare No Output**



Configuration of pins and peripherals in STM32CubeMX

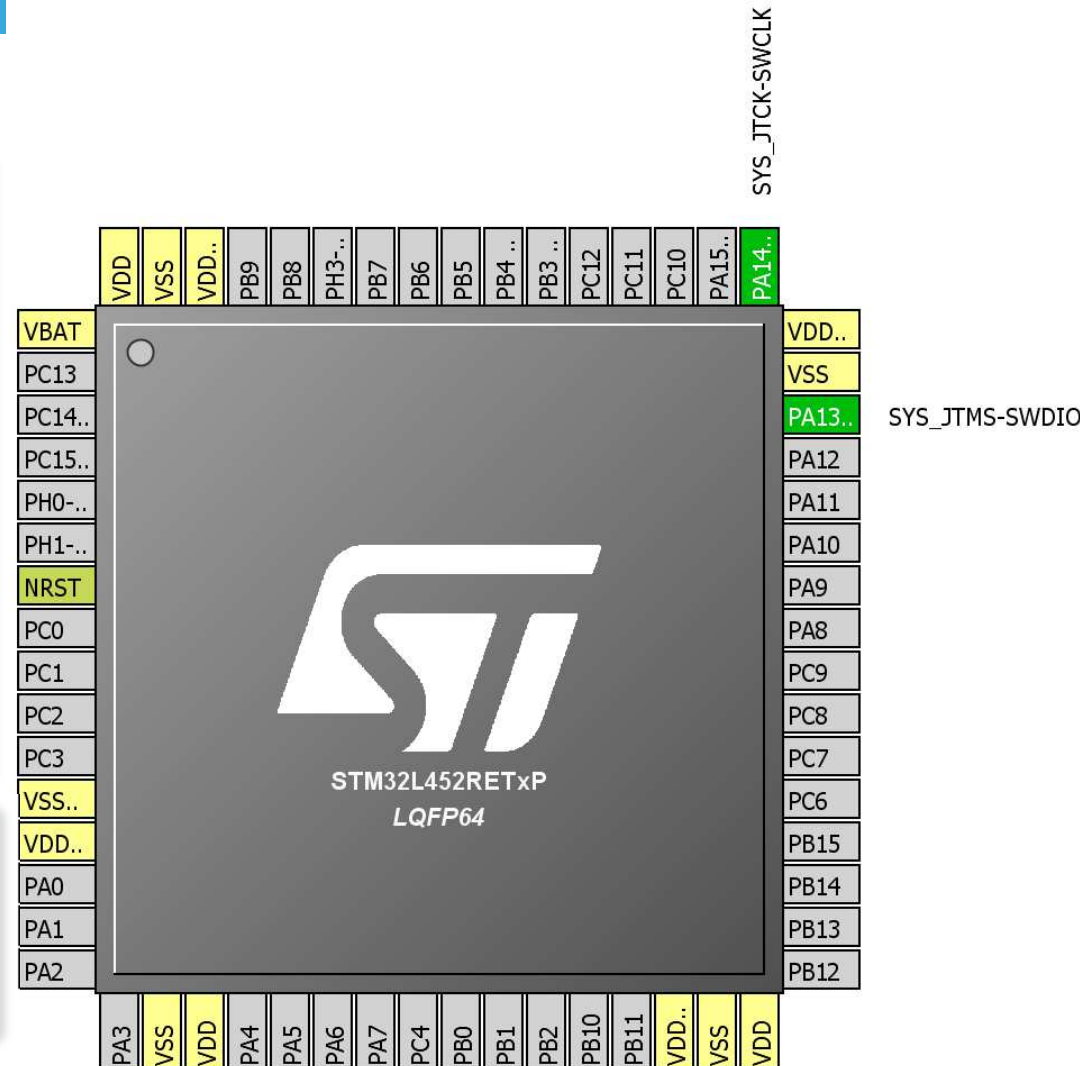
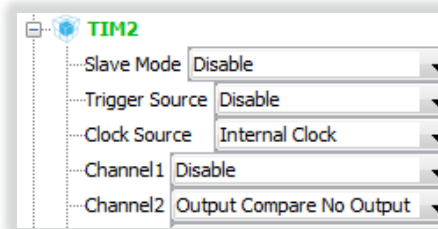
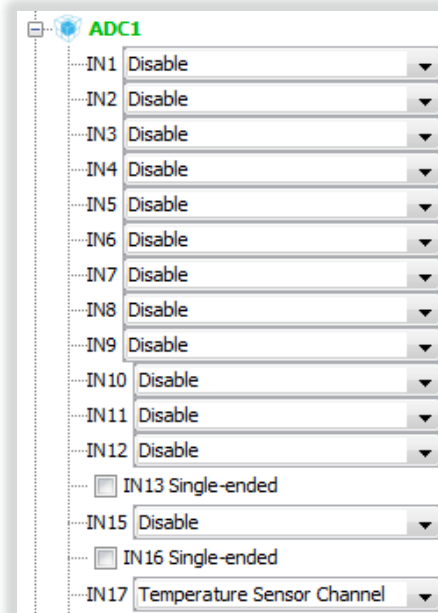
2. MCU's resources

- Enable SWD debug interface
 - Click on **SYS** in Pinout tab
 - Select **Debug Serial Wire**



Configuration of pins and peripherals in STM32CubeMX

- Finally all needed peripherals are enabled:
 - ADC1
 - TIM2
 - SWD

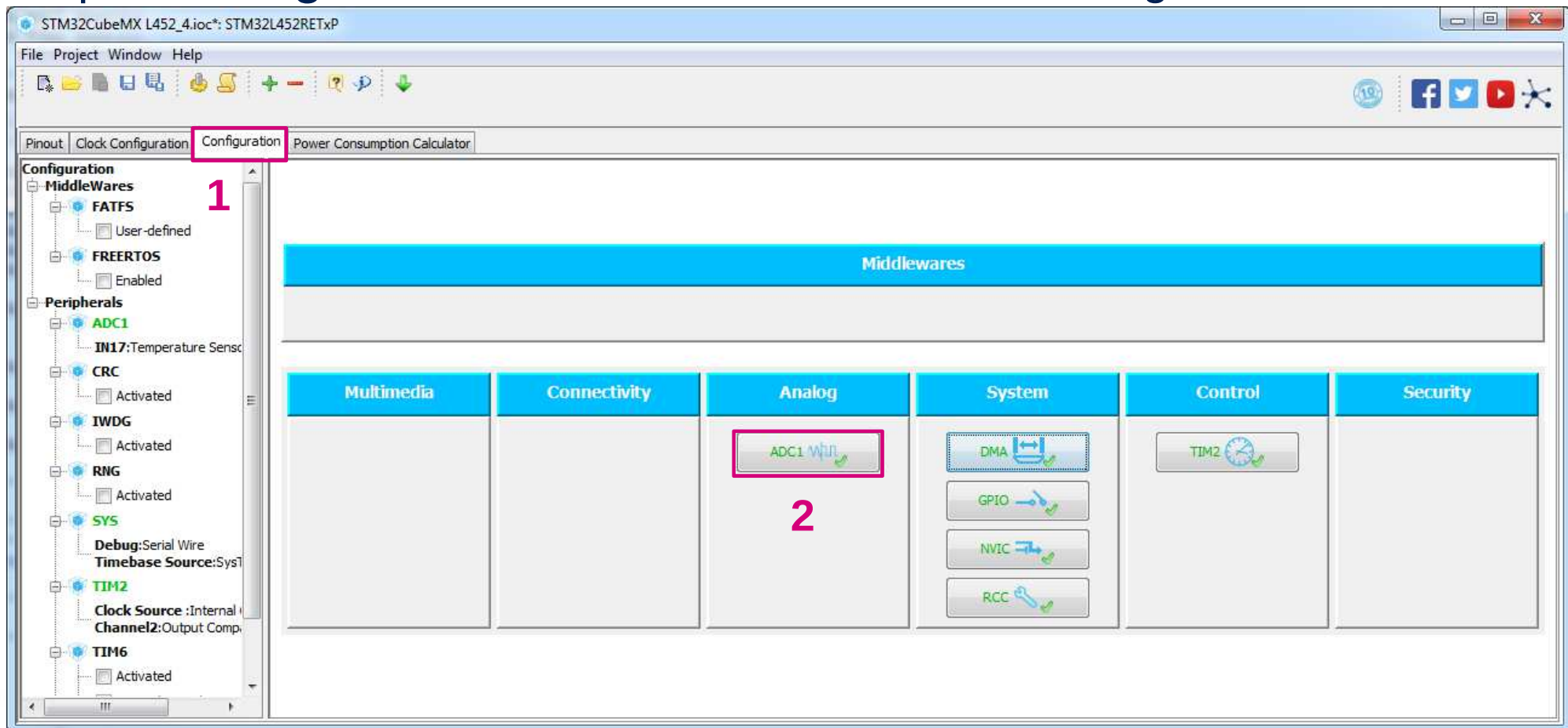


Checkpoint #1
Pinout configuration is
finished



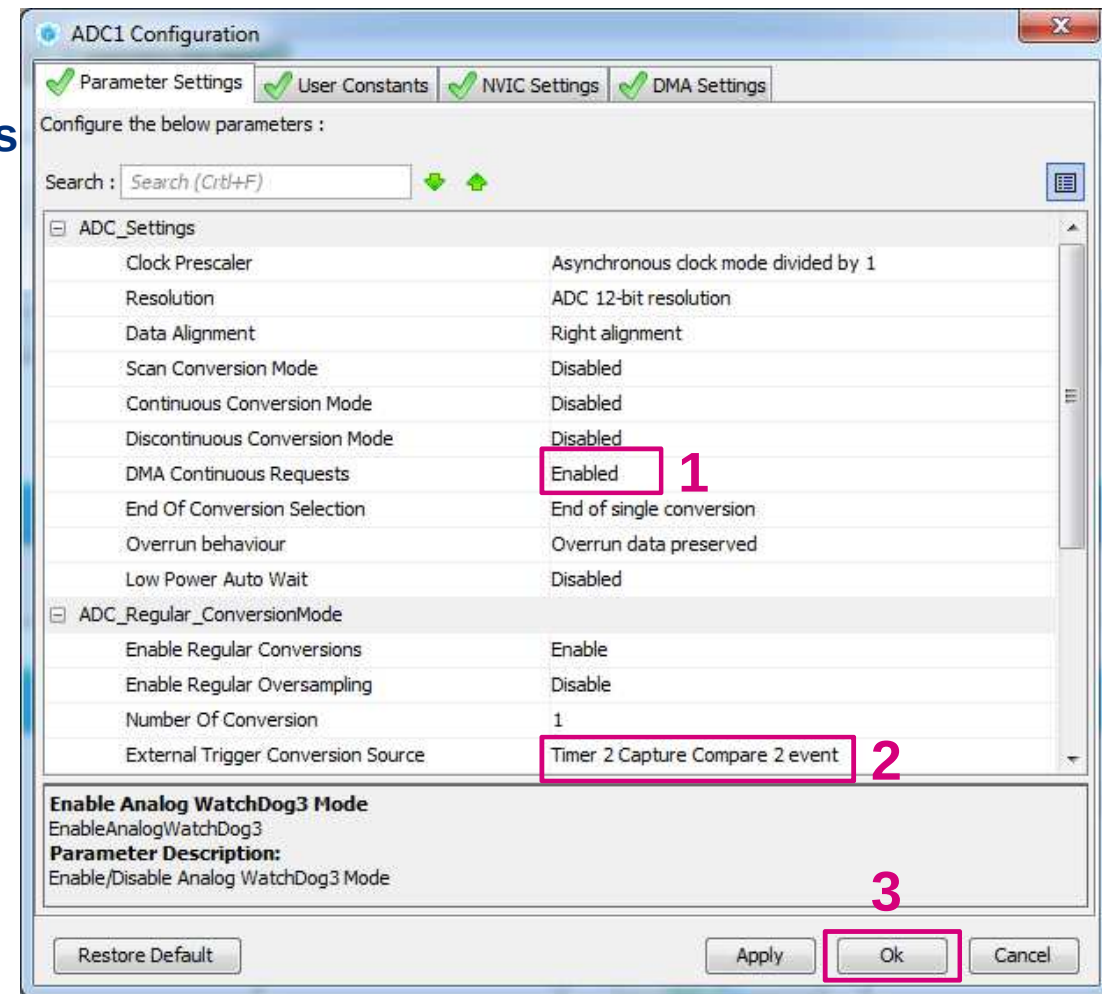
Configuration of pins and peripherals in STM32CubeMX

- Open **Configuration** tab and click on **ADC1** to configure its details



Configuration of pins and peripherals in STM32CubeMX

- In **ADC Configuration** window:
 - Change **DMA Continuous Requests** to **Enabled**
 - Change **External Trigger Conversion Source** to **Timer 2 Capture Compare 2 event**
 - Click **Apply** and **OK**

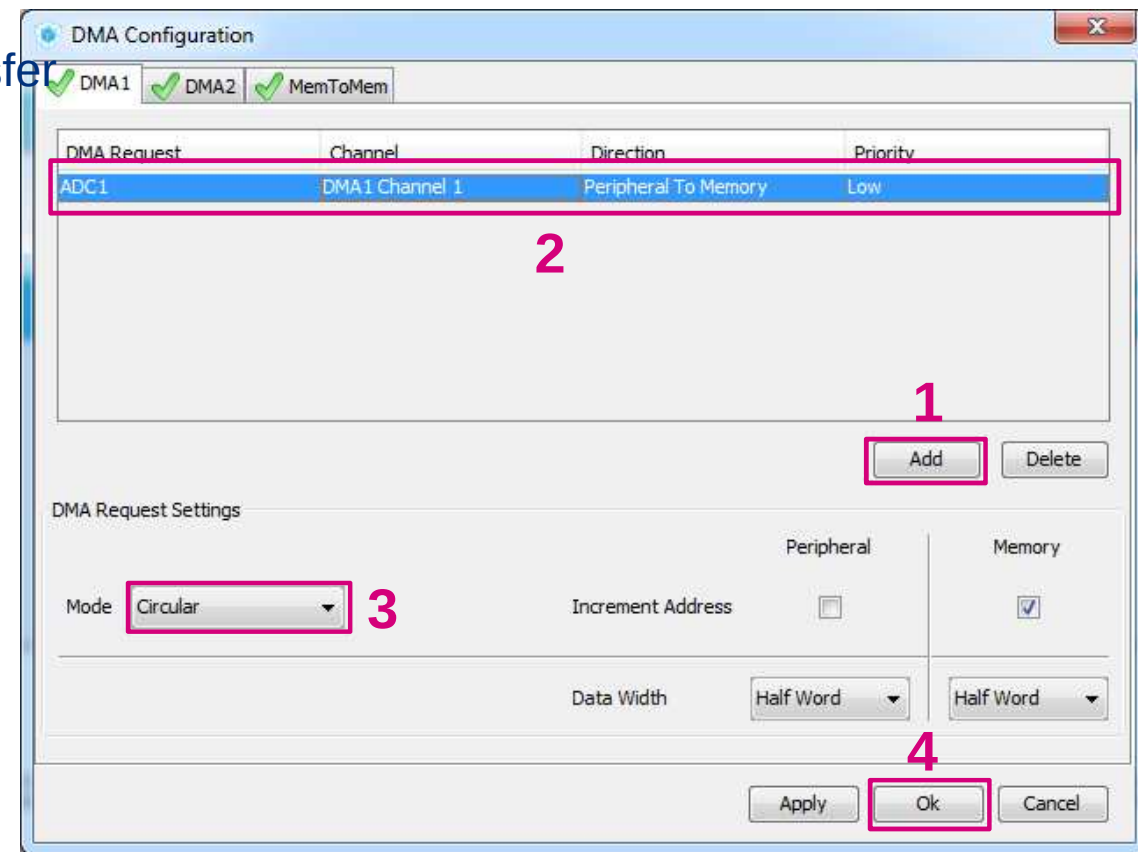


- 



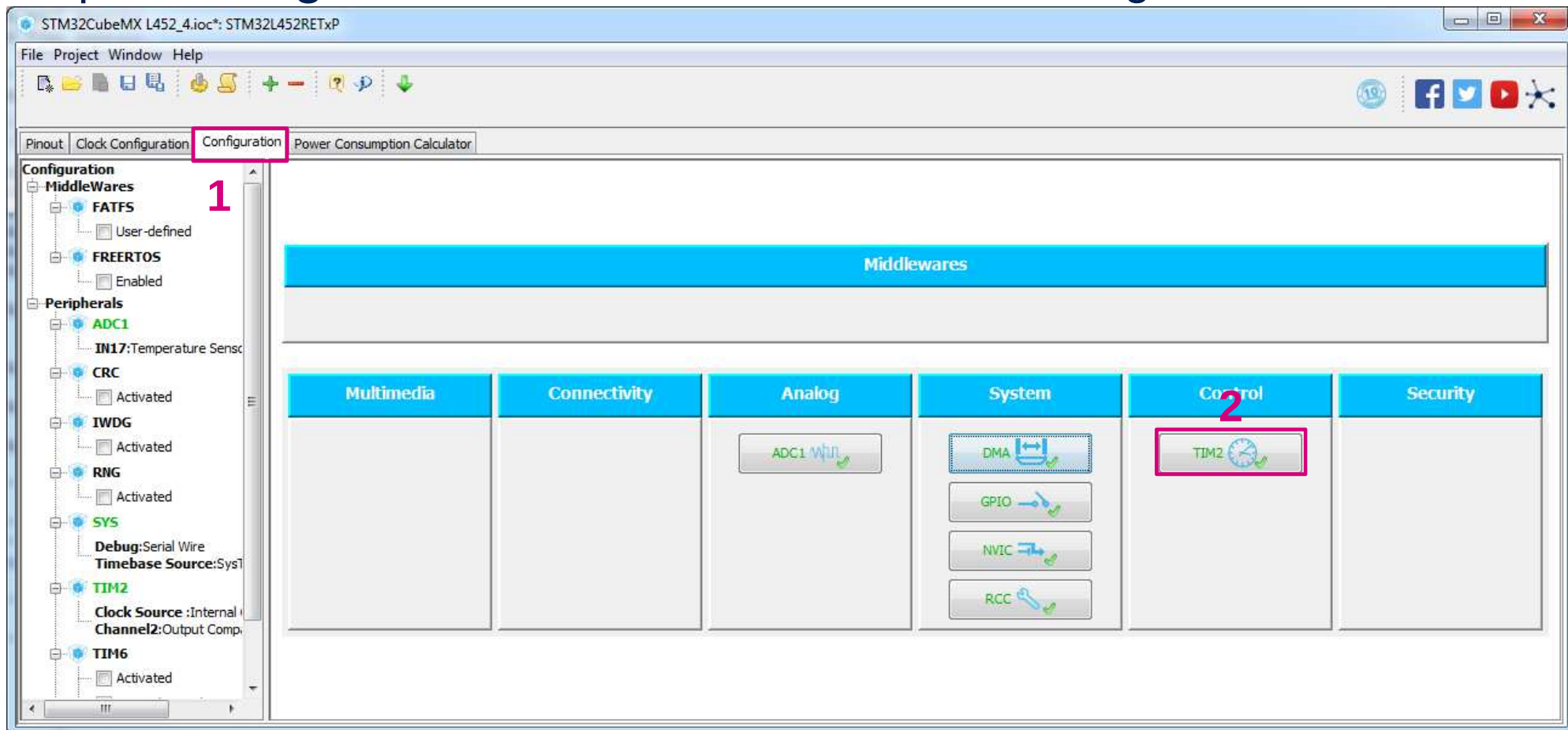
Configuration of pins and peripherals in STM32CubeMX

- In **DMA Configuration** window:
 - Click on **Add** button
 - Configure **DMA Channel 1** to transfer data from ADC1 to memory
 - Select **Mode** as **Circular**
 - Click **Apply** and **OK**



Configuration of pins and peripherals in STM32CubeMX

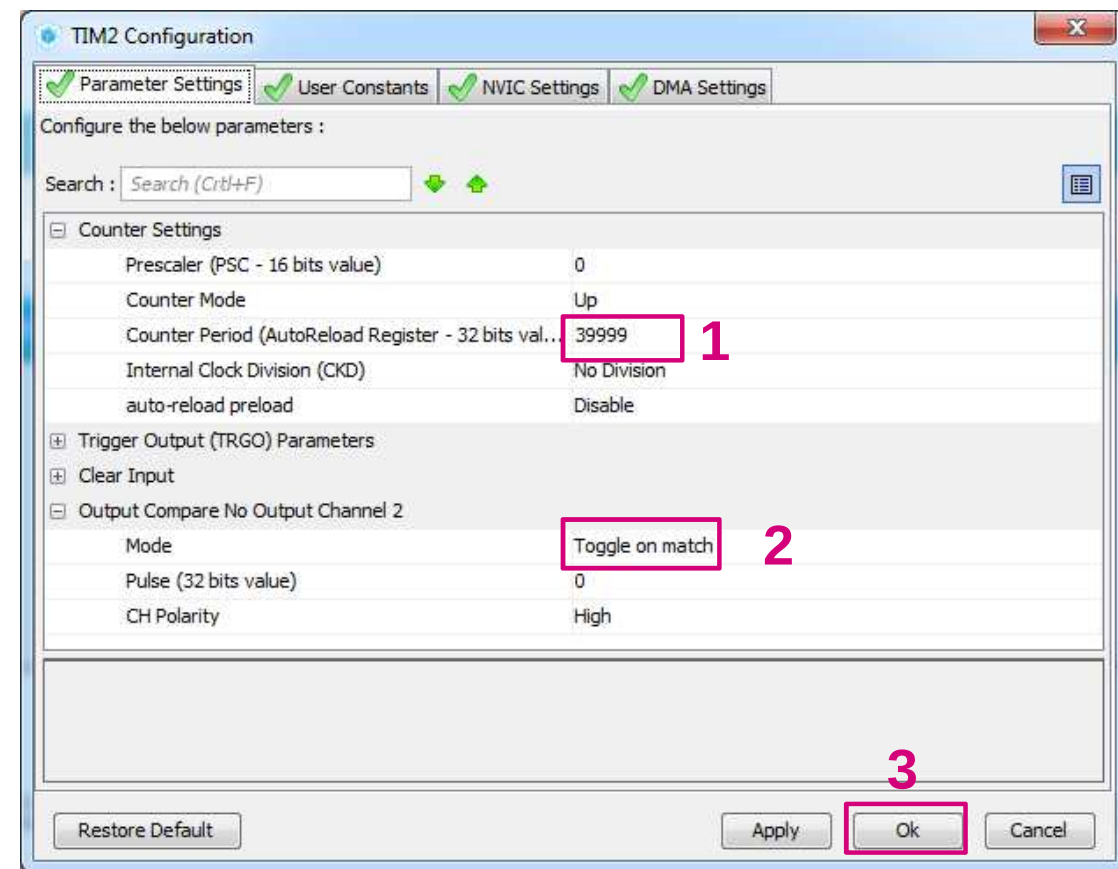
- Open **Configuration** tab and click on **TIM2** to configure its details



Configuration of pins and peripherals in STM32CubeMX

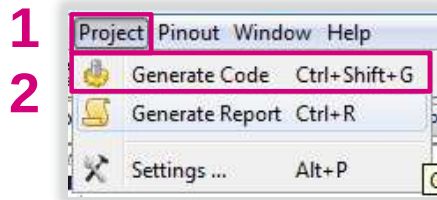
- In **TIM2 Configuration** window:
 - Change **Counter Period** to **39999**
 - Change **Mode** to **Toggle on match**
 - Click **Apply** and **OK**

Checkpoint #2
Peripherals
configuration is finished

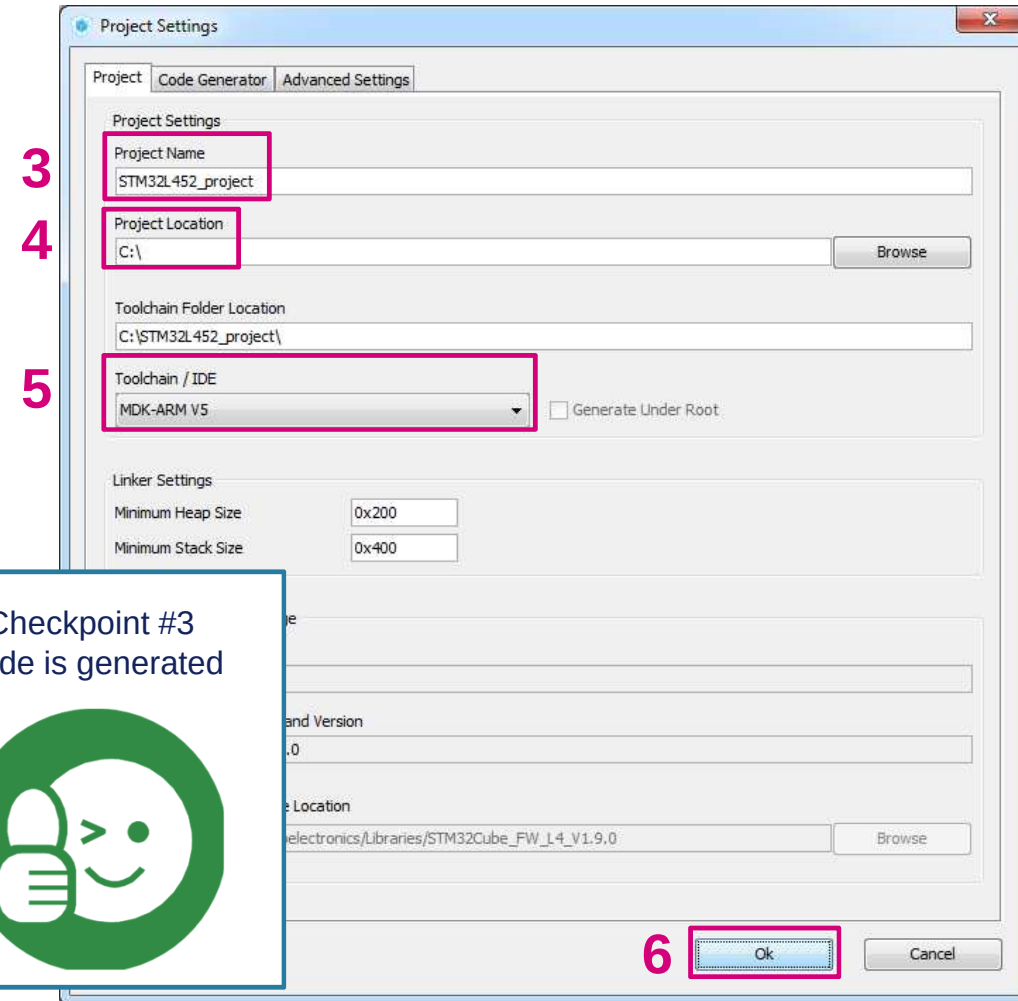


Generating IDE project with code in STM32CubeMX

- From main menu click on **Project -> Generate code**

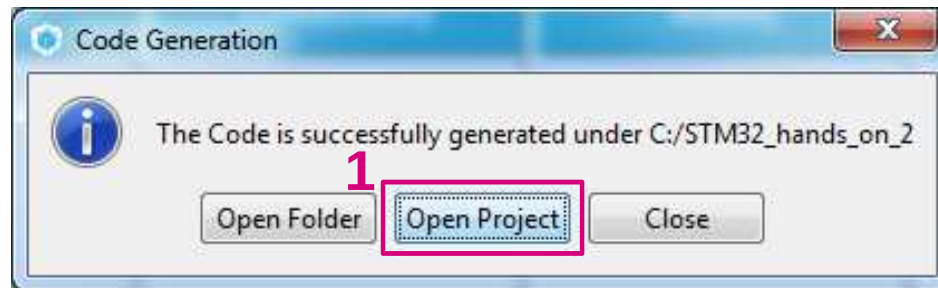


- Fill in **Project Name** and **Project Location** items
- Select preferred toolchain/IDE as **MDK-ARM V5**

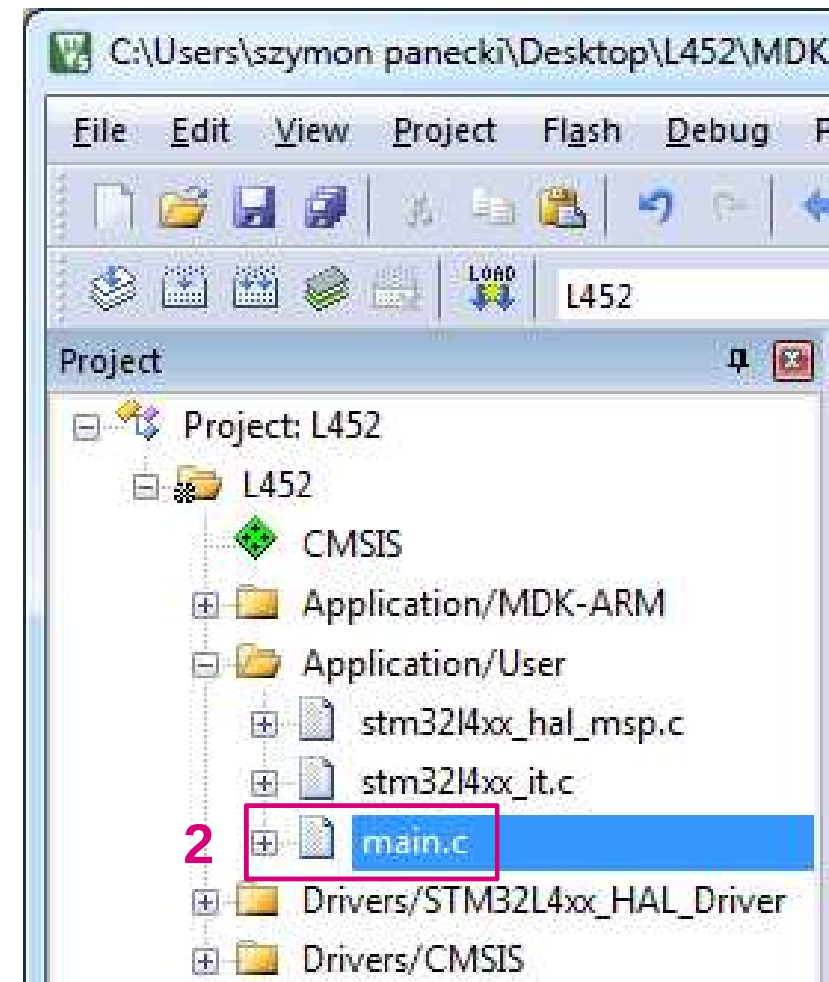


Moving from STM32CubeMX to IDE

- Click on **Open Project** to open it in IDE

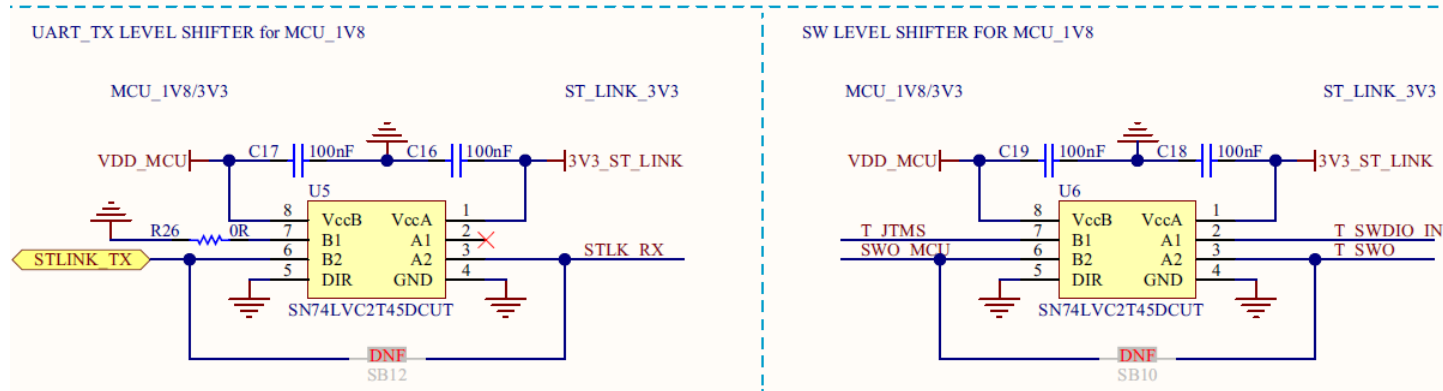


- In IDE open main.c file



User code in main.c

- Code to be added in User Code Section PV:
 1. Definition of array to store ADC (temperature sensor) data in SRAM
- Code to be added in User Code Section 2:
 1. Enable pull-up resistors on PA2 and PB3 to reduce consumption (from level shifters)



2. Disable GPIOs clock except GPIOB (reduce current consumption)
3. Put Flash memory in power down mode (reduce current consumption)
4. Start ADC calibration, start TIM2, Start ADC with DMA support

User code in main.c

- Code to be added in User Code Section PV:

```
/* USER CODE BEGIN PV*/  
int16_t adcbuf[500] = {0};  
/* USER CODE END PV*/
```

- Code to be added in User Code Section 2:

```
/* USER CODE BEGIN 2*/  
HAL_PWREx_EnableGPIOPullUp(PWR_GPIO_A, GPIO_PIN_2);  
HAL_PWREx_EnableGPIOPullUp(PWR_GPIO_B, GPIO_PIN_3);  
HAL_PWREx_EnablePullUpPullDownConfig();  
  
__HAL_RCC_GPIOC_CLK_DISABLE();  
__HAL_RCC_GPIOH_CLK_DISABLE();  
__HAL_RCC_GPIOA_CLK_DISABLE();  
__HAL_RCC_GPIOE_CLK_DISABLE();  
__HAL_RCC_GPIOD_CLK_DISABLE();  
__HAL_RCC_GPIOB_CLK_DISABLE();  
  
__HAL_FLASH_POWER_DOWN_ENABLE();  
HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);  
HAL_TIM_OC_Start(&htim2, TIM_CHANNEL_2);  
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adcbuf, 500);  
/* USER CODE END 2*/
```

User code in main.c

- Code to be added in User Code Section 3:

1. Stop SysTick timer (to not wake up the MCU)
2. Enter SLEEP mode
3. Resume SysTick timer (after MCU is woken up)

```
/* USER CODE BEGIN 3*/
```

```
HAL_SuspendTick();
```

```
HAL_PWR_EnterSLEEPMode(PWR_LOWPOWERREGULATOR_ON, PWR_SLEEPENTRY_WFI);
```

```
HAL_ResumeTick();
```

```
/* USER CODE END 3*/
```

Checkpoint #3
Code development is
finished

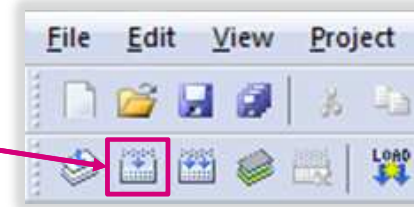


Entering SLEEP mode has to be done in while(1) loop, as DMA wakes up the MCU periodically

Compiling the application, flashing of MCU

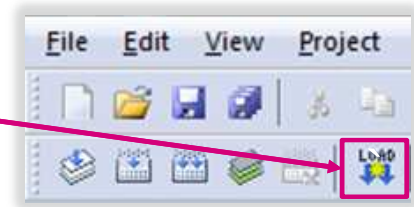
- Compile the code

- By clicking on the **Build** icon
- Or by pressing **F7** keyboard button
- Or by using main menu: **Project -> Build Target**



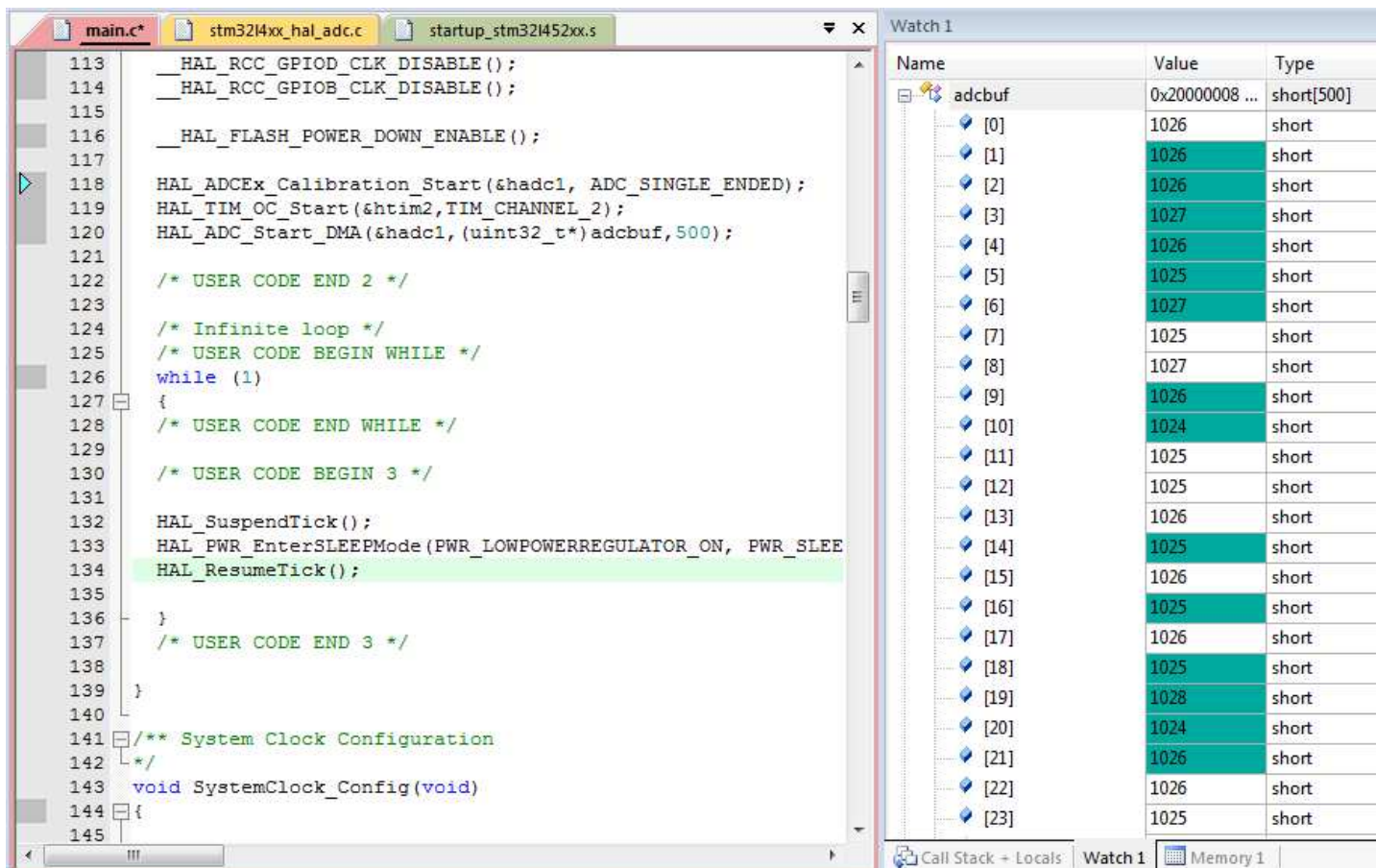
- Flashing

- By clicking **Load** icon
- Or by pressing **F8** keyboard button
- Or by using main menu: **Flash -> Download**



Evaluating of application

- In debug mode watch the data stored in array when MCU is running



The screenshot displays an IDE with a C source file and a Watch window. The source file, `main.c`, contains the following code:

```
113 __HAL_RCC_GPIOC_CLK_DISABLE();
114 __HAL_RCC_GPIOB_CLK_DISABLE();
115
116 __HAL_FLASH_POWER_DOWN_ENABLE();
117
118 HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);
119 HAL_TIM_OC_Start(&htim2, TIM_CHANNEL_2);
120 HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adcbuf, 500);
121
122 /* USER CODE END 2 */
123
124 /* Infinite loop */
125 /* USER CODE BEGIN WHILE */
126 while (1)
127 {
128     /* USER CODE END WHILE */
129
130     /* USER CODE BEGIN 3 */
131
132     HAL_SuspendTick();
133     HAL_PWR_EnterSLEEPMode(PWR_LOWPOWERREGULATOR_ON, PWR_SLEEPMODE_STANDBY);
134     HAL_ResumeTick();
135
136 }
137 /* USER CODE END 3 */
138
139 }
140
141 /** System Clock Configuration
142 */
143 void SystemClock_Config(void)
144 {
145
```

The Watch window, titled "Watch 1", shows the contents of the `adcbuf` array, which is of type `short[500]`. The array elements are indexed from 0 to 23, and their values are displayed in hexadecimal and decimal. The values are as follows:

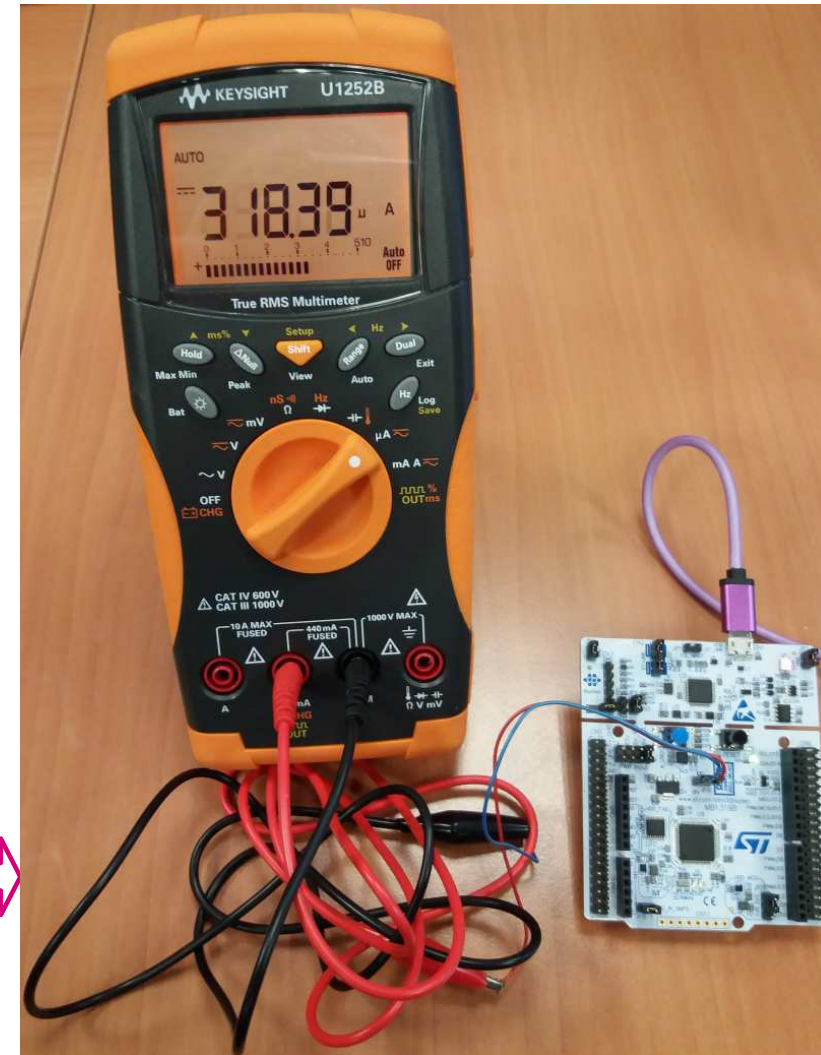
Name	Value	Type
adcbuf	0x20000008 ...	short[500]
[0]	1026	short
[1]	1026	short
[2]	1026	short
[3]	1027	short
[4]	1026	short
[5]	1025	short
[6]	1027	short
[7]	1025	short
[8]	1027	short
[9]	1026	short
[10]	1024	short
[11]	1025	short
[12]	1025	short
[13]	1026	short
[14]	1025	short
[15]	1026	short
[16]	1025	short
[17]	1026	short
[18]	1025	short
[19]	1028	short
[20]	1024	short
[21]	1026	short
[22]	1026	short
[23]	1025	short

Evaluating of application

- Measure the current consumption on JP6



JP6 consumption:
318uA in SLEEP mode



Thank you



life.augmented